<!DOCTYPE html>
<html lang="es-AR">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
<title>Calculadora de taller — corte, materiales y cuentas</title>
<meta name="description" content="Optimizador de planos de corte y costeo de tableros para carpintería.">
<meta name="author" content="Acero y Escamas">
<meta name="theme-color" content="#7a9981">
<link rel="icon" type="image/png"
href="
AAAAwFBMVEV6mYF6mYB5mYF6mIF6mIB5mIF5mIB5mH94mIB4l393lX51k3xzkHpxjnltinZph
HFjfWxgeWlddWZZcGNUal5QZVpMYVdJXFRFWFBBU00+T0o6Skc1RUIyQUAwPj4uPDwtPD
wtOzwsOzsuOjwtOjwtOjstOjosOjwsOjssOjosOjkrOjotOTosOTssOTotOTksOTksODkrOTsrOTorO
TkrOTgrODorODkrODgqOTkqODkqODgsNzgrNzkrNzgqNzg7T8KnAAAiUUlEQVR42u19C3vbtrI
tjw/DHT34EN9viorIAhUFkQeoQYm35P//V2eGkhyn7b63vltuku8zEseWQooYYGbWmsEAVpSP9
tE+2kf7aB/to320j/bRPtpH+2gf7aN9tI/20R7RVFVTFA2+FE23HNc3fq7uayrKAN1fb/w4qydS6EvT
dv0gCIPAcx3LXP3Qow89153QUAw/Y0cm+33pK1HJK1ISQqpdRWTlqQv1h+w7Dr+CPS/3urslI5
eiF4KZBmsprWvKW1rxkuaoUk/aDyeDNg+/7uesKz276KpWSib5c6IEZC/GvpNCjvJM2WbtWuvZ
VH6w/qsLRbEz0jd9aSWMdwfZ8ZESW98KSuUgu64THSWeEhYyC6zlzVx+FMejmJHx5LPpJOg
xtLiADkvJBUvVoKL9MI5jL+SBlqFi85rKSiau/uPMAqiP+0Ws7ILScRTlxisrDgpEqSC2UYh+GBgb
JtAh6L+1ZSBd3TCWeev51h+h/8uQyXQRlqeRUZnpCZPQxjN/zlZxIRuQZpSDmEpPsXLJ+15K
Wp8mlrmrH6P/q6ja8WSZXKiUTemb2+cJ9WRqSkfx4oIR2aANp4ZicVZKOkjRiKE9SZJsvrcWqQ
C5q5g08hiust9haM/cNKMk52RsW8aCzVpZ2fFERD92RWiu3fzI+TSKoevlyMupcL+vFuHwLSNC
ZUNcQ/x+Hs79JbCN5crys+PEW8aL2FmBhfNL3U9MBqt1SMQAVgESiVqS0Pikftf+ryxQfXqWLb
GsppbnEYyXFbFnKksnPUrejkeS+7qySY90oC3JNooL4kqYq7I5bl0Ete+qP1HqgP6ch/FsuPtdBY
5TNu3UlTxxnpZePfGB00sF1rsKnifZ0wsAwabuxCA6IoFyWJ7+/fAAkNfLE5dQAFqercKcgPdshJ
SC14KR1Aafc6FCCgrWaip2IfuRH6pQs3aiOfDAUsywqWBOvpcha4rDq3gjaStpFSum7fhJvUfK
MIgRIKEy9e3UCjGOQ1Nmq3UuxTAcKIkUmwlBXCWSrKGSe0/fx5A/K3bd/BJasjmMkiWObeqqo
nspQMDQSWBuCaBwg4gA8jFbCRnA3HRut2WgeKQVlWNVtZD0QMLVd5AA9MeuRF36FmtbA
VynrIhIQ2ux8hgAFwABsQwqBXA6ySkMt8NE317gq8dXMakFMTcM7KdrSLz+xyVQPyk2aHddO
psSRril4lQjfSgixa1qOYo9CZSIUTDXsa+Yr1hfKhSzPwMrorWpZ7Jl+cqrqvM4Ncf4n54DZD/yt3bk
e8vOC0ml/H3WlYa5aoozUIt0uQGPD8JRefQUMx9F3w9yOA+ylYWlBKyVLFoAbwUXVpNo+Y9
aMqBvUB4E66cMkHZtbPwEnE8vRZevNuwMzoZX5joD+x1EfagcpEAUCREwimZPMktxtqKXvL
RN1vZDD/oX/IPeFJi/kcQha8/tIleh79qGsog4l2O785SYceg/cZSIQxgwXlhsKm4hIT4DkOt7CuOu
w/SBlcv2CM73wBHV6uqf4xTwHDvPN3a5ozW/QKRbEeIrCQOORou1xfjQ/0o8zWenUf4qE2cJR
GJCjO6Bg05V7ijLgHFwrmzilWvsIEyT4HVr8x/CZCDPPp86wwXc2vOhbhtwOI4BDLrj4F5SJpiQL
ig9kae+keECQrCmmcRBSpA2D3TFSsmhHeXU9XzKliGhZ8EPdZcs/5E5+KwY8S9A+le27/lhKrs
Wun7Q3ZJCKJAqTkXBhp/DzWrlbXkjBQMMdhlYgJBFCm8D+k4S3NRFQMjWD8TaVGAdBBSw
CJ6e/gnvb2XHuuYx2O5qqfz3Zss66OQqeW6E4BuDzo5fEpbYipmwpjvxL46yKRALIB4wnPiyP9
SyFS0QUimrwl/lUqZJEkehv1TU9++/s4NQt5FpnJVFljh6LXteeCbrJSAq9Bl6egZ4bRmJVki0x1Ig
CuzgMhkt7ZlDZLS9HFoxNU1zqtJFLGWkfV4ulHfvPpIujwg+gr+UbBqnMQ/tigKPMMOqHnuWBa

ZiJcBzwFv2OOLLGKCWcQiFAXN5UwELaht5mEZAaFAcRp4LoBUVZ6aVpuG7w7GqagFQTzlC
UFtKYDP1F9cr2p6E5nMpOTgTIoAhe8CoxbmXp/9J13oG7n6ofwnADk59Dd7KZb+Ky0CfORd54
tsrQBReVr4B6vTehELV/hWQ5nAGTtnK2I0wct8kBLw9eMqgPVLKa0a2oPH5pR37bgSOA0Ewh
L/AOID98BaA2gGcaGseboD6gdJA5JZezlxmi4gVu+R9JQD9KcHXQyw7Tq1MTIdC1PIswBTr2A
bfhHEvRCyVj7gLLPT/zP32GOi8oFvdBPOm4othVJT/nukrx3FDjJ0vHNgG8DoC/4bqO8Ix2C+rg
O0A0QQHMrBs6RYBeH3EUY4Biwc+EwEWJLAZxjWyLnwFwoWh72uG1K4RFYsVf1efdsEqLRr
Cp7YCneyb0tMJUMP3hOMnxYTxPwsiauDF4+VUBXqdX+S1/cpAOVwgBWO3b0LFO/JxoBQIjp
mByAfalMBSSzp2nNngN5u+0S22g84DCPcQX8pkCVAuqsJ4LzhWtWUCSixkoOuehO4B+ofgG
7uJdbIDYi0BhX3ot5DpyqSi7/gIeGxlvBennklfNWG2RmCiKXiypgHZUtbUmP8CxgoXGT74hxOLV
O29JgBoPjDJULHdVUQxrQDcrEOGKTrotaQshUugJztzmUo6CpIC5/wCkDWSKTIVp5jT0704W
kYtfmP5GkBbALke4RMGSiy7PAGZkpv3USJN1fOuFZet4Re5Z5dUnuV4GEeUAEjyKEdObL1glI
IqhQRiNBquVyFSCJ75pmJGDAx/GKTYVSE4JKBPLqgSqD/4rhOtITCyCO8Ef44X7xIYaArM8IXz
ZJWycuOWvMeE+ThcxvGCUyFOoOQRg46Ap/8FYhSZLfUUXOyYbdZLK+rZjgvR8mZix1h19xR
5E6BfK6uyrIDP5p4hz6I/t8x6jyl4UowtxB0ggOKk3ioFBWLwPEBUiLEQIptjugCxMP3s8IO4CAgI1I
tMvxVrIyMAztMkn0l1zMPNZxgMCAykudk3MnE9z/M9d7PQwZBEvQM+8ngB1CdMKQA96Pe2q
qPHR23ArP9shHxsSaYDqRAsWHoMAkfwoCxVQ+ClTQlMG/hEnuVZGvvARhUrhrmDKNLVwSt
nqysDWpk+mggBTFs/nhMBBQV/D0EtBLtJ0gD17eRwELJB7glCUJ6aLqE9uB07P4CuY4ZRGu
A3z8jwkucTCdf6agWhhOklGLqB5yIRBEH7wjWzFFo+sl6w2IvzvfNwAVQNlKY9g6aDSjAwzGEY
2oMYWXUkhAPZYZkTluhjU3sBtgKKNUq+s53yBLqWLUJSjwB7cRwn+Uj4YZ4+yVM1ICVm5cu
pQwAcAYmV5dp5vBHMFgxPvaC7ByEgMgQiOWVx4GNIc6EyzwmAxAgyukpATri8NydcINiZKt0
rT5RZNsQEbBqaGvwOXCyr7dIt6/FgOKRsxQFjfICQ0Hm8BS9AgZp+GgZcWYGhAoLM8ui61Kg
oy03Kfpsu7XA+d2PbEGRrwDUAnAy/LOuaMtMpK1q5q2KgAryX3IHb4WUau6oFrLqaTQEILjRZ
60GxgQc+2gNlQB7A3eA6naRTKWPHuKGDBsBvZIDElysiVQ2z1riKBDTC4Ntqd5CFaRfNHk0B
AFz2LNhAMw3spQk8jodaCh52GNAZS9Mt8gezCVWFqIQi55Ry6OtpX4bW5zk2UO85Fn9/GqE
DHa6Dgb9ZbuB6QNoNaFgQxZlp52XJYxXgS8oThASmqeuGablBBjPAEzUkcqqBEQ2Cm04DV
vGkPtoDzSsqF9GMVT7XbrwmLKqGCXYYwL6l7VTVnrqpDg1wNssykfMv5ziTpVrAIJYHCq4F+2
1xunAG/BNoXLbwiyInsuaCM2tTFuSxlFRVnGrLYJLPoPuAl1iIon57AdIwilNEQD596XUS4GiRHB
kEk2nkwi1WVOX/5ZN6GA4yX9hVDZy8aXgLkTHovVumpl+SS99lE1gKz41HMjpV0TPWQAT4K
6sD41ZH862Kpc98ZPz3Og3t9dpJiWQ00fx9DXo3XQhpIktRnGzpgYMd+wvXzY4P4J84bQFIzs+
GS7ir6F7GmdRdAmb/WDDWFDNt5DOTofnn7s+rBOAUC991bNNywoJIIMnpGgKsHpAb3Hstjs9
wq7nywb92Pei5Li89jVJCiJQXVq+8PeexhWAcLQLWnAW1lMdawQpAKPjL7oPL09OLGOsZTW
VFpppWLNON7dXskW5TgIPcWyjuXlTA11AAsIlQt7wozbIcQrSiFawITLSuCGgTJfHi02MlmJcS1
ae/+i+IxpHEADbgSgbyC5IaqxRCfQwX+nES50oe2Jygq+SvVwFGBGIHXCkuM4N7El3NE6tLPB
sck+Rc2I9FY2TofzH6ivpZWUWkAQDuW9H1ECOAYpeR/gSOdx7+bqQNS1zTDIUsHEyKQuBjG
iCXbBlhaei7nh8jRxEs0nN4C8KjNClJrj+aEKl/PTNmQnDkGTC9ESLGnsrJW2LwhoFaP0wDlfBa
UdY5Z9xXVr5knWGB/wTrQLdV7quKHXBdswrW+dTyFoBhpW9C9+ndE73g6tD86n7WlPF8hnih
xzhSVcOynhczsGMuCLrQ/ivmEKzFBnDpZumUFElJnvOypnVLz9MZaLdrgvagcu0NsIbVu/cfHrA
BjeCSgS8Hfg9QR1ntrZR/aQogm0RGKhG2cCgBK8D3c5aDONbCA0AAb5R7ABVdA2rXNbRlJg
REcM9Ij5ZTOu++XgPD73MJrr5vGwH0fhjpRGJzLt1SNgxYD2jVQNnVGgEN2R56OjV4iV/uAIHb
Ids4SV2W5AhaSNJVAoYPYR/dOSYh3juvfqvKOoboqQf6IkF3DmPbkczRlH+pcwYJvCVQO3BP
YIxXASwJfGqESLPyNCOTNVgLPaSf19bGDaIkSdLNpuLAXfvx19LXGa+8980yqqu43IHR4vyDv
QrKZaDf6q80Tc8lcOsBmFp0tX6QqRn6cy9R0Eg302MNvpdW4cr3LQP0/bOiJrxhFT83+yrUi2nL
nPfMtj8pwS8ln3CJZZJUVl2ZWC9FfAvFpIce2TGuaF8FUM1qHMYJ80K0SjUjgUgOM5KBnuRNh

qsbseTeJkgvJauiVSYPorTebw6QobYsSuRvoMuHAYYfjPfu9z4rRspnAcCIbwt3ENNhJIy4PLYsX
CjriLTDZaBVpAclQBeEpk2FFMhwwjxcZVzyS6q/26qfpkWShFYu22kaZMNT66V6T0Pq9CzAiUL
sfIhmS1SRL4m+O0M80LVANHChMJK4an8iCYQLvDzVu17mZhxYK1y7yiYOHxupT+o7TYBRP
KfLAGL2HJgqS3Tls3rHBsXKwLfKQU6iq+YJwGVZjuEW2kBPf8fk4WfFvIjzMJxP/ItpJFXd1CMt7
TBnRQzAXeAiEDrdp3dyQQ7jlnH4IedWwgWM6OcXuDYCPmICAJyoADvU0BsC/gK1mxcl+xPz
USKs1GkB//oGU41wgTwdaOk7RS2rUuYIaLblJu+0dKwpQR4hNJXWqpi+pmOhq17OKAQ/WP
Qk5Nx/xDuK4Q4C87hj8QJsfa50aUVz7jlnMVwDLAmQOrLJQcwZd0wrGcZCfx9PpCphZampxD
AkK+N7FgGxgdOiFhCn0PrKKCHcCSDgxVRKSxtOybyK9AT9r0+0qbEqnCUaGMk6YIdLvMF0H
5urvvbroPTeyZOqSpwvltD1f2lqmNv3p2ifo3ys5soDAKL8OjFqABwT3qEEvrEjsCEN/mwqBvEmk
E94s8TsllzqfGGxBRN3xjV/ytJlxOh75doVN1Kgt5jB0SPj9hBVWXr+tQVhBPHPzIEWXhCE9xbYy
6sBuFEY3VoYeehrcAW9DE2YgaGfk/XgS6V8p1w7YJWhKkZk4Kern/4vYdDfffcWm1o25iWFBM
5ROiannFnvuvit/TFOeFI1CLFU+NJe4p/5Z+3p6RN+u4/n9Yp7+/pxDuG9EOTCDwwzjhjev5MVa
C/E4YEP0NRlwveiDjNSxApEdYBta+WH3K7yb72DxWoSmKYd2+sadKnl1o9RoP+3SaIa7QorT
wwFgiKINWjl/VQzgFOQO86Rl/Y6uzSX6X0WnN43TnKMDKseUym6LJcsVn6yBihPGuCpcqCNb
cRFoj1Ih2b/+PQnZ3Rzkuqri17a7GXxndmz/pEZq5+u9/4hj6tqPtti+WDbpStFCdPPjxFA/Yvs0OtH/
x1FVV+jqvbXn4NMl5yEgNi+dJcQ5tmPKUNTFXNjWRvzDzC2slzgCb5jqsiaAZzxImsz/8Wrtfmda
zPXrwYARTEdP8SbjW82IIEZc1ywaedqGPagFCPwZEwBscL4Shye5kJ1oG3sSPpkc9tNcOH8y
DsOjW4TBd+RZ1yYkfKQRZu7BJhTijmwvgPwvG1kvppaCP8lkO+Jls4yb3m6eNAEbMie8/Ieo0P
7BDS/utC2qWveCv4/4QqYnVdtxQHXIOFvXcUoQFH3XFDe02FiLFzfdlyuQwoD0sDdbS2J9L92
EyYNqxj321TxScOTRwkQ0qobW5neNXIBEdXl1ErKO/g6NE0VrVTFnYtG64YCz2/oLACpMT3R
yrYRYk/i5RzNrJMdSN1iAh7DGl5+Lf3GyWaFbiSWwQ6PEgBjYCxlgjD1huzAfWnHYa6PrMPEOh
W0hMDe21VCjvPOW8LyWYVwVUY+jyMuZgxl4aGnWkYVHXtxlFmKOV8xnEiofY3q0q3t2Noi7g
5N9xgvqmKyE2JceV/8UZF1gbcWMrKBuISNvDBnAd2tqOCpe2sYjXhlLXhsWnCRAJ1hGQZkbllj
4jG09JVhxwxiSHq0X6KKVeZYR2LbJejdc/wYBcIq/25CTbimCiGkJZwJBiHN3My02CjzeLc1iV7fC
SrEWTC/cHAPltsoqp7LdgSYuk2v+9z2Ap3+ra8ry97Kbpcx1LnwEVG9qlrw6DFLwbVUsxmrWow
VNNAbALcn3EZsgFXgDNQNiRfLxdy0a51jQwLtMyDWIgVrIY6CFaQwl67yeY4cVMVnvWyPmxc
PpV+kOEt5AeMnD+FyT0pAmqbw7V3TsmSBeKTno6xQn9TbFdf8D2h8y76dgbIGfgz/qz5pCciP
mRaIdblMPt/u1dRViknI+1irqs3qoR8w3XpmmwcIAAQrg1ibWOutlLSxMPlmtfCjtF4wdP4B1wWan
t1swLNQPK9sW4bVoDCwOfbTwU+TWFKhvlpn5r/Ju7+BlyU9iB7GKu/qR5QdILjXFCEIlpyCQoM
ANqPddEudv3JWHi5syD34oLLJQ7zQKxtQIfwUI5b9b6K1Zj/fs1fCw5C3w3Src1KVzzHbtWJfRYu
geIQXBY2J+byPSrEbPozQbQCBSv4ZJbG7CLq0prTeboOrAIADWRRF8Z6LriXQI2MLodZkvgg
P5IH09D4cT4pRArTV+3hhmA8xAU0x69+Hc2OqqpHzWQkW4IRoP311HC8CVPPmgctlnOT+K
sCeivFCKsYmKtsLAbDQ899FL78RgDXt5bYuqapumThhHn3eZHr8iIBSRd/Cj7NpYu03JtJwvaUff
zPVP6rQvhnGMs3SLMty76ZCgH+nU9XCN761Z4u6iIbZ6ldi54Ch3IvngDaaa90JFLsmG2P1GB
4nUfWtjb0JwPfLA4yiUU79vnJexmfOl8zEoeXxaj235YyA+1YiC8JaIgasDT1nwman85X9RCU8IL
6ybXXhBmmhK55suP0QEAAe14JGSCAHDCtvKdrkEjwfEpVr0HI9R+UqQP3VjapXHJAyTxKgIv
wK4+psKZcv5i2lulDMUsr2xhM1xSmIFG5UcZmuNe0RPCgkfOz6tjnRrgU8mxfuEHM70fnKbQXf
Bp75hALsKYm/Vb8WRxu5yMBndg82dRAjAMH6dozMOpF8kHvjCvHrjDLxG5D0mj1kApDHTbg
OPE0Sv85jh8XFqlHgEhcPsXJsafiyTHFEYQYAiW8qtF7NVIIi4KmLZD5kYq1iajqs6CB4Ys0TZyV
cjqcqmKsxcLjoGcKxE4Kw9hAe5MJknmWeYcvzudvJAgkZpWfByiQMkz0ZGllY2F1wo/vs1qKrA6
gPobpQNqwV9REXN57Am0k6cTnFvusnl44LrM9Rr1nfl6ic6EtZ+crTQ3go0E6OtTFrHFdd97C6r
AT3o4V7TPpLDqbxe8PrY2LMXAi3f07gSKeJJOpMJVoOuv+kRlhI9wWXXADIt/LUtz2DmAzDRy
Cpt7WozeHAzy04vcJ5zEI3PKuCmOu4ub9h7CndlTiQi4DJtm1p3XBKZRkjvrllAS9OrGGnercFvw

LvlPvdjMlmuatLFqO/0q5lKxCO/VJD7CPLxLwagJmV5ekkeRcaD8pnwcDljBTZ6pougX9jeF3WM
2o6KSMS7IIxmnur2etjwQw5whf4rGKegZzxPJpt1y844cWs2U94Cg7B/JVkFck8DEex/3kBd+LJM
Y87skRzfd/zXzghPGRewTDmHMnKDpIsz5JwzizgwPqeNxejzxc5t3d8hC9VXXv4Ue4tolMMN4S
b0ySwbyesgAKFge/PBbUP38ah/rvlgcVaXy+Ut5+0M/dwudZXmvLnM24+PbTkD1Xn0x9e3554/4
+X7NqrtNwtM6fdf5hZIbSnV6m+m5aqX6V6UlX1nz1lAp+nfpebP9rfiiTUF935GI5/Ps+/AodqoBUv
Xc8zf66Fltm7LMK8iFTgqG5eFvrPp0QzM+aWgoHoQ7Ij30GJHCKTpRIy/m4bJN9XAG0Ry71jUp
Eb3/MUp/8on0FllkriKT/jBMwSBATYarrSfllY0NR1Jk/kp7Tgl4CU8VjVftb+wxQ4DJOpPzEc/+QCY
Jqq+LHKNf6SWP5btvmkWGHo/KcC/H+R2advYiv15ZMU5Y+7aZ7unPnTnyO51yvw38Zjfwzu/vq
Ne9b1m7jtP1EICGNXK/V1zD13Ed5cKn99fKuqfVL/TXCt/v1nLm8HEr+hbTzgwPf1Us+4ZmV1N
0rTNPHNl0+DeN68vonHQ75+hg13e+59BQHItevNy06vagqM+bNfiLamWB5eZHydJ/XlmcFm8T
YJooLtCCG/4F+Gta14Ll5WHS+yI5iou3YEyEJECZdMYgFrsL5THk0x6oowljsvS49GRwgrc/0+ee
rTli4OBAu87xmCZVrg+vhXr4WJsZywy0VyxnGDwhtaSJuG01ND9+W+LDe4Cc47XmhzYqeaCp
Ka6rzXxCIIW7e0rCmnskrNl9Vql5R1s8XFkJukQKpbvpsXLO8Oiuxovdu9yKhaR3hovcvuR9yomu
ZXHa/rE2solcx9y0bpkHMhJykvcpqm0gZd2RDeHnDjl6S0LoI5keUeu9MgWtxnL6ioWXbLCKqLh
HLcvU6tFwGKUZyFrG4yAstIpTj3dfV1koJfsOy9flnAhPGp9mKcqrLilO54tn6LAFLIocixZVm2URa
L5MjHjgaun3QCT3eEOcGNzyMoUBbHwDmbEx7gd50Akw59J0dK/BedKqZOtPxavH7fpj/2TePct
XGdcazRx2r223rrOmvocCl814vliAngN81Ay4m31vW1vl7r4AX0YuzODMsDFk4eYFm6ukrZQbY
8c9ZYHxAx8qJCuPIrcXeAvOkDCnDB1/d9WYpZth1cQ+/jjeHPvEIo7vtW1PksCEoxsaxussRQPr
1FACb2e/cVIpnlmU75SsWzvHR1Xj5ySoqnsOk3l+hFtz0v88q4HObeEPu2cmTkE26r4bjIoSra55j
wSXZte1chdTHvWBd93zD/tty3kW0Lz1TwmcYbD38KuWiJpy1uOULowHbse4apWfV6HJmqxqQ
RDKKtxQsy3E3YBvV4TkLWNCz+7xcBhqnvJ9QqUCIbj6ADGZubAODPWC9l7l/g3+v6LbhVwW
Q/78XUFm/FgZDTqnSVhXaFRhXMklHWdYmt3yqnUavh+S/R1izmzftFBDyTa1Yt7+vb0gWuVHX
oFsQXULQVnoh5mQ7gfu8C+KRqWbzOLpRfpw0rMhieP5+4xl9vpv1/2EDP0vjarDn3TSjtKCOZf
9tvaOF5m/zVWrH6gIBfOnHemYv0UuF5VS8qJLf5KCiLNMX75TQwiDbbm8/BKiEpauJi1WhDrtv
nnhS3aIXkcl+HlvbWGWB0nA8QBA9Z4oCAz5REcDz/l4cGZsDtqgUj/fMuF1wYow2PF4oP36/6M
M9AO2XOTgheWXpxADBxQtLyuwA24WLKDXXDZHsprrsnAAc4E5TXMA9vXbQPwUV07X6/q/f
b6jajVswrPN1eknnvhlPJgf/5KBoVF5FbPDZSNfHMMHYV38gvYtrqeAACT2LWUh5rEef8agOgd
UzOq9+4b+NVyfcmYeV0oo0g7G17ssAGRvmMv3Viz4q7J9GssCQXIWspQKvsY9Nf/lixMjsP3A
RTGKq2SNAZR+ptBqjc6mvcZDbhIS0AaRG7CfBJMetxaNkGHERQwvTdyjDwTisANtEOVOzsN
83AMxfPsXdthnL/xSCK4aUEz9NKFgpu2BHc+gMzxZXxClQ/xJ+x1K7fmrgGbhRSTIUBIRqVTd0
3CHERr0+zCoHW7Wsu0/m4EhBf3Ou3rs904ZliPnjoTW4Uy8u+JcKf0BWsfTwZi5vXAqavR0BcX
TdWjBWyPfM0iuMowZOyUR9mIGtBAHD3jA/QnXSt4ZnCsxHPpZaiZxneEzdybI7R/fOe8JlLFx7Z
M/ttAlSVd6t+02a3CZxfe4I5VqFbzRE+LDw2UpYW+NprCbelzt7J3WNNIlj/gc3nxfBsdXOjOAOK+
YX3cyEHCECa9irAhs+HOBxxZ5ZswX3sMZkH5rTEWjWYGBBbcO9NXKhtSvebbHkcXTEXvKO
osHhvQykejXKjKGv/lwA3WS2SZw6sp97u65LR32TXMDR5RA0UADjbnoOj1GYB2nm7ogrj1Qy
DpLumaXZt23Wn647uZRTffgtPwnjD3igAqMfGhj+bjQ19XMVEZq65WOgBb4EnmDMU4wlOW9
8yDMvLCCWxgfDQg1i4xW1XEQJXNgzNGHBgngFVXWdtjWCmgg3w+oj1U0aO5x1ipRp4Dbin
u5ZhLiOMF8zlQvefe3lo36hCHX7kEeaUA3leRlUN/vNLkuTHvmtkskS9zjieREtkWbPjxMdtDpYZV
LW45IEfYPMTCQqzhWEEAQ4SZ2DOVHizskVcnBgK4FZ0kDwMrvfEQDI42Sja7ZlpAohM+fObj
DgCTTwITpv6UO8ATRc+Gyt+kvwZz074fd5cBGTluvWzn2T7K+d0F62Rxx3aY3j/HDzaB0gVCiD
pLABQK8ddzdX6Eac1qhC62+Y5ud+jF2fRUsAEj0iIlGAcx0bskRe+RQD8vR/AXM54inqJ9BB/t4+
EyKgREymdmdrCHMQlQCXljWjaikGUAKQYAIRb2uK6PwP6JkYG+gAz8LvcvS6BUPEcldmYQ
Oskr1x18aQ9ocmCbXQXMGPFySuAwrpqEP/fdupQWJb7PZjhdr/dbrG4AzobZBK3cMo8vPP+J

wgO4n3HkHDkuD1Ag0B3u6XJ8iWYd7fbumw3oG7ltsqNW+n11UdGtChgLNRwW2zLTH8hsxt4b
AFOfIG/F0li4YUsIvNtKWLdfN30mQ0o+sYLAs82vlJb+L6Yd2L4+C4C5+36r7zi9sZi/r785iEGvgVc
ZP7+6pd+Xa81rr+ZCp553ebx4KPDPtpH+2gf7aN9tl/20T7aR/toH+2jfbSP9tE+2k/e/hewVa6j8XFf
NQAAAABJRU5ErkJggg==">
<link rel="apple-touch-icon"
href="
AAAAwFBMVEV6mYF6mYB5mYF6mIF6mIB5mIF5mIB5mH94mIB4l393lX51k3xzkHpxjnltinZph
HFjfWxgeWlddWZZcGNUal5QZVpMYVdJXFRFWFBBU00+T0o6Skc1RUIyQUAwPj4uPDwtPD
wtOzwsOzsuOjwtOjwtOjstOjosOjwsOjssOjosOjkrOjotOTosOTssOTotOTksOTksODkrOTsrOTorO
TkrOTgrODorODkrODgqOTkqODkqODgsNzgrNzkrNzgqNzg7T8KnAAAiUUlEQVR42u19C3vbtrl
tjw/DHT34EN9viorIAhUFkQeoQYm35P//V2eGkhyn7b63vltuku8zEseWQooYYGbWmsEAVpSP9
tE+2kf7aB/to320j/bRPtpH+2gf7aN9tl/20R7RVFVTFA2+FE23HNc3fq7uayrKAN1fb/w4qydS6EvT
dv0gClPAcx3LXP3Qow89153QUAw/Y0cm+33pK1HJK1ISQqpdRWTlqQv1h+w7Dr+CPS/3ursII5
eiF4KZBmsprWvKW1rxkuaoUk/aDyeDNg+/7uesKz276KpWSib5c6IEZC/GvpNCjvJM2WbtWuvZ
VH6w/qsLRbEz0jd9aSWMdwfZ8ZESW98KSuUgu64THSWeEhYyC6zlzVx+FMejmJHx5LPpJOg
xtLiADkvJBUvVoKL9MI5jL+SBlqFi85rKSiau/uPMAqiP+0Ws7ILScRTlxisrDgpEqSC2UYh+GBgb
JtAh6L+1ZSBd3TCWeev51h+h/8uQyXQRlqeRUZnpCZPQxjN/zlZxIRuQZpSDmEpPsXLJ+15K
Wp8mlrmrH6P/q6ja8WSZXKiUTemb2+cJ9WRqSkfx4oIR2aANp4ZicVZKOkjRiKE9SZJsvrcWqQ
C5q5g08hiust9haM/cNKMk52RsW8aCzVpZ2fFERD92RWiu3fzI+TSKoevlyMupcL+vFuHwLSNC
ZUNcQ/x+Hs79JbCN5crys+PEW8aL2FmBhfNL3U9MBqt1SMQAVgESiVqS0Pikftf+ryxQfXqWLb
GsppbnEYyXFbFnKksnPUrejkeS+7qySY90oC3JNooL4kqYq7I5bl0Ete+qP1HqgP6ch/FsuPtdBY
5TNu3UlTxxnpZePfGB00sF1rsKnifZ0wsAwabuxCA6IoFyWJ7+/fAAkNfLE5dQAFqercKcgPdshJ
SC14KR1Aafc6FCCgrWaip2IfuRH6pQs3aiOfDAUsywqWBOvpcha4rDq3gjaStpFSum7fhJvUfK
MIgRIKEy9e3UCjGOQ1Nmq3UuxTAcKIkUmwlBXCWSrKGSe0/fx5A/K3bd/BJasjmMkiWObeqqo
nspQMDQSWBuCaBwg4gA8jFbCRnA3HRut2WgeKQVlWNVtZD0QMLVd5AA9MeuRF36FmtbA
VynrIhIQ2ux8hgAFwABsQwqBXA6ySkMt8NE317gq8dXMakFMTcM7KdrSLz+xyVQPyk2aHddO
psSRril4lQjfSgixa1qOYo9CZSIUTDXsa+Yr1hfKhSzPwMrorWpZ7Jl+cqrqvM4Ncf4n54DZD/yt3bk
e8vOC0ml/H3WlYa5aoozUIt0uQGPD8JRefQUMx9F3w9yOA+ylYWlBKyVLFoAbwUXVpNo+Y9
aMqBvUB4E66cMkHZtbPwEnE8vRZevNuwMzoZX5joD+x1EfagcpEAUCREwimZPMktxtqKXvL
RN1vZDD/oX/IPeFJi/kcQha8/tlIeh79qGsog4l2O785SYceg/cZSIQxgwXlhsKm4hIT4DkOt7CuOu
w/SBlcv2CM73wBHV6uqf4xTwHDvPN3a5ozW/QKRbEeIrCQOORou1xfjQ/0o8zWenUf4qE2cJR
GJCjO6Bg05V7ijLgHFwrmzilWvsIEyT4HVr8x/CZCDPPp86wwXc2vOhbhtwOl4BDLrj4F5SJpiQL
ig9kae+keECQrCmmcRBSpA2D3TFSsmhHeXU9XzKliGhZ8EPdZcs/5E5+KwY8S9A+le27/lhKrs
Wun7Q3ZJCKJAqTkXBhp/DzWrlbXkjBQMMdhlYgJBFCm8D+k4S3NRFQMjWD8TaVGAdBBSw
CJ6e/gnvb2XHuuYx2O5qqfz3Zss66OQqeW6E4BuDzo5fEpbYipmwpjvxL46yKRALIB4wnPiyP9
SyFS0QUimrwl/lUqZJEkehv1TU9++/s4NQt5FpnJVFljh6LXteeCbrJSAq9Bl6egZ4bRmJVki0x1Ig
CuzgMhkt7ZIDZLS9HFoxNU1zqtJFLGWkfV4ulHfvPpIujwg+gr+UbBqnMQ/tigKPMMOqHnuWBa
ZiJcBzwFv2OOLLGKCWcQiFAXN5UwELaht5mEZAaFAcRp4LoBUVZ6aVpuG7w7GqagFQTzIC
UFtKYDP1F9cr2p6E5nMpOTgTIoAhe8CoxbmXp/9J13oG7n6ofwnADk59Dd7KZb+Ky0CfORd54
tsrQBReVr4B6vTehELV/hWQ5nAGTtnK2I0wct8kBLw9eMqgPVLKa0a2oPH5pR37bgSOA0Ewh
L/AOID98BaA2gGcaGseboD6gdJA5JZezlxmi4gVu+R9JQD9KcHXQyw7Tq1MTIdC1PIswBTr2A
bfhHEvRCyVj7gLLPT/zP32GOi8oFvdBPOm4othVJT/nukrx3FDjJ0vHNgG8DoC/4bqO8Ix2C+rg
O0A0QQHMrBs6RYBeH3EUY4Biwc+EwEWJLAZxjWyLnwFwoWh72uG1K4RFYsVf1efdsEqLRr

Cp7YCneyb0tMJUMP3hOMnxYTxPwsiauDF4+VUBXqdX+S1/cpAOVwgBWO3b0LFO/JxoBQIjp
mByAfaIMBSSzp2nNngN5u+0S22g84DCPcQX8pkCVAuqsJ4LzhWtWUCSixkoOuehO4B+ofgG
7uJdbIDYi0BhX3ot5DpyqSi7/gIeGxlvBennklfNWG2RmCiKXiypgHZUtbUmP8CxgoXGT74hxOLV
O29JgBoPjDJULHdVUQxrQDcrEOGKTrotaQshUugJztzmUo6CpIC5/wCkDWSKTlVp5jT0704W
kYtfmP5GkBbALke4RMGSiy7PAGZkpv3USJN1fOuFZet4Re5Z5dUnuV4GEeUAEjyKEdObL1gll
IqhQRiNBquVyFSCJ75pmJGDAx/GKTYVSE4JKBPLqgSqD/4rhOtITCyCO8Ef44X7xIYaArM8IXz
ZJWycuOWvMeE+ThcxvGCUyFOoOQRg46Ap/8FYhSZLfUUXOyYbdZLK+rZjgvR8mZix1h19xR
5E6BfK6uyrIDP5p4hz6l/t8x6jyl4UowtxB0ggOKk3ioFBWLwPEBUiLEQlptjugCxMP3s8IO4CAgI1l
tMvxVrIyMAztMkn0l1zMPNZxgMCAykudk3MnE9z/M9d7PQwZBEvQM+8ngB1CdMKQA96Pe2q
qPHR23ArP9shHxsSaYDqRAsWHoMAkfwoCxVQ+ClTQlMG/hEnuVZGvvARhUrhrmDKNLVwSt
nqysDWpk+mggBTFs/nhMBBQV/D0EtBLtJ0gD17eRwELJB7glCUJ6aLqE9uB07P4CuY4ZRGu
A3z8jwkucTCdf6agWhhOklGLqB5yIRBEH7wjWzFFo+sl6w2IvzvfNwAVQNlKY9g6aDSjAwzGEY
2oMYWXUkhAPZYZkTluhjU3sBtgKKNUq+s53yBLqWLUJSjwB7cRwn+Uj4YZ4+yVM1ICVm5cu
pQwAcAYmV5dp5vBHMFgxPvaC7ByEgMgQiOWVx4GNIc6EyzwmAxAgyukpATri8NydcINiZKt0
rT5RZNsQEbBqaGvwOXCyr7dIt6/FgOKRsxQFjfICQ0Hm8BS9AgZp+GgZcWYGhAoLM8ui61Kg
oy03Kfpsu7XA+d2PbEGRrwDUAnAy/LOuaMtMpK1q5q2KgAryX3IHb4WUau6oFrLqaTQEILjRZ
60GxgQc+2gNlQB7A3eA6naRTKWPHuKGDBsBvZIDElysiVQ2z1riKBDTC4Ntqd5CFaRfNHk0B
AFz2LNhAMw3spQk8jodaCh52GNAZS9Mt8gezCVWFqIQi55Ry6OtpX4bW5zk2UO85Fn9/GqE
DHa6Dgb9ZbuB6QNoNaFgQxZlp52XJYxXgS8oThASmqeuGablBBjPAEzUkcqqBEQ2Cm04DV
vGkPtoDzSsqF9GMVT7XbrwmLKqGCXYYwL6l7VTVnrqpDg1wNssykfMv5ziTpVrAlJYHCq4F+2
1xunAG/BNoXLbwiyInsuaCM2tTFuSxlFRVnGrLYJLPoPuAl1iIon57AdIwilNEQD596XUS4GiRHB
kEk2nkwi1WVOX/5ZN6GA4yX9hVDZy8aXgLkTHovVumpl+SS99IE1gKz41HMjpV0TPWQAT4K
6sD41ZH862Kpc98ZPz3Og3t9dpJiWQ00fx9DXo3XQhpIktRnGzpgYMd+wvXzY4P4J84bQFIzs+
GS7ir6F7GmdRdAmb/WDDWFDNt5DOTofnn7s+rBOAUC991bNNywoJIIMnpGgKsHpAb3Hstjs9
wq7nywb92Pei5Li89jVJCiJQXVq+8PeexhWAcLQLWnAW1lMdawQpAKPjL7oPL09OLGOsZTW
VFpppWLNON7dXskW5TgIPcWyjuXlTA11AAsIlQt7wozbIcQrSiFawITLSuCGgTJfHi02MlmJcS1
ae/+i+IxpHEADbgSgbyC5IaqxRCfQwX+nES50oe2Jygq+SvVwFGBGIHXCkuM4N7El3NE6tLPB
sck+Rc2I9FY2TofzH6ivpZWUWkAQDuW9H1ECOAYpeR/gSOdx7+bqQNS1zTDIUsHEyKQuBjG
iCXbBlhaei7nh8jRxEs0nN4C8KjNClJrj+aEKl/PTNmQnDkGTC9ESLGnsrJW2LwhoFaP0wDlfBa
UdY5Z9xXVr5knWGB/wTrQLdV7quKHXBdswrW+dTyFoBhpW9C9+ndE73g6tD86n7WlPF8hnih
xzhSVcOynhczsGMuCLrQ/ivmEKzFBnDpZumUFElJnvOypnVLz9MZaLdrgvagcu0NsIbVu/cfHrA
BjeCSgS8Hfg9QR1ntrZR/aQogm0RGKhG2cCgBK8D3c5aDONbCA0AAb5R7ABVdA2rXNbRlJg
REcM9lj5ZTOu++XgPD73MJrr5vGwH0fhjpRGJzLt1SNgxYD2jVQNnVGgEN2R56OjV4iV/uAIHb
lds4SV2W5AhaSNJVAoYPYR/dOSYh3juvfqvKOoboqQf6IkF3DmPbkczRlH+pcwYJvCVQO3BP
YIxXASwJfGqESLPyNCOTNVgLPaSf19bGDaIkSdLNpuLAXfvx19LXGa+8980yqqu43IHR4vyDv
QrKZaDf6q80Tc8lcOsBmFp0tX6QqRn6cy9R0Eg302MNvpdW4cr3LQP0/bOiJrxhFT83+yrUi2nL
nPfMtj8pwS8ln3CJZZJUVl2ZWC9FfAvFpIce2TGuaF8FUM1qHMYJ80K0SjUjgUgOM5KBnuRNh
qsbseTeJkgvJauiVSYPorTebw6QobYsSuRvoMuHAYYfjPfu9z4rRspnAcCIbwt3ENNhJIy4PLYsX
CjriLTDZaBVpAclQBeEpk2FFMhwwjxcZVzyS6q/26qfpkWShFYu22kaZMNT66V6T0Pq9CzAiUL
sfIhmS1SRL4m+O0M80LVANHChMJK4an8iCYQLvDzVu17mZhxYK1y7yiYOHxupT+o7TYBRP
KfLAGL2HJgqS3Tls3rHBsXKwLfKQU6iq+YJwGVZjuEW2kBPf8fk4WfFvIjzMJxP/ItpJFXd1CMt7
TBnRQzAXeAiEDrdp3dyQQ7jlnH4ledWwgWM6OcXuDYCPmICAJyoADvU0BsC/gK1mxcl+xPz
USKs1GkB//oGU41wgTwdaOk7RS2rUuYIaLblJu+0dKwpQR4hNJXWqpi+pmOhq17OKAQ/WP

Qk5Nx/xDuK4Q4C87hj8QJsfa50aUVz7jlnMVwDLAmQOrLJQcwZd0wrGcZCfx9PpCphZampxD
AkK+N7FgGxgdOiFhCn0PrKKCHcCSDgxVRKSxtOybyK9AT9r0+0qbEqnCUaGMk6YIdLvMF0H
5urvvbroPTeyZOqSpwvltD1f2lqmNv3p2ifo3ys5soDAKL8OjFqABwT3qEEvrEjsCEN/mwqBvEmk
E94s8TsIlzqfGGxBRN3xjV/ytJlxOh75doVN1Kgt5jB0SPj9hBVWXr+tQVhBPHPzIEWXhCE9xbYy
6sBuFEY3VoYeehrcAW9DE2YgaGfk/XgS6V8p1w7YJWhKkZk4Kern/4vYdDfffcWm1o25iWFBM
5ROiannFnvuvit/TFOeFI1CLFU+NJe4p/5Z+3p6RN+u4/n9Yp7+/pxDuG9EOTCDwwzjhjev5MVa
C/E4YEP0NRlwveiDjNSxApEdYBta+WH3K7yb72DxWoSmKYd2+sadKnl1o9RoP+3SaIa7QorT
wwFgiKINWjl/VQzgFOQO86Rl/Y6uzSX6X0WnN43TnKMDKseUym6LJcsVn6yBihPGuCpcqCNb
cRFoj1Ih2b/+PQnZ3Rzkuqri17a7GXxndmz/pEZq5+u9/4hj6tqPtti+WDbpStFCdPPjxFA/Yvs0OtH/
x1FVV+jqvbXn4NMl5yEgNi+dJcQ5tmPKUNTFXNjWRvzDzC2slzgCb5jqsiaAZzxImsz/8Wrtfmda
zPXrwYARTEdP8SbjW82IIEZc1ywaedqGPagFCPwZEwBscL4Shye5kJ1oG3sSPpkc9tNcOH8y
DsOjW4TBd+RZ1yYkfKQRZu7BJhTijmwvgPwvG1kvppaCP8lkO+Jls4yb3m6eNAEbMie8/Ieo0P
7BDS/utC2qWveCv4/4QqYnVdtxQHXIOFvXcUoQFH3XFDe02FiLFzfdlyuQwoD0sDdbS2J9L92
EyYNqxj321TxScOTRwkQ0qobW5neNXIBEdXI1ErKO/g6NE0VrVTFnYtG64YCz2/oLACpMT3R
yrYRYk/i5RzNrJMdSN1iAh7DGl5+Lf3GyWaFbiSWwQ6PEgBjYCxlgjD1huzAfWnHYa6PrMPEOh
W0hMDe21VCjvPOW8LyWYVwVUY+jyMuZgxl4aGnWkYVHXtxlFmKOV8xnEiofY3q0q3t2Noi7g
5N9xgvqmKyE2JceV/8UZF1gbcWMrKBuISNvDBnAd2tqOCpe2sYjXhlLXhsWnCRAJ1hGQZkbllj
4jG09JVhxwxiSHq0X6KKVeZYR2LbJejdc/wYBcIq/25CTbimCiGkJZwJBiHN3My02CjzeLc1iV7fC
SrEWTC/cHAPltsoqp7LdgSYuk2v+9z2Ap3+ra8ry97Kbpcx1LnwEVG9qlrw6DFLwbVUsxmrWow
VNNAbALcn3EZsgFXgDNQNiRfLxdy0a51jQwLtMyDWIgVrIY6CFaQwl67yeY4cVMVnvWyPmxc
PpV+kOEt5AeMnD+FyT0pAmqbw7V3TsmSBeKTno6xQn9TbFdf8D2h8y76dgbIGfgz/qz5pCciP
mRaIdblMPt/u1dRViknI+1irqs3qoR8w3XpmmwcIAAQrg1ibWOutlLSxMPlmtfCjtF4wdP4B1wWan
t1swLNQPK9sW4bVoDCwOfbTwU+TWFKhvlpn5r/Ju7+BIyU9iB7GKu/qR5QdILjXFCEIlpyCQoM
ANqPddEudv3JWHi5syD34oLLJQ7zQKxtQIfwUI5b9b6K1Zj/fs1fCw5C3w3Src1KVzzHbtWJfRYu
geIQXBY2J+byPSrEbPozQbQCBSv4ZJbG7CLq0prTeboOrAIADWRRF8Z6LriXQI2MLodZkvgg
P5IH09D4cT4pRArTV+3hhmA8xAU0x69+Hc2OqqpHzWQkW4IRoP311HC8CVPPmgctlnOT+K
sCeivFCKsYmKtsLAbDQ899FL78RgDXt5bYuqapumThhHn3eZHr8iIBSRd/Cj7NpYu03JtJwvaUff
zPVP6rQvhnGMs3SLMty76ZCgH+nU9XCN761Z4u6iIbZ6ldi54Ch3IvngDaaa90JFLsmG2P1GB
4nUfWtjb0JwPfLA4yiUU79vnJexmfOl8zEoeXxaj235YyA+1YiC8JaIgasDT1nwman85X9RCU8IL
6ybXXhBmmhK55suP0QEAAe14JGSCAHDCtvKdrkEjwfEpVr0HI9R+UqQP3VjapXHJAyTxKgIv
wK4+psKZcv5i2lulDMUsr2xhM1xSmIFG5UcZmuNe0RPCgkfOz6tjnRrgU8mxfuEHM70fnKbQXf
Bp75hALsKYm/Vb8WRxu5yMBndg82dRAjAMH6dozMOpF8kHvjCvHrjDLxG5D0mj1kApDHTbg
OPE0Sv85jh8XFqlHgEhcPsXJsafiyTHFEYQYAiW8qtF7NVIIi4KmLZD5kYq1iajqs6CB4Ys0TZyV
cjqcqmKsxcLjoGcKxE4Kw9hAe5MJknmWeYcvzudvJAgkZpWfByiQMkz0ZGllY2F1wo/vs1qKrA6
gPobpQNqwV9REXN57Am0k6cTnFvusnl44LrM9Rr1nfl6ic6EtZ+crTQ3go0E6OtTFrHFdd97C6r
AT3o4V7TPpLDqbxe8PrY2LMXAi3f07gSKeJJOpMJVoOuv+kRlhI9wWXXADIt/LUtz2DmAzDRy
Cpt7WozeHAzy04vcJ5zEI3PKuCmOu4ub9h7CndlTiQi4DJtm1p3XBKZRkjvrllAS9OrGGnercFvw
LvlPvdjMlmuatLFqO/0q5lKxCO/VJD7CPLxLwagJmV5ekkeRcaD8pnwcDljBTZ6pougX9jeF3WM
2o6KSMS7IIxmnur2etjwQw5whf4rGKegZzxPJpt1y844cWs2U94Cg7B/JVkFck8DEex/3kBd+LJM
Y87skRzfd/zXzghPGRewTDmHMnKDpIsz5JwzizgwPqeNxejzxc5t3d8hC9VXXv4Ue4tolMMN4S
b0ySwbyesgAKFge/PBbUP38ah/rvlgcVaXy+Ut5+0M/dwudZXmvLnM24+PbTkD1Xn0x9e3554/4
+X7NqrtNwtM6fdf5hZIbSnV6m+m5aqX6V6UlX1nz1lAp+nfpebP9rfiiTUF935GI5/Ps+/AodqoBUv
Xc8zf66Fltm7LMK8iFTgqG5eFvrPp0QzM+aWgoHoQ7lj30GJHCKTpRIy/m4bJN9XAG0Ry71jUp

Eb3/MUp/8on0FllkriKT/jBMwSBATYarrSflIY0NR1Jk/kp7Tgl4CU8VjVftb+wxQ4DJOpPzEc/+QCY
Jqq+LHKNf6SWP5btvmkWGHo/KcC/H+R2advYiv15ZMU5Y+7aZ7unPnTnyO51yvw38Zjfwzu/vq
Ne9b1m7jtP1EICGNXK/V1zD13Ed5cKn99fKuqfVL/TXCt/v1nLm8HEr+hbTzgwPf1Us+4ZmV1N
0rTNPHNl0+DeN68vonHQ75+hg13e+59BQHItevNy06vagqM+bNfiLamWB5eZHydJ/XlmcFm8T
YJooLtCCG/4F+Gta14Ll5WHS+yI5iou3YEyEJECZdMYgFrsL5THk0x6oowljsvS49GRwgrc/0+ee
rTli4OBAu87xmCZVrg+vhXr4WJsZywy0VyxnGDwhtaSJuG01ND9+W+LDe4Cc47XmhzYqeaCp
Ka6rzXxCIIW7e0rCmnskrNl9Vql5R1s8XFkJukQKpbvpsXLO8Oiuxovdu9yKhaR3hovcvuR9yomu
ZXHa/rE2solcx9y0bpkHMhJykvcpqm0gZd2RDeHnDjl6S0LoI5keUeu9MgWtxnL6ioWXbLCKqLh
HLcvU6tFwGKUZyFrG4yAstIpTj3dfV1koJfsOy9flnAhPGp9mKcqrLilO54tn6LAFLIocixZVm2URa
L5MjHjgaun3QCT3eEOcGNzyMoUBbHwDmbEx7gd50Akw59J0dK/BedKqZOtPxavH7fpj/2TePct
XGdcazRx2r223rrOmvocCl814vliAngN81Ay4m31vW1vl7r4AX0YuzODMsDFk4eYFm6ukrZQbY
8c9ZYHxAx8qJCuPIrcXeAvOkDCnDB1/d9WYpZth1cQ+/jjeHPvEIo7vtW1PksCEoxsaxussRQPr
1FACb2e/cVIpnlmU75SsWzvHR1Xj5ySoqnsOk3l+hFtz0v88q4HObeEPu2cmTkE26r4bjIoSra55j
wSXZte1chdTHvWBd93zD/tty3kW0Lz1TwmcYbD38KuWiJpy1uOULowHbse4apWfV6HJmqxqQ
RDKKtxQsy3E3YBvV4TkLWNCz+7xcBhqnvJ9QqUCIbj6ADGZubAODPWC9l7l/g3+v6LbhVwW
Q/78XUFm/FgZDTqnSVhXaFRhXMklHWdYmt3yqnUavh+S/R1izmzftFBDyTa1Yt7+vb0gWuVHX
oFsQXULQVnoh5mQ7gfu8C+KRqWbzOLpRfpw0rMhieP5+4xl9vpv1/2EDP0vjarDn3TSjtKCOZf
9tvaOF5m/zVWrH6gIBfOnHemYv0UuF5VS8qJLf5KCiLNMX75TQwiDbbm8/BKiEpauJi1WhDrtv
nnhS3aIXkcl+HlvbWGWB0nA8QBA9Z4oCAz5REcDz/l4cGZsDtqgUj/fMuF1wYow2PF4oP36/6M
M9AO2XOTgheWXpxADBxQtLyuwA24WLKDXXDZHsprrsnAAc4E5TXMA9vXbQPwUV07X6/q/f
b6jajVswrPN1eknnvhlPJgf/5KBoVF5FbPDZSNfHMMHYV38gvYtrqeAACT2LWUh5rEef8agOgd
UzOq9+4b+NVyfcmYeV0oo0g7G17ssAGRvmMv3Viz4q7J9GssCQXIWspQKvsY9Nf/lixMjsP3A
RTGKq2SNAZR+ptBqjc6mvcZDbhIS0AaRG7CfBJMetxaNkGHERQwvTdyjDwTisANtEOVOzsN
83AMxfPsXdthnL/xSCK4aUEz9NKFgpu2BHc+gMzxZXxClQ/xJ+x1K7fmrgGbhRSTIUBIRqVTd0
3CHERr0+zCoHW7Wsu0/m4EhBf3Ou3rs904ZliPnjoTW4Uy8u+JcKf0BWsfTwZi5vXAqavR0BcX
TdWjBWyPfM0iuMowZOyUR9mIGtBAHD3jA/QnXSt4ZnCsxHPpZaiZxneEzdybI7R/fOe8JlLFx7Z
M/ttAlSVd6t+02a3CZxfe4I5VqFbzRE+LDw2UpYW+NprCbelzt7J3WNNIlj/gc3nxfBsdXOjOAOK+
YX3cyEHCECa9irAhs+HOBxxZ5zswX3sMZkH5rTEWjWYGBBbcO9NXKhtSvebbHkcXTEXvKO
osHhvQykejXKjKGv/lwA3WS2SZw6sp97u65LR32TXMDR5RA0UADjbnoOj1GYB2nm7ogrj1Qy
DpLumaXZt23Wn647uZRTffgtPwnjD3igAqMfGhj+bjQ19XMVEZq65WOgBb4EnmDMU4wlOW9
8yDMvLCCWxgfDQg1i4xW1XEQJXNgzNGHBgngFVXWdtjWCmgg3w+oj1U0aO5x1ipRp4Dbin
u5ZhLiOMF8zlQvefe3lo36hCHX7kEeaUA3leRlUN/vNLkuTHvmtkskS9zjieREtkWbPjxMdtDpYZV
LW45IEfYPMTCQqzhWEEAQ4SZ2DOVHizskVcnBgK4FZ0kDwMrvfEQDI42Sja7ZlpAohM+fObj
DgCTTwITpv6UO8ATRc+Gyt+kvwZz074fd5cBGTluvWzn2T7K+d0F62Rxx3aY3j/HDzaB0gVCiD
pLABQK8ddzdX6Eac1qhC62+Y5ud+jF2fRUsAEj0iIlGAcx0bskRe+RQD8vR/AXM54inqJ9BB/t4+
EyKgREymdmdrCHMQlQCXljWjaikGUAKQYAIRb2uK6PwP6JkYG+gAz8LvcvS6BUPEcldmYQ
Oskr1x18aQ9ocmCbXQXMGPFySuAwrpqEP/fdupQWJb7PZjhdr/dbrG4AzobZBK3cMo8vPP+J
wgO4n3HkHDkuD1Ag0B3u6XJ8iWYd7fbumw3oG7ltsqNW+n11UdGtChgLNRwW2zLTH8hsxt4b
AFOfIG/F0li4YUsIvNtKWLdfN30mQ0o+sYLAs82vlJb+L6Yd2L4+C4C5+36r7zi9sZi/r785iEGvgVc
ZP7+6pd+Xa81rr+ZCp553ebx4KPDPtpH+2gf7aN9tl/20T7aR/toH+2jfbSP9tE+2k/e/hewVa6j8XFf
NQAAAABJRU5ErkJggg==">
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

```
<link
href="https://fonts.googleapis.com/css2?family=Archivo:wght@400;500;600;700&family=Archiv
o+Narrow:wght@400;600;700&display=swap" rel="stylesheet">
<style>
/*
================================================================================
=
   TOKENS
   Paleta: taller. Fondo gris-verde de pared de taller, panel de papel,
   tinta pino profunda, verde-teal para acciones y rojo de línea de sierra
   reservado exclusivamente para corte / merma / eliminar.

================================================================================
= */
:root{
  --ink:#143e45;
  --ink-2:#2f5a61;
  --ink-3:#36595f;
  --paper:#7a9981;
  --panel:#d0e0d3;
  --panel-2:#e0ebe2;
  --rule:#9bb8a1;
  --rule-2:#b9d0be;
  --accent:#143e45;
  --accent-d:#0d2c32;
  --accent-soft:#e4efe7;
  --cut:#a52816;
  --cut-soft:#fbe6e2;
  --keep:#25601f;
  --field:#f3f8f4;

  --sans:"Archivo","Segoe UI",system-ui,-apple-system,Helvetica,Arial,sans-serif;
  --narrow:"Archivo Narrow","Arial Narrow","Archivo",system-ui,sans-serif;

  --r:4px;
  --shadow:0 1px 0 rgba(20,62,69,.06), 0 6px 18px -14px rgba(20,62,69,.5);
}

*{box-sizing:border-box}
html,body{margin:0;padding:0}
body{
  background:var(--paper);
  color:var(--ink);
```

```
  font-family:var(--sans);
  font-size:15px;
  line-height:1.45;
  -webkit-text-size-adjust:100%;
}
h1,h2,h3{margin:0;font-weight:600;letter-spacing:-.01em}
button,input,select{font:inherit;color:inherit}
:focus-visible{outline:2px solid var(--accent);outline-offset:2px;border-radius:2px}

/* ---------- encabezado: cinta métrica ---------- */
.topbar{
  position:sticky;top:0;z-index:30;
  background:var(--panel);
  border-bottom:1px solid var(--rule);
}
.ruler{
  height:13px;
  background-color:var(--panel);
  background-image:
    repeating-linear-gradient(90deg,var(--rule) 0 1px,transparent 1px 8px),
    repeating-linear-gradient(90deg,var(--ink-2) 0 1px,transparent 1px 40px);
  background-size:100% 5px,100% 12px;
  background-repeat:repeat-x;
  background-position:left bottom,left bottom;
}
.topbar .inner{
  display:flex;align-items:center;gap:16px;
  padding:10px 18px 12px;
  flex-wrap:wrap;
}
.brand{display:flex;align-items:baseline;gap:10px;margin-right:auto}
.brand h1{font-size:19px}
.brand span{font-family:var(--narrow);color:var(--ink-2);font-size:14px}
.saved{
  position:fixed;left:50%;bottom:calc(16px +
env(safe-area-inset-bottom,0px));transform:translateX(-50%);
  z-index:60;max-width:90vw;text-align:center;pointer-events:none;
  background:var(--ink);color:var(--panel);font-family:var(--narrow);font-size:13.5px;
  padding:7px 14px;border-radius:999px;box-shadow:0 8px 20px -10px rgba(0,0,0,.55);
}
.saved:empty{display:none}

/* ---------- pestañas de vista ---------- */
```

```
.tabs{display:flex;gap:4px;padding:0 18px;background:var(--panel)}
.tab{
  border:none;background:transparent;color:var(--ink-2);
  padding:9px 4px;font-size:14px;font-weight:500;cursor:pointer;
  border-bottom:2px solid transparent;margin-bottom:-1px;
}
.tab:hover{color:var(--ink)}
.tab[aria-selected=true]{color:var(--accent-d);border-bottom-color:var(--accent);font-weight:600}
[hidden]{display:none !important}
.tab .short{display:none}

/* ---------- botones ---------- */
.btn{
  border:1px solid var(--ink);background:var(--panel);color:var(--ink);
  padding:7px 14px;border-radius:var(--r);cursor:pointer;
  font-size:14px;font-weight:500;
  transition:background .12s ease,color .12s ease;
}
.btn:hover{background:var(--panel-2)}
.btn.primary{background:var(--accent);border-color:var(--accent);color:#fff}
.btn.primary:hover{background:var(--accent-d)}
.btn.quiet{border-color:var(--rule);color:var(--ink-2);background:transparent}
.btn.quiet:hover{background:var(--panel-2);color:var(--ink)}
.btn.danger{border-color:var(--rule);color:var(--cut);background:transparent}
.btn.danger:hover{background:var(--cut-soft)}
.btn:active{transform:translateY(1px)}

/* ---------- estructura ---------- */
.wrap{
  display:grid;
  grid-template-columns:minmax(330px,390px) minmax(0,1fr);
  gap:20px;
  padding:20px;
  align-items:start;
  max-width:1680px;margin:0 auto;
}
@media (max-width:1040px){
  .wrap{grid-template-columns:1fr;padding:14px;gap:14px}
}

.card{
  background:var(--panel);
  border:1px solid var(--rule);
```

```
  border-radius:var(--r);
  box-shadow:var(--shadow);
}
.card + .card{margin-top:16px}
.card > h2{
  display:flex;align-items:baseline;gap:9px;
  font-size:16px;
  padding:12px 16px 10px;
  border-bottom:1px solid var(--rule-2);
}
.card > h2 .n{
  font-family:var(--narrow);font-size:13px;font-weight:700;
  color:var(--accent);
  width:19px;height:19px;flex:none;
  display:grid;place-items:center;
  border:1px solid var(--accent);border-radius:50%;
  align-self:center;
}
.card > h2
small{margin-left:auto;font-family:var(--narrow);font-weight:400;font-size:13px;color:var(--ink-3)}
.card .body{padding:14px 16px 16px}

/* ---------- formularios ---------- */
.grid{display:grid;gap:10px}
.g2{grid-template-columns:1fr 1fr}
.g3{grid-template-columns:1fr 1fr 1fr}
label{display:block}
.lab{
  font-family:var(--narrow);font-size:13px;color:var(--ink-2);
  display:block;margin-bottom:3px;letter-spacing:.01em;
}
input[type=text],input[type=number],select{
  width:100%;
  background:var(--field);
  border:1px solid var(--rule);
  border-radius:var(--r);
  padding:7px 9px;
  font-size:15px;
  font-variant-numeric:tabular-nums;
}
input[type=number]{font-family:var(--narrow);letter-spacing:.02em}
input:hover,select:hover{border-color:var(--ink-3)}
input:focus,select:focus{border-color:var(--accent);outline:none;box-shadow:0 0 0 2px
```

```
var(--accent-soft)}
input::-webkit-outer-spin-button,input::-webkit-inner-spin-button{opacity:.35}
.check{
  display:flex;align-items:center;gap:8px;
  font-size:14px;cursor:pointer;
  padding:6px 0;
}
.check input{width:17px;height:17px;accent-color:var(--accent);margin:0;flex:none}
.hint{font-family:var(--narrow);font-size:13px;color:var(--ink-3);margin-top:7px}
.hint b{color:var(--ink-2);font-weight:600}

/* lectura automática área / precio m² */
.readout{
  display:flex;gap:18px;flex-wrap:wrap;
  margin-top:12px;padding-top:11px;
  border-top:1px dashed var(--rule);
}
.readout div{display:flex;flex-direction:column;gap:1px}
.readout .k{font-family:var(--narrow);font-size:12.5px;color:var(--ink-3)}
.readout
.v{font-family:var(--narrow);font-size:17px;font-weight:700;font-variant-numeric:tabular-nums}

/* selector de merma */
.chips{display:flex;gap:6px;flex-wrap:wrap}
.chip{
  border:1px solid var(--rule);background:var(--panel);
  padding:5px 11px;border-radius:999px;cursor:pointer;
  font-family:var(--narrow);font-size:14px;
}
.chip[aria-pressed=true]{background:var(--accent);border-color:var(--accent);color:#fff;font-weig
ht:600}

/* ---------- tabla de piezas ---------- */
.addrow{display:grid;grid-template-columns:1fr 78px 78px 60px;gap:8px;align-items:end}
.addrow .full{grid-column:1/-1}
@media (max-width:480px){.addrow{grid-template-columns:1fr 1fr}.addrow
.nm{grid-column:1/-1}}

.tablewrap{margin-top:14px;overflow-x:auto}
table{width:100%;border-collapse:collapse;font-size:14px}
thead th{
  font-family:var(--narrow);font-weight:600;font-size:12.5px;color:var(--ink-2);
  text-align:left;padding:0 6px 6px;border-bottom:1px solid var(--rule);white-space:nowrap;
```

```
}
thead th.r,td.r{text-align:right}
tbody td{padding:4px 6px;border-bottom:1px solid var(--rule-2);vertical-align:middle}
tbody tr:hover{background:var(--panel-2)}
td input[type=text]{padding:4px 6px;border-color:transparent;background:transparent}
td input[type=number]{padding:4px
5px;border-color:transparent;background:transparent;text-align:right;font-size:14px}
td input:hover{border-color:var(--rule);background:var(--field)}
td input:focus{background:var(--field)}
td .swatch{width:11px;height:26px;border-radius:2px;border:1px solid
rgba(20,62,69,.35);display:inline-block;vertical-align:middle}
.num{font-family:var(--narrow);font-variant-numeric:tabular-nums;white-space:nowrap}
.rowbtns{display:inline-flex;gap:1px;align-items:center}
.x{
  border:none;background:transparent;color:var(--ink-3);cursor:pointer;
  padding:4px 6px;border-radius:var(--r);font-size:16px;line-height:1;
  display:inline-flex;align-items:center;
}
.x:hover{background:var(--cut-soft);color:var(--cut)}
.x svg{display:block;pointer-events:none}
.x.dup:hover{background:var(--accent-soft);color:var(--accent)}
.grainbtn{
  border:1px solid var(--rule);background:transparent;border-radius:var(--r);
  font-family:var(--narrow);font-size:12px;padding:2px 6px;cursor:pointer;color:var(--ink-3);
}
.grainbtn[aria-pressed=true]{border-color:var(--ink);color:var(--ink);background:var(--panel-2);fon
t-weight:600}
.empty{font-family:var(--narrow);color:var(--ink-3);padding:16px 2px;font-size:14px}

/* La lista vive en una columna angosta (≈350 px en PC) o a pantalla completa
   (celular), así que el diseño se decide por el ancho del contenedor y no por
   el de la ventana. Debajo de 600 px cada pieza pasa a ser una tarjeta. */
.tablewrap{container-type:inline-size;container-name:piezas}
@container piezas (max-width:600px){
  #tbl,#tbl thead,#tbl tbody,#tbl tr,#tbl td{display:block;width:100%}
  #tbl thead{display:none}
  #tbl tbody tr{
    display:grid;grid-template-columns:repeat(3,minmax(0,1fr));gap:8px 10px;align-items:end;
    border:1px solid var(--rule);border-left:5px solid var(--c,var(--rule));
    border-radius:var(--r);padding:10px;margin-bottom:8px;background:var(--panel);
  }
  #tbl tbody tr:hover{background:var(--panel)}
  #tbl tbody
```

```
td{border:none;padding:0;display:flex;flex-direction:column;align-items:stretch;gap:2px;min-widt
h:0;text-align:left}
  #tbl tbody
td::before{content:attr(data-l);font-family:var(--narrow);font-size:12px;color:var(--ink-3)}
  #tbl tbody td:nth-child(1){display:none}                              /* el color pasa al borde de la
tarjeta */
  #tbl tbody td:nth-child(2){grid-area:1 / 1 / 2 / 3}                    /* nombre */
  #tbl tbody td:nth-child(2)::before{display:none}
  #tbl tbody td:nth-child(9){grid-area:1 / 3 / 2 / 4;justify-self:end;align-self:start} /* duplicar /
eliminar */
  #tbl tbody td:nth-child(9)::before{display:none}
  #tbl tbody td:nth-child(6){align-items:flex-start}                     /* botón veta */
  #tbl tbody td.r{text-align:left}
  #tbl td input[type=text],#tbl td
input[type=number]{border-color:var(--rule);background:var(--field);text-align:left;width:100%}
}

/* ---------- plano ---------- */
.planhead{
  display:flex;align-items:baseline;gap:14px;flex-wrap:wrap;
  padding:12px 16px;border-bottom:1px solid var(--rule-2);
}
.planhead h2{font-size:16px;display:flex;align-items:center;gap:9px}
.planhead h2 .n{
  font-family:var(--narrow);font-size:13px;font-weight:700;color:var(--accent);
  width:19px;height:19px;flex:none;display:grid;place-items:center;
  border:1px solid var(--accent);border-radius:50%;
}
.planhead .stats{margin-left:auto;display:flex;gap:16px;flex-wrap:wrap}
.planhead .stats span{font-family:var(--narrow);font-size:13.5px;color:var(--ink-2)}
.planhead .stats b{font-weight:700;color:var(--ink);font-variant-numeric:tabular-nums}

.boards{padding:16px;display:flex;flex-direction:column;gap:22px}
.board{break-inside:avoid}
.board header{
  display:flex;align-items:baseline;gap:12px;flex-wrap:wrap;margin-bottom:7px;
}
.board header h3{font-size:15px;font-family:var(--sans)}
.board header
.meta{font-family:var(--narrow);font-size:13px;color:var(--ink-2);font-variant-numeric:tabular-num
s}
.board header .meta em{font-style:normal;color:var(--keep)}
.board svg{display:block;width:100%;height:auto;background:var(--field);border:1px solid
```

```
var(--rule);border-radius:var(--r)}
svg text{font-family:var(--narrow)}

.legend{display:flex;gap:16px;flex-wrap:wrap;padding:0 16px
16px;font-family:var(--narrow);font-size:13px;color:var(--ink-2)}
.legend
i{display:inline-block;width:20px;height:11px;vertical-align:-1px;margin-right:6px;border:1px solid
var(--ink-3)}
.legend .l-waste{background:repeating-linear-gradient(45deg,var(--rule) 0 1px,transparent 1px
5px);background-color:var(--panel-2)}
.legend .l-keep{border:1.5px dashed var(--keep);background:transparent}
.legend .l-kerf{background:var(--cut);border-color:var(--cut);height:3px;vertical-align:3px}

.notice{
  margin:0 16px 16px;padding:10px 12px;border-radius:var(--r);
  border:1px solid var(--cut);background:var(--cut-soft);color:#7d1e10;font-size:14px;
}
.notice b{font-weight:700}
.placeholder{
  margin:16px;padding:44px 20px;text-align:center;
  border:1.5px dashed var(--rule);border-radius:var(--r);
  color:var(--ink-3);font-family:var(--narrow);
}
.placeholder
strong{display:block;color:var(--ink-2);font-size:16px;font-weight:600;margin-bottom:4px}

/* ---------- costos ---------- */
.costs .body{padding:0}
.costline{
  display:flex;align-items:baseline;gap:12px;
  padding:9px 16px;border-bottom:1px solid var(--rule-2);
}
.costline .k{font-size:14px}
.costline .k small{display:block;font-family:var(--narrow);font-size:12.5px;color:var(--ink-3)}
.costline
.v{margin-left:auto;font-family:var(--narrow);font-size:16px;font-weight:600;font-variant-numeric:t
abular-nums;white-space:nowrap}
.costline.sub .k{color:var(--ink-2);padding-left:14px;position:relative}
.costline.sub
.k::before{content:"";position:absolute;left:0;top:.55em;width:7px;height:1px;background:var(--rul
e)}
.costline.sub .v{font-weight:400;font-size:15px}
.costline.total{
```

```
  background:var(--ink);color:var(--panel);border:none;
  border-radius:0 0 var(--r) var(--r);padding:14px 16px;
}
.costline.total .k{font-size:15px;font-weight:600}
.costline.total .v{font-size:23px;font-weight:700}
.costline.total .k small{color:#9fb0ab}

/* ---------- tarjetas de material (cotizador) ---------- */
.mitem{
  border:1px solid var(--rule);border-radius:var(--r);
  background:var(--panel);margin-bottom:10px;overflow:hidden;
}
.mitem-head{
  display:flex;gap:8px;align-items:center;flex-wrap:wrap;
  padding:9px 10px;background:var(--panel-2);border-bottom:1px solid var(--rule-2);
}
.mitem-head .mname{flex:1 1 160px;min-width:120px;background:var(--field)}
.mitem-head .mkind{flex:0 0 auto;width:auto;min-width:168px;background:var(--field)}
.mitem-body{padding:10px}
.mitem-body .hint{margin-top:8px}
.mitem-foot{
  display:flex;align-items:baseline;gap:10px;
  padding:7px 10px;border-top:1px dashed var(--rule);
}
.mitem-foot .mnote{font-family:var(--narrow);font-size:12.5px;color:var(--ink-3)}
.mitem-foot
.mcost{margin-left:auto;font-family:var(--narrow);font-weight:700;font-size:15px;white-space:now
rap}
.mitem-link{display:flex;gap:6px;align-items:center;margin-top:10px;padding-top:10px;border-top
:1px dashed var(--rule)}
.mitem-link input{flex:1 1 auto;min-width:0;font-size:13.5px}
.linkbtn{flex:none;text-decoration:none;font-size:13px;padding:6px
10px;white-space:nowrap;max-width:45%;overflow:hidden;text-overflow:ellipsis}
.pstatus{font-family:var(--narrow);font-size:12px;color:var(--ink-3);white-space:nowrap}
.pstatus.stale{color:var(--cut);font-weight:600}
@media (max-width:480px){
  .mitem-head .mkind{flex:1 1 100%}
  .mitem-link{flex-wrap:wrap}
  .linkbtn{max-width:100%}
}

.savebar{display:flex;gap:8px;margin-top:14px}
.savebar .btn{flex:1}
```

```
.wrap.stack{grid-template-columns:1fr;padding-top:0}
.costline.total.price{background:var(--accent-d)}
.costline.sub.muted .v{font-family:var(--narrow);font-weight:400;font-size:13px;color:var(--keep)}
.sendbar{padding:12px 16px 14px;border-top:1px solid var(--rule-2)}
.sendbar .btn{width:100%}
.sendbar .hint{margin-top:8px}
.costline.warnline{background:var(--cut-soft)}
.costline.warnline .k{color:#7d1e10}
.costline.warnline .k small{color:#9a3a29}
.costline.warnline .v{color:var(--cut);font-size:15px}

/* ---------- lista de muebles guardados ---------- */
.mbook-row{
  display:flex;align-items:center;gap:12px;flex-wrap:wrap;
  padding:10px 12px;border:1px solid var(--rule);border-radius:var(--r);
  background:var(--panel);margin-bottom:8px;
}
.mbook-row.editing{border-color:var(--accent);background:var(--accent-soft)}
.mbook-num{font-family:var(--narrow);font-weight:700;color:var(--ink-3);width:20px;text-align:cen
ter;flex:none}
.mbook-info{flex:1 1 200px;min-width:150px}
.mbook-name{font-weight:600;font-size:14.5px}
.mbook-meta{font-family:var(--narrow);font-size:12.5px;color:var(--ink-2);margin-top:1px}
.mbook-price{text-align:right;flex:0 0 auto;padding:0 4px}
.mbook-price .k{display:block;font-family:var(--narrow);font-size:11.5px;color:var(--ink-3)}
.mbook-price
.v{font-family:var(--narrow);font-weight:700;font-size:15.5px;color:var(--accent-d);white-space:no
wrap}
.x[disabled]{opacity:.28;cursor:default}
@media (max-width:560px){
  .mbook-row{padding:10px}
  .mbook-price{flex:1 1 100%;text-align:left;order:3;display:flex;gap:8px;align-items:baseline}
  .mbook-price .k{display:inline}
}

/* ---------- calculadora ---------- */
.calc .body{padding:14px}
.calc-screen{
  background:var(--ink);color:var(--panel);border-radius:var(--r);
  padding:12px 14px 10px;min-height:100px;
  display:flex;flex-direction:column;justify-content:flex-end;gap:2px;overflow:hidden;
}
```

```
.calc-expr{
  font-family:var(--narrow);font-size:16px;color:#9fb0ab;text-align:right;min-height:22px;
  overflow-x:auto;white-space:nowrap;scrollbar-width:none;
}
.calc-expr::-webkit-scrollbar{display:none}
.calc-res{
  font-family:var(--narrow);font-weight:700;font-size:38px;line-height:1.12;text-align:right;
  overflow-wrap:anywhere;font-variant-numeric:tabular-nums;
}
.calc-res.err{font-size:17px;font-weight:600;color:#ffb4a8;line-height:1.3}
.calc-actions{display:flex;justify-content:flex-end;margin:8px 0 10px}
.calc-actions .btn{font-size:13px;padding:5px 11px}
.calc-keys{display:grid;grid-template-columns:repeat(4,minmax(0,1fr));gap:8px}
.calc-keys .k{
  min-height:58px;border:1px solid var(--rule);border-radius:var(--r);background:var(--panel-2);
  font-family:var(--narrow);font-size:23px;font-weight:600;color:var(--ink);cursor:pointer;
  transition:background .1s ease,transform .05s ease;
  -webkit-tap-highlight-color:transparent;touch-action:manipulation;
}
.calc-keys .k.num{background:var(--field)}
.calc-keys .k.op{background:var(--accent-soft);color:var(--accent-d);border-color:#8fb09b}
.calc-keys .k.clr{background:var(--cut-soft);color:var(--cut);border-color:#efc4bc}
.calc-keys .k.eq{background:var(--accent);color:#fff;border-color:var(--accent)}
.calc-keys .k:hover{filter:brightness(.96)}
.calc-keys .k:active{transform:scale(.96)}
.chist-row{
  display:block;width:100%;text-align:right;cursor:pointer;
  background:var(--panel);border:1px solid var(--rule);border-radius:var(--r);
  padding:8px 12px;margin-bottom:6px;
}
.chist-row:hover{background:var(--panel-2)}
.chist-row
.e{display:block;font-family:var(--narrow);font-size:13.5px;color:var(--ink-2);overflow-wrap:anywh
ere}
.chist-row
.r{display:block;font-family:var(--narrow);font-weight:700;font-size:19px;font-variant-numeric:tab
ular-nums}
@media print{#viewCalc{display:none !important}}

/* ---------- presupuestos a clientes ---------- */
.qline{border:1px solid
var(--rule);border-radius:var(--r);background:var(--panel-2);padding:10px;margin-bottom:8px}
.qline.off{opacity:.65;border-style:dashed}
```

```
.qrow3{display:grid;grid-template-columns:1fr 1.3fr
1.2fr;gap:8px;margin-top:8px;align-items:end}
.qsub{display:block;font-family:var(--narrow);font-size:16px;padding:8px 0}
.qint{font-size:12.5px;color:var(--ink-2);margin-top:6px}
.qint:empty{display:none}
.qint.bad{color:var(--cut);font-weight:600}
.qact{display:flex;flex-wrap:wrap;gap:6px;align-items:center;margin-top:8px}
.qact .btn{padding:5px 10px;font-size:13px}
.qact .check{flex:1 1 100%}
.qadd{display:flex;gap:8px;margin-top:8px}
.qadd select{flex:1;min-width:0}
.qhelp{margin-top:10px;font-size:13px;color:var(--ink-2)}
.qhelp ol{padding-left:18px;margin:6px 0 0}
.qhelp li{margin-bottom:4px}
#qNotes{width:100%;font:inherit;font-size:15px;padding:8px 10px;border:1px solid
var(--rule);border-radius:var(--r);background:var(--field);color:var(--ink);resize:vertical}
.qbtns{display:flex;flex-wrap:wrap;gap:8px;margin-top:14px}
.qbtns .btn{flex:1 1 140px}
.qdoc{background:var(--field);border:1px solid
var(--rule);border-radius:var(--r);padding:16px;font-size:14px;color:#143e45}
.qd-top{display:flex;justify-content:space-between;gap:12px;border-bottom:2px solid
#143e45;padding-bottom:8px;margin-bottom:10px}
.qd-top h3{margin:0;font-size:18px}
.qdoc small{color:#4a6b70}
.qdoc .tw{overflow-x:auto}
.qdoc table{width:100%;border-collapse:collapse}
.qdoc th{font-size:12px;text-align:left;color:#4a6b70;border-bottom:1px solid
#143e45;padding:4px}
.qdoc td{padding:6px 4px;border-bottom:1px solid #e3e3e3;vertical-align:top}
.qdoc .r{text-align:right}
.qdoc td.r{white-space:nowrap}
.qd-tot{margin:10px 0 0 auto;max-width:340px}
.qd-tot div{display:flex;justify-content:space-between;gap:12px;padding:3px 0}
.qd-tot .big{font-size:18px;font-weight:700;border-top:2px solid
#143e45;margin-top:4px;padding-top:6px}
.qd-h{font-weight:700;margin:14px 0 4px}
.qd-meta{margin:12px 0 0;color:#143e45}
.qd-notes{white-space:pre-line;font-size:12.5px;color:#2f5a61;margin:8px 0 0}
.qi{display:flex;justify-content:space-between;gap:10px;padding:6px 0;border-bottom:1px solid
var(--rule)}
.qrow{display:flex;align-items:center;flex-wrap:wrap;gap:6px;border:1px solid
var(--rule);border-radius:var(--r);padding:8px 10px;margin-bottom:6px;background:var(--panel)}
.qrow.cur{border-color:var(--accent);background:var(--accent-soft)}
```

```
.qmain{flex:1 1 170px;min-width:0}
.qmain b,.qmain small{display:block;overflow-wrap:anywhere}
.qmain small{color:var(--ink-2)}
.qrow select{width:auto;flex:0 0 auto}
.qrow .btn{min-width:36px;min-height:36px;padding:0 8px}
.qbanner{display:flex;gap:10px;align-items:center;flex-wrap:wrap;margin:12px 18px
0;padding:10px 12px;background:var(--accent-soft);border:1px solid
#8fb09b;border-radius:var(--r);color:var(--accent-d);font-size:14px}
.qbanner span{flex:1 1 220px}
@media (max-width:700px){#qNotes{font-size:16px}.qbanner{margin:10px 12px 0}}
@media print{.qdoc{border:none;padding:0;font-size:11pt;background:#fff}#viewQuote
.wrap{display:block}}

#metalCard[hidden],#oKgBox[hidden],#metalPick[hidden],#maxCutBox[hidden]{display:none}

/* ---------- diseño 3D ---------- */
#d3Canvas{display:block;width:100%;height:clamp(320px,50vh,540px);background:#edf3ee;bor
der:1px solid var(--rule);border-radius:var(--r);touch-action:none;cursor:grab}
#d3Canvas.mode-mover{cursor:move}
#d3Canvas.mode-regla,#d3Canvas.mode-angulo{cursor:crosshair}
.d3pad{padding:12px 16px 0}
.d3status{display:flex;gap:12px;flex-wrap:wrap;align-items:baseline;padding:8px 2px
12px;font-family:var(--narrow);font-size:13px;color:var(--ink-2)}
.d3status b{color:var(--ink);font-weight:600}
.d3modes{display:flex;gap:6px;flex-wrap:wrap}
.d3tramos{display:grid;grid-template-columns:repeat(3,1fr);gap:8px;margin-top:8px}
@media (max-width:480px){.d3tramos{grid-template-columns:1fr 1fr}}
.d3out{display:flex;gap:16px;flex-wrap:wrap;margin-top:10px;padding-top:10px;border-top:1px
dashed var(--rule);font-family:var(--narrow);font-size:13px;color:var(--ink-2)}
.d3out div{display:flex;flex-direction:column;gap:1px}
.d3out .k{font-size:12px;color:var(--ink-3)}
.d3out .v{font-size:16px;font-weight:700;font-variant-numeric:tabular-nums;color:var(--ink)}
.d3cutlist{width:100%;border-collapse:collapse;font-size:13.5px;margin-top:10px}
.d3cutlist
th{font-family:var(--narrow);text-align:left;font-size:12px;color:var(--ink-2);border-bottom:1px
solid var(--rule);padding:2px 6px 4px;white-space:nowrap}
.d3cutlist td{padding:3px 6px;border-bottom:1px solid
var(--rule-2);font-variant-numeric:tabular-nums}
.d3cutlist td.r,.d3cutlist th.r{text-align:right}
.d3warn{margin:0 0 12px;padding:8px 10px;border-radius:var(--r);border:1px dashed
var(--rule);color:var(--ink-3);font-family:var(--narrow);font-size:12.5px}
.d3warn b{color:var(--ink-2)}
.d3kbd{font-family:var(--narrow);background:var(--panel-2);border:1px solid
```

```
var(--rule);border-radius:3px;padding:0 4px;white-space:nowrap}
.d3tel-fit{display:flex;align-items:center;gap:8px;flex-wrap:wrap;padding:6px
0;border-bottom:1px solid var(--rule-2)}
.d3tel-fit .btn{padding:3px 9px;font-size:12.5px}
.d3sel-grid{display:grid;grid-template-columns:repeat(3,1fr);gap:8px;margin-top:8px}
.d3nudge{display:flex;gap:4px;flex-wrap:wrap;margin-top:8px}
.d3nudge .btn{padding:4px 8px;font-size:12.5px;min-width:38px}
.d3cat-row{border:1px solid
var(--rule);border-radius:var(--r);background:var(--panel-2);padding:8px
10px;margin-bottom:8px;display:grid;gap:8px}
.d3cat-row .head{display:flex;align-items:baseline;gap:8px;flex-wrap:wrap}
.d3cat-row .head b{font-size:14px}
.d3mini{font-family:var(--narrow);font-size:12.5px;color:var(--ink-3)}
@media (max-width:1040px){#d3Canvas{height:min(62vw,420px)}}

.d3canvas-wrap{position:relative;width:100%}
.d3viewcube{
  position:absolute;top:8px;right:8px;z-index:10;
  width:72px;height:72px;transform-style:preserve-3d;
  transform:rotateX(-30deg) rotateY(45deg);
  pointer-events:auto;transition:transform .35s ease;
}
.d3viewcube:hover{transform:rotateX(-30deg) rotateY(45deg) scale(1.05)}
.vc-face{
  position:absolute;width:48px;height:48px;
  display:flex;align-items:center;justify-content:center;
  font-family:var(--narrow);font-size:10px;font-weight:700;color:#143e45;
  background:rgba(208,224,211,.95);border:1px solid var(--rule);
  border-radius:4px;cursor:pointer;
  box-shadow:0 2px 6px rgba(20,62,69,.15);
  transition:background .15s ease,transform .15s ease;
}
.vc-face:hover{background:var(--accent-soft);transform:translateZ(4px)}
.vc-face span{pointer-events:none;text-align:center;line-height:1.1}
.vc-top{transform:rotateX(-90deg) translateZ(24px)}
.vc-bottom{transform:rotateX(90deg) translateZ(24px)}
.vc-front{transform:translateZ(24px)}
.vc-back{transform:rotateY(180deg) translateZ(24px)}
.vc-right{transform:rotateY(90deg) translateZ(24px)}
.vc-left{transform:rotateY(-90deg) translateZ(24px)}
.vc-corner{position:absolute;top:-6px;right:-6px;width:28px;height:28px;border-radius:50%;
  background:var(--accent);color:#fff;font-size:8px;font-weight:700;
  display:flex;align-items:center;justify-content:center;cursor:pointer;
```

```
  box-shadow:0 2px 6px rgba(20,62,69,.3);border:2px solid var(--panel)}
.vc-corner:hover{background:var(--accent-d);transform:scale(1.1)}
@media (max-width:480px){
  .d3viewcube{width:56px;height:56px;top:4px;right:4px}
  .vc-face{width:36px;height:36px;font-size:9px}
  .vc-top{transform:rotateX(-90deg) translateZ(18px)}
  .vc-bottom{transform:rotateX(90deg) translateZ(18px)}
  .vc-front{transform:translateZ(18px)}
  .vc-back{transform:rotateY(180deg) translateZ(18px)}
  .vc-right{transform:rotateY(90deg) translateZ(18px)}
  .vc-left{transform:rotateY(-90deg) translateZ(18px)}
  .vc-corner{width:22px;height:22px;font-size:7px;top:-4px;right:-4px}
}

@media (max-width:1040px){#d3Canvas{height:min(62vw,420px)}}

/* ---------- identidad: Acero y Escamas ---------- */
.topbar{background:var(--paper);border-bottom-color:rgba(20,62,69,.35)}
.ruler{background-color:var(--paper);background-image:repeating-linear-gradient(90deg,rgba(20
,62,69,.35) 0 1px,transparent 1px 8px),repeating-linear-gradient(90deg,var(--ink) 0
1px,transparent 1px 40px)}
.brand{align-items:center}
.brand span{color:var(--ink)}
.brand-logo{height:46px;width:auto;display:block}
.qd-brand{display:flex;align-items:center;gap:12px}
.qd-logo{height:64px;width:auto;display:block}
@media (max-width:700px){.brand-logo{height:40px}}

/* ---------- ajustes para celular (van al final para ganarle a las reglas base) ---------- */
@media (max-width:700px){
  .topbar{position:static}
  .topbar .inner{padding:8px 12px 10px;gap:8px}
  .brand{flex:1 1 100%}
  .brand span{display:none}
  .topbar .inner .btn{flex:1 1 0;padding:9px 4px;font-size:13px;line-height:1.2}
  .tabs{position:sticky;top:0;z-index:30;padding:0 4px;gap:0;border-bottom:1px solid var(--rule)}
  .tab{flex:1 1 0;text-align:center;padding:13px 2px;font-size:13.5px}
  .tab .long{display:none}
  .tab .short{display:inline}
  /* 16 px evita el zoom automático de iOS al enfocar un campo */
  input[type=text],input[type=number],input[type=url],select,td input[type=text],td
input[type=number]{font-size:16px}
}
```

```
@media (pointer:coarse){
  .x{padding:10px 12px}
  .grainbtn{padding:7px 12px;font-size:13px}
  .chip{padding:9px 15px}
  .btn{min-height:42px}
  .check input{width:22px;height:22px}
  .addrow .btn,.savebar .btn{min-height:46px}
  .tab{min-height:44px}
}

/* ---------- impresión ---------- */
.printhead{display:none}
@media print{
  @page{size:A4 portrait;margin:11mm}
  html,body{background:#fff}
  .topbar,.setup,.legend,.noprint{display:none !important}
  .wrap{display:block;padding:0;max-width:none}
  .card{border:none;box-shadow:none;background:#fff}
  .card + .card{margin-top:10mm}
  .planhead{padding:0 0 4mm;border-bottom:1px solid #000}
  .boards{padding:4mm 0 0;gap:8mm}
  .board svg{border:1px solid #444}
  .costs{page-break-inside:avoid}
  .costline{padding:4px 0;border-bottom:1px solid #ddd}
  .costline.total{background:#fff;color:#000;border-top:2px solid #000;padding:8px 0}
  .costline.total .k small{color:#555}
  .printhead{
    display:block;border-bottom:2px solid #000;padding-bottom:3mm;margin-bottom:5mm;
  }
  .printhead h1{font-size:17pt;margin-bottom:2mm}
  .printhead
.ph{font-family:var(--narrow);font-size:10pt;color:#333;display:flex;gap:14px;flex-wrap:wrap}
  .printhead .ph b{font-weight:700;color:#000}
  .printlist{display:block !important}
  .printlist table{font-size:9pt}
  .printlist thead th{border-bottom:1px solid #000}
  .printlist tbody td{border-bottom:1px solid #e3e3e3}
}
.printlist{display:none}

/* ---------- autoría / créditos ---------- */
.cred{border-top:1px solid var(--rule-2);background:var(--panel);color:var(--ink-2)}
.cred .cred-inner{max-width:1680px;margin:0 auto;display:flex;gap:8px
```

```
16px;flex-wrap:wrap;justify-content:space-between;padding:10px
18px;font-family:var(--narrow);font-size:12.5px}
.cred a{color:var(--accent-d);text-decoration:underline}
@media print{.cred{display:none !important}}

@media (prefers-reduced-motion:reduce){
  *{transition:none !important;animation:none !important}
}
</style>
</head>
<body>

<div class="topbar noprint">
  <div class="ruler" aria-hidden="true"></div>
  <div class="inner">
    <div class="brand">
      <img id="brandLogo" class="brand-logo" alt="Acero y Escamas">
      <h1>Calculadora de taller</h1>
      <span>plano de corte</span>
    </div>
    <span class="saved" id="saved" aria-live="polite"></span>
    <button class="btn quiet" id="btnDemo" type="button">Cargar ejemplo</button>
    <button class="btn quiet" id="btnReset" type="button">Empezar de cero</button>
    <button class="btn quiet" id="btnExport" type="button" title="Guarda el pedido en JSON para
resolverlo con el solver exacto CP-SAT (cpsat_corte.py)">Exportar JSON</button>
    <button class="btn primary" id="btnPrint" type="button">Imprimir plano</button>
  </div>
</div>

<div class="tabs noprint" role="tablist" aria-label="Vistas de la calculadora">
  <button class="tab" id="tabCorte" type="button" role="tab" aria-selected="true"
aria-controls="viewCorte"><span class="long">Plano de corte</span><span
class="short">Corte</span></button>
  <button class="tab" id="tabMat" type="button" role="tab" aria-selected="false"
aria-controls="viewMat"><span class="long">Cotizador de materiales</span><span
class="short">Materiales</span></button>
  <button class="tab" id="tabQuote" type="button" role="tab" aria-selected="false"
aria-controls="viewQuote"><span class="long">Presupuestos a clientes</span><span
class="short">Presupuestos</span></button>
  <button class="tab" id="tabCalc" type="button" role="tab" aria-selected="false"
aria-controls="viewCalc">Calculadora</button>
  <button class="tab" id="tabD3" type="button" role="tab" aria-selected="false"
aria-controls="viewD3"><span class="long">Diseño 3D</span><span
```

```
class="short">3D</span></button>
</div>

<div class="printhead" id="printhead"></div>
<div class="printlist" id="printlist"></div>

<div class="view" id="viewCorte">
<div class="wrap">

  <!-- ============================ COLUMNA DE CARGA
============================ -->
  <div class="setup">

    <section class="card">
      <h2><span class="n">1</span> <span id="ttlBoard">Tablero base</span></h2>
      <div class="body">
        <label style="margin-bottom:10px">
          <span class="lab">Material</span>
          <select id="material"><option value="wood">Madera / tablero</option><option
value="metal">Chapa metálica</option></select>
        </label>
        <div id="metalPick" hidden>
          <div class="grid g2" style="margin-bottom:8px">
            <label><span class="lab">Tipo de metal</span>
              <select id="metalType"><option value="acero">Acero / chapa negra</option><option
value="galva">Galvanizada</option><option value="inox">Inoxidable</option><option
value="alu">Aluminio</option></select></label>
            <label><span class="lab">Calibre</span>
              <select id="gauge"><option value="">Otro (a mano)</option><option value="0.56">24 ·
0,56 mm</option><option value="0.71">22 · 0,71 mm</option><option value="0.89">20 · 0,89
mm</option><option value="1.24">18 · 1,24 mm</option><option value="1.65">16 · 1,65
mm</option><option value="2.11">14 · 2,11 mm</option></select></label>
            <label><span class="lab">Medida de plancha</span>
              <select id="sheetPreset"><option value="">Otra (a mano)</option><option
value="2000x1000">1000 × 2000</option><option value="2440x1220">1220 ×
2440</option><option value="2500x1250">1250 × 2500</option><option
value="3000x1500">1500 × 3000</option></select></label>
            <label><span class="lab">El precio es</span>
              <select id="priceBy"><option value="sheet">Por plancha</option><option
value="kg">Por kg</option></select></label>
          </div>
          <p class="hint" style="margin:0 0 10px">Calibres y medidas son de referencia:
confirmalos con tu proveedor. Si no coinciden, escribí los valores abajo.</p>
```

```
      </div>
      <div class="grid g2">
        <label>
          <span class="lab" id="lblPrice">Precio del tablero</span>
          <input type="number" id="price" min="0" step="0.01" inputmode="decimal"
placeholder="0">
        </label>
        <label>
          <span class="lab" id="lblThick">Grosor del material (mm)</span>
          <input type="number" id="thick" min="0" step="0.5" inputmode="decimal"
placeholder="18">
        </label>
        <label>
          <span class="lab" id="lblBL">Largo del tablero (mm)</span>
          <input type="number" id="bl" min="1" step="1" inputmode="numeric">
        </label>
        <label>
          <span class="lab" id="lblBW">Ancho del tablero (mm)</span>
          <input type="number" id="bw" min="1" step="1" inputmode="numeric">
        </label>
      </div>
      <div class="readout">
        <div><span class="k" id="lblArea">Superficie del tablero</span><span class="v"
id="oArea">—</span></div>
        <div><span class="k">Precio por m²</span><span class="v"
id="oPerM2">—</span></div>
        <div id="oKgBox" hidden><span class="k">Peso de la plancha</span><span class="v"
id="oKg">—</span></div>
      </div>
    </div>
  </section>

  <section class="card" id="metalCard" hidden>
    <h2><span class="n">M</span> Cómo cortás la chapa</h2>
    <div class="body">
      <label><span class="lab">Método de corte</span>
        <select id="method"><option value="">Sin definir</option><option value="cizalla">Cizalla
/ guillotina</option><option value="amoladora">Amoladora</option><option
value="plasma">Plasma</option><option value="laser">Láser</option></select>
        <span class="hint" id="methodHint" style="display:block"></span></label>
      <label id="maxCutBox" style="margin-top:10px" hidden><span class="lab">Largo de la
cuchilla de la cizalla (mm)</span>
        <input type="number" id="maxCut" min="0" step="1" inputmode="numeric"
```

placeholder="Vacío = sin límite">
El plano solo usa cortes que entren en ese largo.</label>
<details class="qhelp"><summary>Extras: cargo mínimo, plegados y perforaciones</summary>

<label>Cargo mínimo del corte ($)<input type="number" id="minCharge" min="0" step="0.01" inputmode="decimal" placeholder="0"></label>

<label>Plegados (cantidad)<input type="number" id="bends" min="0" step="1" inputmode="numeric" placeholder="0"></label>
<label>Precio por plegado ($)<input type="number" id="bendPrice" min="0" step="0.01" inputmode="decimal" placeholder="0"></label>
<label>Perforaciones (cantidad)<input type="number" id="holes" min="0" step="1" inputmode="numeric" placeholder="0"></label>
<label>Precio por perforación ($)<input type="number" id="holePrice" min="0" step="0.01" inputmode="decimal" placeholder="0"></label>

</details>
<details class="qhelp"><summary>Calculadora: costo por metro cortando con amoladora</summary>

<label>Precio del disco ($)<input type="number" id="hDisc" min="0" step="0.01" inputmode="decimal"></label>
<label>Metros que rinde un disco<input type="number" id="hDiscM" min="0" step="0.1" inputmode="decimal"></label>
<label>Minutos por metro de corte<input type="number" id="hMin" min="0" step="0.1" inputmode="decimal"></label>
<label>Tu valor por hora ($)<input type="number" id="hRate" min="0" step="0.01" inputmode="decimal"></label>

<button class="btn quiet" id="hUse" type="button">Usar como servicio «por metro de corte»</button>
</details>
<details class="qhelp"><summary>¿Cómo se usa el modo chapa?</summary>

Arriba, en Material, elegí «Chapa metálica» y completá tipo de metal, calibre, medida de plancha y si el precio es por plancha o por kg.
Acá elegí tu método de corte: el disco se ajusta solo. Con cizalla, cargá el largo de la cuchilla.
Cargá las piezas en la sección 3: el plano se arma solo, y los costos muestran peso y metros de corte.

```
          <li>Con amoladora, usá la calculadora de costo por metro y tocá «Usar».</li>
          <li>Al final, «Pasar al cotizador» y después armás el presupuesto.</li>
        </ol>
      </details>
    </div>
  </section>

  <section class="card">
    <h2><span class="n">2</span> Corte y merma</h2>
    <div class="body">
      <label style="margin-bottom:10px">
        <span class="lab" id="lblCostMode">Cómo se cobra el material</span>
        <select id="costMode">
          <option value="used" id="optUsed">Solo la madera usada</option>
          <option value="board" id="optBoard">Tablero completo</option>
        </select>
        <span class="hint" id="costModeHint" style="display:block"></span>
      </label>
      <div class="grid g2">
        <label>
          <span class="lab">Grosor del disco (mm)</span>
          <input type="number" id="kerf" min="0" max="20" step="0.1" inputmode="decimal">
        </label>
        <label>
          <span class="lab">Merma adicional (%)</span>
          <input type="number" id="waste" min="0" max="100" step="0.5" inputmode="decimal">
        </label>
      </div>
      <div class="chips" style="margin-top:8px" role="group" aria-label="Merma sugerida">
        <button class="chip" type="button" data-waste="0">Sin merma</button>
        <button class="chip" type="button" data-waste="10">10 %</button>
        <button class="chip" type="button" data-waste="15">15 %</button>
      </div>

      <label class="check" style="margin-top:10px">
        <input type="checkbox" id="rotate">
        <span>Permitir rotar piezas 90°</span>
      </label>
      <p class="hint" id="hintRot">Aprovecha mejor la placa, pero cruza la veta. Podés bloquear la rotación pieza por pieza con el botón <b>veta</b> de la lista.</p>
      <label style="margin-top:12px">
        <span class="lab">Refilado (mm por lado)</span>
        <input type="number" id="trim" min="0" step="0.5" inputmode="decimal" placeholder="0">
```

```
        <span class="hint" style="display:block">Descuenta el borde irregular de la plancha. El plano muestra la medida útil, ya sin el refilado.</span>
      </label>

      <div class="grid g2" style="margin-top:12px">
        <label>
          <span class="lab">Servicio de corte ($)</span>
          <input type="number" id="service" min="0" step="0.01" inputmode="decimal" placeholder="0">
        </label>
        <label>
          <span class="lab">Se cobra</span>
          <select id="serviceMode">
            <option value="total">Por el pedido</option>
            <option value="board">Por tablero</option>
            <option value="cut">Por corte</option>
            <option value="meter">Por metro de corte</option>
          </select>
        </label>
      </div>
    </div>
  </section>

  <section class="card">
    <h2><span class="n">3</span> Piezas a cortar <small id="pieceCount"></small></h2>
    <div class="body">
      <div class="addrow">
        <label class="nm">
          <span class="lab">Nombre de la pieza</span>
          <input type="text" id="pName" placeholder="Lateral, estante, fondo…" autocomplete="off">
        </label>
        <label>
          <span class="lab">Largo</span>
          <input type="number" id="pL" min="1" step="1" inputmode="numeric" placeholder="mm">
        </label>
        <label>
          <span class="lab">Ancho</span>
          <input type="number" id="pW" min="1" step="1" inputmode="numeric" placeholder="mm">
        </label>
        <label>
```

```
          <span class="lab">Cant.</span>
          <input type="number" id="pQ" min="1" step="1" inputmode="numeric" value="1">
        </label>
        <div class="full" style="display:flex;gap:8px">
          <button class="btn primary" id="btnAdd" type="button" style="flex:1">Agregar
pieza</button>
          <button class="btn quiet" id="btnClearList" type="button">Vaciar lista</button>
        </div>
      </div>

      <div class="tablewrap">
        <table id="tbl">
          <thead>
            <tr>
              <th style="width:30px"></th>
              <th>Pieza</th>
              <th class="r" style="width:74px">Largo</th>
              <th class="r" style="width:74px">Ancho</th>
              <th class="r" style="width:62px">Cant.</th>
              <th style="width:54px">Veta</th>
              <th class="r" style="width:74px">m²</th>
              <th class="r" style="width:96px">Costo</th>
              <th style="width:64px"></th>
            </tr>
          </thead>
          <tbody id="tbody"></tbody>
        </table>
        <div class="empty" id="tblEmpty">Todavía no cargaste piezas. Agregá la primera arriba o
probá con el ejemplo.</div>
      </div>
    </div>
  </section>
 </div>

 <!-- ============================ COLUMNA DE SALIDA
============================ -->
 <div class="output">

  <section class="card">
    <div class="planhead">
      <h2><span class="n">4</span>Esquema de corte</h2>
      <div class="stats" id="planStats"></div>
    </div>
```

```
      <div id="notices"></div>
      <div class="boards" id="boards"></div>
      <div class="legend noprint">
        <span><i class="l-waste"></i>Desperdicio y recortes</span>
        <span><i class="l-keep"></i>Mayor recorte reutilizable</span>
        <span><i class="l-kerf"></i><span id="lblKerfLeg">Separación por disco de
sierra</span></span>
      </div>
    </section>

    <section class="card costs">
      <h2><span class="n">5</span> Costos del despiece</h2>
      <div class="body" id="costs"></div>
    </section>

  </div>
</div>
</div><!-- /viewCorte -->

<div class="view" id="viewMat" hidden>
<div class="qbanner noprint" id="qBanner" hidden><span>Estás armando una variante para un
presupuesto. Cuando termines, tocá «Actualizar mueble» y volvé.</span><button class="btn
primary" id="qBack" type="button">Volver al presupuesto</button></div>
<div class="wrap">

  <!-- ============================ COLUMNA DE CARGA
============================ -->
  <div class="setup">

    <section class="card">
      <h2><span class="n">1</span> Datos del pedido</h2>
      <div class="body">
        <label>
          <span class="lab">Nombre del mueble</span>
          <input type="text" id="mName" placeholder="Escritorio de oficina, mesa ratona…"
autocomplete="off">
        </label>
        <div class="grid g2" style="margin-top:10px">
          <label>
            <span class="lab">Cantidad a cotizar</span>
            <input type="number" id="mQty" min="1" step="1" inputmode="numeric" value="1">
          </label>
          <label>
```

```
          <span class="lab">Contingencia / imprevistos (%)</span>
          <input type="number" id="mWaste" min="0" max="100" step="0.5"
inputmode="decimal">
        </label>
       </div>
       <p class="hint">Cargá precios y medidas <b>de un solo mueble</b>; el pedido completo
se calcula multiplicando por la cantidad.</p>
     </div>
   </section>

   <section class="card">
     <h2><span class="n">2</span> Precio de venta</h2>
     <div class="body">
       <div class="grid g2">
         <label>
           <span class="lab">Margen deseado (%)</span>
           <input type="number" id="mMargin" min="0" max="95" step="1" value="40">
         </label>
         <label>
           <span class="lab">Se calcula</span>
           <select id="mMarginMode">
             <option value="markup">Sobre el costo</option>
             <option value="margin">Sobre el precio final</option>
           </select>
         </label>
       </div>
       <p class="hint" id="mMarginHint"></p>
     </div>
   </section>

   <section class="card">
     <h2><span class="n">3</span> Materiales <small id="mItemCount"></small></h2>
     <div class="body">
       <div id="mItems"></div>
       <p class="hint" id="mEmpty">Todavía no agregaste materiales. Sumá el primero abajo o
probá con el ejemplo del encabezado.</p>
       <button class="btn primary" id="btnMAdd" type="button"
style="width:100%;margin-top:4px">+ Agregar material</button>
     </div>
   </section>

   <div class="savebar">
     <button class="btn quiet" id="btnMNew" type="button">Nuevo mueble</button>
```

```
      <button class="btn primary" id="btnMSave" type="button">Guardar mueble</button>
    </div>
  </div>

  <!-- ============================ COLUMNA DE SALIDA
============================ -->
  <div class="output">
    <section class="card costs">
      <h2><span class="n">4</span> Cotización de materiales</h2>
      <div class="body" id="mCosts"></div>
    </section>
  </div>

</div>

<div class="wrap stack noprint">
  <section class="card">
    <h2><span class="n">5</span> Muebles guardados <small id="bookCount"></small></h2>
    <div class="body">
      <div id="bookList"></div>
      <p class="hint" id="bookEmpty">Todavía no guardaste ningún mueble. Armá uno arriba y
presioná “Guardar mueble”.</p>
    </div>
  </section>
</div>

</div><!-- /viewMat -->

<div class="view" id="viewQuote" hidden>
<div class="wrap">
  <div class="setup">
    <section class="card">
      <h2><span class="n">1</span> Cliente y pedido</h2>
      <div class="body">
        <div class="grid g2">
          <label><span class="lab">Cliente</span><input type="text" id="qClient"
autocomplete="off"></label>
          <label><span class="lab">WhatsApp del cliente</span><input type="text" id="qPhone"
inputmode="tel" placeholder="11 5555 1234"></label>
          <label><span class="lab">Validez (días)</span><input type="number" id="qValid"
min="0" step="1" inputmode="numeric"></label>
          <label><span class="lab">Plazo de entrega</span><input type="text" id="qDelivery"
placeholder="25 días hábiles"></label>
```

```
      </div>
    </div>
  </section>
  <section class="card">
    <h2><span class="n">2</span> Ítems</h2>
    <div class="body">
      <p class="hint" id="qEmpty">Agregá un mueble guardado como base, o una línea libre. Para otro material u otra medida, tocá «Armar variante».</p>
      <div id="qLines"></div>
      <div class="qadd"><select id="qPick" aria-label="Mueble guardado"></select><button class="btn primary" id="qAddFrom" type="button">+ Agregar</button></div>
      <button class="btn quiet" id="qAddFree" type="button" style="width:100%;margin-top:8px">+ Línea libre (flete, instalación, mano de obra…)</button>
      <details class="qhelp"><summary>¿Cómo cotizo otro material u otra medida?</summary>
        <ol>
          <li>Agregá el mueble base (por ejemplo, la mesa de pino) y anotá en «Lo que pidió» el material y las medidas.</li>
          <li>Tocá «Armar variante»: se copia el mueble y se abre en Materiales. Cambiá el tablero, las medidas y los precios (la madera nueva la calculás en Plano de corte → «Pasar al cotizador»).</li>
          <li>Tocá «Actualizar mueble» y después «Volver al presupuesto»: el precio se actualiza solo.</li>
        </ol>
      </details>
    </div>
  </section>
  <section class="card">
    <h2><span class="n">3</span> Condiciones y tu taller</h2>
    <div class="body">
      <div class="grid g2">
        <label><span class="lab">Descuento (%)</span><input type="number" id="qDisc" min="0" max="100" step="any" inputmode="decimal"></label>
        <label><span class="lab">Seña para empezar (%)</span><input type="number" id="qDep" min="0" max="100" step="any" inputmode="decimal"></label>
      </div>
      <label style="margin-top:10px"><span class="lab">Notas para el cliente</span><textarea id="qNotes" rows="3"></textarea></label>
      <div class="grid g2" style="margin-top:10px">
        <label><span class="lab">Nombre de tu taller</span><input type="text" id="qShop" autocomplete="off"></label>
        <label><span class="lab">Tu teléfono</span><input type="text" id="qShopPhone" inputmode="tel"></label>
      </div>
```

```
      <span class="hint" style="display:block">El nombre y el teléfono se guardan y salen en todos los presupuestos.</span>
     </div>
   </section>
  </div>
  <div style="display:grid;gap:18px;align-content:start;min-width:0">
   <section class="card">
    <div class="body">
     <div class="qdoc" id="qDoc"></div>
     <div class="qbtns noprint">
      <button class="btn primary" id="qWa" type="button">Abrir WhatsApp</button>
      <button class="btn quiet" id="qCopy" type="button">Copiar texto</button>
      <button class="btn quiet" id="qPrint" type="button">Imprimir / PDF</button>
      <button class="btn quiet" id="qSave" type="button">Guardar presupuesto</button>
      <button class="btn quiet" id="qNew" type="button">Nuevo</button>
     </div>
    </div>
   </section>
   <section class="card noprint">
    <h2>Para vos (no se manda)</h2>
    <div class="body" id="qInternal"></div>
   </section>
  </div>
</div>
<div class="wrap stack noprint">
  <section class="card">
   <h2>Presupuestos guardados</h2>
   <div class="body">
    <p class="hint" id="qBookSummary"></p>
    <p class="hint" id="qBookEmpty">Todavía no guardaste ninguno. Al guardar, queda numerado y podés marcar si lo enviaste, te lo aceptaron o lo rechazaron.</p>
    <div id="qBook"></div>
   </div>
  </section>
</div>
</div><!-- /viewQuote -->


<div class="view" id="viewCalc" hidden>
<div class="wrap">

  <div class="setup">
   <section class="card calc">
```

```
    <h2><span class="n">1</span> Calculadora</h2>
    <div class="body">
      <div class="calc-screen" aria-live="polite">
        <div class="calc-expr" id="cExpr"> </div>
        <div class="calc-res" id="cRes">0</div>
      </div>
      <div class="calc-actions">
        <button class="btn quiet" id="cCopy" type="button" title="Copia el número sin separador de miles y con punto decimal, listo para pegar en los campos">Copiar resultado</button>
      </div>
      <div class="calc-keys" id="cKeys">
        <button type="button" class="k clr" data-k="c"  aria-label="Borrar todo">C</button>
        <button type="button" class="k"     data-k="bk" aria-label="Borrar el último carácter">⌫</button>
        <button type="button" class="k op"  data-k="%"  aria-label="Porcentaje">%</button>
        <button type="button" class="k op"  data-k="/"  aria-label="Dividir">÷</button>

        <button type="button" class="k num" data-k="7">7</button>
        <button type="button" class="k num" data-k="8">8</button>
        <button type="button" class="k num" data-k="9">9</button>
        <button type="button" class="k op"  data-k="*"  aria-label="Multiplicar">×</button>

        <button type="button" class="k num" data-k="4">4</button>
        <button type="button" class="k num" data-k="5">5</button>
        <button type="button" class="k num" data-k="6">6</button>
        <button type="button" class="k op"  data-k="-"  aria-label="Restar">−</button>

        <button type="button" class="k num" data-k="1">1</button>
        <button type="button" class="k num" data-k="2">2</button>
        <button type="button" class="k num" data-k="3">3</button>
        <button type="button" class="k op"  data-k="+"  aria-label="Sumar">+</button>

        <button type="button" class="k"     data-k="paren" aria-label="Paréntesis">( )</button>
        <button type="button" class="k num" data-k="0">0</button>
        <button type="button" class="k num" data-k="." aria-label="Coma decimal">,</button>
        <button type="button" class="k eq"  data-k="eq" aria-label="Calcular">=</button>
      </div>
      <p class="hint">También podés usar el teclado. <b>50 + 10 %</b> suma el 10 % de 50 (da 55), igual que en un celular.</p>
    </div>
  </section>
</div>
```

```
  <div class="output">
    <section class="card">
      <h2><span class="n">2</span> Historial <small id="cHistCount"></small></h2>
      <div class="body">
        <div id="cHist"></div>
        <p class="hint" id="cHistEmpty">Todavía no hiciste ninguna cuenta. Tocá una cuenta del
historial para volver a usar su resultado.</p>
        <button class="btn quiet" id="cHistClear" type="button" style="margin-top:6px">Borrar
historial</button>
      </div>
    </section>
  </div>

</div>
</div><!-- /viewCalc -->

<div class="view" id="viewD3" hidden>
<div class="wrap">

  <!-- ============================== COLUMNA DE CARGA
============================== -->
  <div class="setup">

    <section class="card">
      <h2><span class="n">1</span> Agregar pieza</h2>
      <div class="body">
        <label style="margin-bottom:10px">
          <span class="lab">Tipo de pieza</span>
          <select id="d3Kind">
            <option value="madera">Tablero de madera</option>
            <option value="perfil">Perfil metálico (caño)</option>
            <option value="chapa">Chapa plana</option>
            <option value="plegada">Chapa plegada (L, U, C, Z, bandeja)</option>
          </select>
        </label>

        <div id="d3mBox">
          <div class="grid g2">
            <label><span class="lab">Nombre</span>
              <input type="text" id="d3mName" placeholder="Tapa, lateral, estante…"
autocomplete="off"></label>
            <label><span class="lab">Puesta</span>
              <select id="d3mAxis"><option value="x">Acostada (largo en X)</option><option
```

```
value="y">Parada (alto en Y)</option></select></label>
          <label><span class="lab">Largo (mm)</span><input type="number" id="d3mL" min="1"
step="1" inputmode="numeric" value="800"></label>
          <label><span class="lab">Ancho (mm)</span><input type="number" id="d3mW"
min="1" step="1" inputmode="numeric" value="600"></label>
          <label><span class="lab">Espesor (mm)</span><input type="number" id="d3mT"
min="1" step="0.5" inputmode="decimal" value="18"></label>
          <span></span>
        </div>
      </div>

      <div id="d3pBox" hidden>
        <div class="grid g2">
          <label><span class="lab">Perfil (catálogo)</span><select
id="d3pProf"></select></label>
          <label><span class="lab">Pared (mm)</span><select id="d3pWall"></select></label>
          <label><span class="lab">Largo (mm)</span><input type="number" id="d3pLen"
min="1" step="1" inputmode="numeric" value="700"></label>
          <label><span class="lab">Puesta</span>
            <select id="d3pAxis"><option value="y">Vertical (pata)</option><option
value="x">Horizontal a lo largo (X)</option><option value="z">Horizontal a lo ancho
(Z)</option></select></label>
        </div>
        <div id="d3CatList" style="margin-top:10px"></div>
      </div>

      <div id="d3cBox" hidden>
        <div class="grid g2">
          <label><span class="lab">Largo (mm)</span><input type="number" id="d3cL" min="1"
step="1" inputmode="numeric" value="500"></label>
          <label><span class="lab">Ancho (mm)</span><input type="number" id="d3cW" min="1"
step="1" inputmode="numeric" value="300"></label>
          <label><span class="lab">Espesor / calibre (mm)</span><input type="number"
id="d3cT" min="0.3" max="4" step="0.01" inputmode="decimal" value="1.2"></label>
          <span></span>
        </div>
      </div>

      <div id="d3fBox" hidden>
        <div class="grid g2">
          <label><span class="lab">Plantilla</span>
            <select id="d3fTmpl">
              <option value="L">L (angular plegado)</option>
```

```
              <option value="U">U (canal)</option>
              <option value="C">C (montante)</option>
              <option value="Z">Z (zócalo)</option>
              <option value="bandeja">Bandeja / caja</option>
            </select></label>
          <label><span class="lab">Espesor (mm)</span><input type="number" id="d3fT"
min="0.3" max="4" step="0.01" inputmode="decimal" value="1.2"></label>
          <label><span class="lab">Largo de la pieza (mm)</span><input type="number"
id="d3fL" min="1" step="1" inputmode="numeric" value="500"></label>
          <label><span class="lab">Radio interior (mm)</span><input type="number" id="d3fR"
min="0" step="0.1" inputmode="decimal" value="1.2"></label>
          <label><span class="lab">Factor K</span><input type="number" id="d3fK" min="0"
max="1" step="0.01" inputmode="decimal" value="0.4"></label>
          <span></span>
        </div>
        <div class="d3tramos" id="d3fTramos"></div>
        <div class="d3out" id="d3fOut"></div>
      </div>

      <button class="btn primary" id="d3Add" type="button"
style="width:100%;margin-top:10px">+ Agregar al diseño</button>
      <p class="hint">Se apoya en el piso, al centro de la vista. Después la movés con el mouse
(imán) o con los números de «Pieza seleccionada».</p>
    </div>
  </section>

  <section class="card">
    <h2><span class="n">2</span> Herramientas <small id="d3PieceCount"></small></h2>
    <div class="body">
      <div class="d3modes">
        <button class="chip" id="d3mMover" type="button" aria-pressed="true">Mover</button>
        <button class="chip" id="d3mCam" type="button" aria-pressed="false">Cámara</button>
        <button class="chip" id="d3mRegla" type="button" aria-pressed="false">Regla</button>
        <button class="chip" id="d3mAng" type="button"
aria-pressed="false">Transportador</button>
      </div>
      <label class="check" style="margin-top:8px"><input type="checkbox" id="d3Magnet"
checked><span>Imán: alinea mientras arrastrás</span></label>
      <div class="grid g2">
        <label><span class="lab">Captura del imán (mm)</span><input type="number"
id="d3Capture" min="1" max="50" step="1" inputmode="numeric" value="8"></label>
        <label><span class="lab">Rotación en saltos de</span>
          <select id="d3RotStep"><option value="0">Libre</option><option value="15"
```

```
selected>15°</option><option value="45">45°</option><option
value="90">90°</option></select></label>
        <label><span class="lab">Holgura telescópica (mm/lado)</span><input type="number"
id="d3Holgura" min="0" step="0.5" inputmode="decimal" value="1"></label>
        <label><span class="lab">Disco para cortar barras (mm)</span><input type="number"
id="d3KerfBarra" min="0" step="0.1" inputmode="decimal" value="3"></label>
        <label><span class="lab">Plegadora: largo máx. (mm)</span><input type="number"
id="d3PleMax" min="0" step="10" inputmode="numeric" placeholder="0 = sin límite"></label>
        <label><span class="lab">Cizalla: largo máx. (mm)</span><input type="number"
id="d3CizMax" min="0" step="10" inputmode="numeric" placeholder="0 = sin límite"></label>
      </div>
      <div class="d3modes" style="margin-top:10px">
        <button class="btn quiet d3view" type="button" data-v="frente">Frente</button>
        <button class="btn quiet d3view" type="button" data-v="lado">Lado</button>
        <button class="btn quiet d3view" type="button" data-v="arriba">Arriba</button>
        <button class="btn quiet d3view" type="button" data-v="iso">Isométrica</button>
        <button class="btn quiet d3view" type="button" data-v="fit">Ajustar</button>
        <button class="btn quiet d3view" type="button" data-v="in">+</button>
        <button class="btn quiet d3view" type="button" data-v="out">−</button>
      </div>
      <button class="btn quiet" id="d3ClearCotas" type="button" style="margin-top:10px">Borrar
mediciones</button>
      <p class="hint">En <b>Mover</b>: arrastrás una pieza por el piso; con <span
class="d3kbd">Shift</span> la subís o bajás. Arrastrar el fondo gira la cámara y la ruedita hace
zoom. <b>El imán solo ayuda mientras arrastrás</b>: la pieza no queda pegada ni atada, la
separás cuando quieras.</p>
    </div>
  </section>

  <section class="card">
    <h2><span class="n">3</span> Pieza seleccionada</h2>
    <div class="body" id="d3SelBox">
      <p class="empty">Nada seleccionado. Tocá una pieza en el visor.</p>
    </div>
  </section>

  <section class="card">
    <h2><span class="n">4</span> Telescopía</h2>
    <div class="body" id="d3TelBox">
      <p class="empty">Seleccioná un perfil para ver qué caños entran adentro.</p>
    </div>
  </section>
```

```
  </div>

  <!-- ============================ COLUMNA DE SALIDA
============================ -->
  <div class="output">

    <section class="card">
      <div class="planhead">
        <h2><span class="n">5</span> Visor 3D</h2>
        <div class="stats" id="d3Stats"></div>
      </div>
      <div class="d3pad">
        <div class="d3canvas-wrap">
          <canvas id="d3Canvas"></canvas>
          <div class="d3viewcube" id="d3ViewCube" aria-label="Selector de vista">
            <div class="vc-face vc-top" data-view="arriba" title="Arriba"><span>Arriba</span></div>
            <div class="vc-face vc-front" data-view="frente"
title="Frente"><span>Frente</span></div>
            <div class="vc-face vc-right" data-view="lado" title="Lado
derecho"><span>Lado</span></div>
            <div class="vc-face vc-back" data-view="atras" title="Atrás"><span>Atrás</span></div>
            <div class="vc-face vc-left" data-view="lado-izq" title="Lado
izquierdo"><span>Izq</span></div>
            <div class="vc-face vc-bottom" data-view="abajo"
title="Abajo"><span>Abajo</span></div>
            <div class="vc-corner vc-iso" data-view="iso"
title="Isométrica"><span>ISO</span></div>
          </div>
        </div>
        <div class="d3status" id="d3Status"></div>
      </div>
      <p class="d3warn" style="margin:0 16px 14px">⚠️ El diseñador ayuda a armar y
presupuestar: <b>no calcula resistencia estructural</b>. Las medidas y las uniones son
responsabilidad del proyectista.</p>
    </section>

    <section class="card">
      <h2><span class="n">6</span> Despiece y salida</h2>
      <div class="body">
        <div id="d3Summary"></div>
        <div class="tablewrap"><table class="d3cutlist" id="d3CutTable"></table></div>
        <div class="savebar noprint" style="flex-wrap:wrap">
          <button class="btn primary" id="d3ToCorte" type="button">Madera/chapa → Plano de
```

```
corte</button>
        <button class="btn primary" id="d3ToMat" type="button">Perfiles → Cotizador</button>
       </div>
       <div class="savebar noprint" style="flex-wrap:wrap;margin-top:8px">
        <button class="btn quiet" id="d3Exp" type="button">Exportar JSON</button>
        <button class="btn quiet" id="d3Imp" type="button">Importar JSON</button>
        <button class="btn quiet" id="d3ResetMarks" type="button" title="Vuelve a marcar todas
las piezas como no enviadas">Reenviar todo</button>
        <input type="file" id="d3ImpFile" accept=".json,application/json" hidden>
       </div>
       <p class="hint">Las piezas enviadas quedan marcadas para no duplicarlas. Si cambiás el
diseño y querés mandar todo de nuevo, tocá <b>Reenviar todo</b> antes.</p>
     </div>
   </section>

   <section class="card">
     <h2><span class="n">7</span> IA Foto → 3D · Tienda</h2>
     <div class="body">
       <label><span class="lab">Pegar JSON generado por IA</span>
         <textarea id="d3AiJson" rows="4"
style="width:100%;font-family:monospace;font-size:12px;padding:8px;border:1px solid
var(--rule);border-radius:var(--r);background:var(--field)" placeholder='{"name":"Isla Industrial
130x50x95","pieces":[...]}'></textarea>
       </label>
       <div class="savebar" style="margin-top:8px">
         <button class="btn primary" id="d3BtnLoadAi" type="button">⚡ Cargar JSON IA y
Costear</button>
       </div>
     </div>
   </section>

  </div>
</div>
</div><!-- /viewD3 -->

<script>
"use strict";
/*
==========================================================================
=
  1. ESTADO Y PERSISTENCIA

==========================================================================
```

```
= */
const KEY = "plano_corte_v1";
const MKEY = "cotizador_materiales_v1";
const TABKEY = "calc_tab_activa_v1";
const EPS = 1e-6;

const COLORS = [
  "#cddbc4","#e7d7a6","#c8d7e5","#e7c8ba","#d7c9e2","#bfdcd7",
  "#ead9c2","#d1dab6","#efcac1","#c5cfda","#dee4b8","#dbc8ae"
];

/* ícono de "duplicar" reutilizado en piezas, materiales y muebles guardados */
const DUP_SVG = '<svg viewBox="0 0 16 16" width="14" height="14" fill="none"
stroke="currentColor" stroke-width="1.5" stroke-linejoin="round" aria-hidden="true"><rect
x="2.2" y="2.2" width="8.2" height="8.2" rx="1"/><path d="M5.6 13.8h8.2V5.6"/></svg>';

let uidSeed = 1;
const uid = () => "p" + (uidSeed++) + "_" + Math.random().toString(36).slice(2, 7);

const defaults = () => ({
  board: { price: 0, length: 2600, width: 1830, thick: 18 },
  cut:   { kerf: 3, waste: 10, rotate: true, service: 0, serviceMode: "total", costMode: "used",
           material: "wood", metalType: "acero", priceBy: "sheet", method: "", maxCut: 0, trim: 0,
minCharge: 0,
           bends: 0, bendPrice: 0, holes: 0, holePrice: 0 },
  pieces: []
});

let state = defaults();
let last = null;          // último cálculo
let saveTimer = null;
let activeTab = "corte";

/* -------- cotizador de materiales (segunda pestaña) -------- */
const MBOOK_KEY = "muebles_guardados_v1";
const PRICE_STALE_DAYS = 30;

/* Sólo se aceptan enlaces http/https: evita guardar javascript: u otros
   esquemas que podrían ejecutarse al hacer clic. */
function safeUrl(u){
  const s = String(u || "").trim();
  if (!s) return "";
  if (!/^https?:\/\//i.test(s)) return "";
```

```
  try{ return new URL(s).href; }catch(e){ return ""; }
}
const todayISO = () => new Date().toISOString().slice(0, 10);
function daysSince(iso){
  if (!iso) return null;
  const t = Date.parse(iso + "T00:00:00");
  if (!isFinite(t)) return null;
  return Math.floor((Date.now() - t) / 86400000);
}
function hostOf(u){
  try{ return new URL(u).hostname.replace(/^www\./, ""); }catch(e){ return "enlace"; }
}

const numOr = (v, def, min, max) => {
  let n = parseFloat(v);
  if (!isFinite(n)) n = def;
  if (min !== undefined) n = Math.max(min, n);
  if (max !== undefined) n = Math.min(max, n);
  return n;
};

const matDefaults = () => ({ name:"", qty: 1, waste: 8, marginMode:"markup", marginPct:40,
items: [] });let mat = matDefaults();
let lastMat = null;
let matSaveTimer = null;
let matEditingId = null;      // id del mueble guardado que se está editando (si hay uno)

let book = { entries: [] };   // lista de muebles guardados
let bookSaveTimer = null;

const PRESETS = {
  tinte:   {yieldQty:12, yieldUnit:"m2", label:"Tinte para madera"},
  barniz:  {yieldQty:9,  yieldUnit:"m2", label:"Barniz / poliuretano"},
  pintura: {yieldQty:7,  yieldUnit:"m2", label:"Pintura / antióxido"},
  suelda:  {yieldQty:25, yieldUnit:"un", label:"Suelda con flux"},
  otro:    {yieldQty:1,  yieldUnit:"m2", label:"Otro consumible"}
};

function matItemDefaults(kind){
  const base = {name:"", price:0, url:"", priceDate:""};
  if (kind === "tablero")    return Object.assign({kind}, base, {l:0, w:0});
  if (kind === "barra")      return Object.assign({kind}, base, {barLen:6000, need:0});
  if (kind === "consumible") return Object.assign({kind}, base, {consumo:0, preset:"tinte"},
```

```
PRESETS.tinte);
  return Object.assign({kind:"directo"}, base, {unitQty:0});
}
function sanitizeMatItem(it){
  const kind = ["tablero","barra","consumible","directo"].includes(it.kind) ? it.kind : "tablero";
  const merged = Object.assign(matItemDefaults(kind), it, {kind});
  merged.id = it.id || uid();
  merged.name = String(it.name || "");
  merged.url = safeUrl(it.url);
  merged.priceDate = String(it.priceDate || "");
  return merged;
}

function load(){
  try{
    const raw = localStorage.getItem(KEY);
    if (!raw) return;
    const d = JSON.parse(raw);
    if (d && d.board && d.cut && Array.isArray(d.pieces)){
      state = {
        board: Object.assign(defaults().board, d.board),
        cut:   Object.assign(defaults().cut, d.cut),
        pieces: d.pieces.map(p => ({
          id: p.id || uid(),
          name: String(p.name || "Pieza"),
          l: +p.l || 0, w: +p.w || 0,
          qty: Math.max(1, Math.round(+p.qty) || 1),
          grain: !!p.grain
        }))
      };
    }
  }catch(e){ /* almacenamiento no disponible: se sigue en memoria */ }
}
function save(){
  clearTimeout(saveTimer);
  saveTimer = setTimeout(() => {
    try{
      localStorage.setItem(KEY, JSON.stringify(state));
      flag("Guardado");
    }catch(e){
      flag("No se pudo guardar en este navegador");
    }
  }, 250);
```

```
}
function flag(txt){
  const el = document.getElementById("saved");
  el.textContent = txt;
  clearTimeout(flag._t);
  flag._t = setTimeout(() => { el.textContent = ""; }, 1600);
}

function loadMat(){
  try{
    const raw = localStorage.getItem(MKEY);
    if (!raw) return;
    const d = JSON.parse(raw);
    if (d && Array.isArray(d.items)){
      mat = {
        name: String(d.name || ""),
        qty: Math.round(numOr(d.qty, 1, 1)),
        waste: numOr(d.waste, 0, 0),
        marginMode: d.marginMode === "margin" ? "margin" : "markup",
        marginPct: numOr(d.marginPct, 40, 0, 99),
        items: d.items.map(sanitizeMatItem)
      };
      matEditingId = d.editingId || null;
    }
  }catch(e){ /* se sigue en memoria */ }
}
function saveMat(){
  clearTimeout(matSaveTimer);
  matSaveTimer = setTimeout(() => {
    try{
      localStorage.setItem(MKEY, JSON.stringify(Object.assign({}, mat, {editingId:
matEditingId})));
      flag("Guardado");
    }
    catch(e){ flag("No se pudo guardar en este navegador"); }
  }, 250);
}

function loadBook(){
  try{
    const raw = localStorage.getItem(MBOOK_KEY);
    if (!raw) return;
    const d = JSON.parse(raw);
```

```
    if (d && Array.isArray(d.entries)){
      book.entries = d.entries.map(e => ({
        id: e.id || uid(),
        name: String(e.name || "Mueble sin nombre"),
        qty: Math.round(numOr(e.qty, 1, 1)),
        waste: numOr(e.waste, 0, 0),
        marginMode: e.marginMode === "margin" ? "margin" : "markup",
        marginPct: numOr(e.marginPct, 40, 0, 99),
        items: Array.isArray(e.items) ? e.items.map(sanitizeMatItem) : []
      }));
    }
  }catch(e){ /* se sigue en memoria */ }
}
function saveBook(){
  clearTimeout(bookSaveTimer);
  bookSaveTimer = setTimeout(() => {
    try{ localStorage.setItem(MBOOK_KEY, JSON.stringify(book)); flag("Guardado"); }
    catch(e){ flag("No se pudo guardar en este navegador"); }
  }, 250);
}

/*
================================================================================
  2. FORMATO

================================================================================ */
const nf = (d, x) => new Intl.NumberFormat("es-AR", {minimumFractionDigits:d,
maximumFractionDigits:(x===undefined?d:x)});
const f0 = nf(0), f1 = nf(1), f2 = nf(2), f3 = nf(3);
const money = n => "$ " + f2.format(isFinite(n) ? n : 0);
const m2 = n => f3.format(isFinite(n) ? n : 0) + " m²";
const pct = n => f1.format(isFinite(n) ? n : 0) + " %";
const mmv = n => f0.format(Math.round(isFinite(n) ? n : 0));

/*
================================================================================
  3. ALGORITMO DE EMPAQUETADO 2D
  Guillotina con lista de rectángulos libres. Todos los cortes son pasantes
  dentro de cada rectángulo, así que el plano es cortable en una sierra
  escuadradora o de panel. El disco (kerf) se descuenta en cada división.
```

Se prueban varias heurísticas y se elige el mejor resultado.

```
================================================================================
= */

/* Un corte de guillotina atraviesa el rectángulo libre entero. Devuelve la orientación elegida y el
largo
   de corte (mm); si alguna pasada supera el largo máximo (cuchilla de la cizalla) prueba la otra
orientación. */
function splitOf(f, pw, ph, kerf, rule, M){
  const rw = f.w - pw - kerf, rh = f.h - ph - kerf;
  let horizontal;
  if (rule === "shortAxis")      horizontal = (f.w <= f.h);
  else if (rule === "longAxis")  horizontal = (f.w >  f.h);
  else if (rule === "shortLeft") horizontal = (rw <= rh);
  else                           horizontal = (rw >  rh);
  const needV = f.w - pw > EPS, needH = f.h - ph > EPS;
  const lens = h => h ? {v: needV ? ph : 0, h: needH ? f.w : 0} : {v: needV ? f.h : 0, h: needH ? pw
: 0};
  let l = lens(horizontal);
  if (Math.max(l.v, l.h) > M + EPS){
    horizontal = !horizontal; l = lens(horizontal);
    if (Math.max(l.v, l.h) > M + EPS) return null;
  }
  return {horizontal, len: l.v + l.h};
}

function packBoard(items, BL, BW, kerf, rule, maxCut, mode){
  const M = maxCut > 0 ? maxCut : Infinity;
  const firstFit = (mode === "first");   // "first" = FFD: primer rectángulo libre que alcanza, de
mayor a menor
  let cutLen = 0;   // largo total de corte de esta placa (mm)
  const free = [{x:0, y:0, w:BL, h:BW}];
  const placed = [];
  const left = [];
  let cuts = 0;   // pasadas de sierra reales: una por cada lado donde la pieza no coincide con el
rectángulo libre

  for (const it of items){
    let best = null;

    for (let i = 0; i < free.length; i++){
      const f = free[i];
```

```
    const opts = it.rot ? [[it.l, it.w, false], [it.w, it.l, true]] : [[it.l, it.w, false]];
    for (const o of opts){
      const pw = o[0], ph = o[1];
      if (pw > f.w + EPS || ph > f.h + EPS) continue;
      const sp = splitOf(f, pw, ph, kerf, rule, M);
      if (!sp) continue;                          // exigiría un corte más largo que la cuchilla
      if (firstFit){ best = {i, pw, ph, rotated:o[2], score:0, tie:0, sp}; break; }
      const score = f.w * f.h - pw * ph;          // menor sobra de área
      const tie   = Math.min(f.w - pw, f.h - ph);   // menor sobra en el lado corto
      if (!best || score < best.score - EPS ||
          (Math.abs(score - best.score) < EPS && tie < best.tie)){
        best = {i, pw, ph, rotated:o[2], score, tie, sp};
      }
    }
    if (firstFit && best) break;
  }

  if (!best){ left.push(it); continue; }

  const f = free[best.i];
  placed.push({
    id:it.id, name:it.name, color:it.color, key:it.key,
    x:f.x, y:f.y, w:best.pw, h:best.ph, rotated:best.rotated
  });
  free.splice(best.i, 1);

  if (f.w - best.pw > EPS) cuts++;   // corte vertical
  if (f.h - best.ph > EPS) cuts++;   // corte horizontal

  const rw = f.w - best.pw - kerf;   // sobra a la derecha, ya sin el disco
  const rh = f.h - best.ph - kerf;   // sobra abajo, ya sin el disco

  const horizontal = best.sp.horizontal;
  cutLen += best.sp.len;

  if (horizontal){
    if (rw > EPS) free.push({x:f.x + best.pw + kerf, y:f.y, w:rw, h:best.ph});
    if (rh > EPS) free.push({x:f.x, y:f.y + best.ph + kerf, w:f.w, h:rh});
  }else{
    if (rw > EPS) free.push({x:f.x + best.pw + kerf, y:f.y, w:rw, h:f.h});
    if (rh > EPS) free.push({x:f.x, y:f.y + best.ph + kerf, w:best.pw, h:rh});
  }
}
```

```
  return {placed, left, free, cuts, cutLen};
}

function packAll(items, BL, BW, kerf, rule, maxCut, mode){
  const boards = [];
  let rest = items;
  let guard = 0;
  while (rest.length && guard++ < 400){
    const r = packBoard(rest, BL, BW, kerf, rule, maxCut, mode);
    if (!r.placed.length) break;              // nada entra: se corta el bucle
    boards.push(r);
    rest = r.left;
  }
  return {boards, unplaced: rest};
}

const SORTS = {
  area:     (a,b) => (b.l*b.w) - (a.l*a.w),
  maxSide:  (a,b) => Math.max(b.l,b.w) - Math.max(a.l,a.w) || (b.l*b.w) - (a.l*a.w),
  largo:    (a,b) => (b.l - a.l) || (b.w - a.w),
  ancho:    (a,b) => (b.w - a.w) || (b.l - a.l),
  perim:    (a,b) => (b.l + b.w) - (a.l + a.w)
};
const RULES = ["shortAxis", "longAxis", "shortLeft", "longLeft"];

function biggestFree(board){
  let m = 0, rect = null;
  for (const f of board.free){
    const a = f.w * f.h;
    if (a > m){ m = a; rect = f; }
  }
  return rect;
}

/* Mayor rectángulo vacío real de la placa: barre cada franja vertical
   delimitada por bordes de piezas y busca el hueco más alto que queda libre.
   Es lo que a un carpintero le interesa guardar como recorte reutilizable. */
function largestEmptyRect(placed, BL, BW, minSide){
  minSide = minSide || 0;
  if (!placed.length) return {x:0, y:0, w:BL, h:BW};
  if (placed.length > 140) return null;

  const xsL = new Set([0]), xsR = new Set([BL]);
```

```
  for (const p of placed){ xsL.add(p.x + p.w); xsR.add(p.x); }
  const L = [...xsL].filter(v => v >= -EPS && v < BL).sort((a,b) => a-b);
  const R = [...xsR].filter(v => v >  EPS && v <= BL).sort((a,b) => a-b);

  let best = null, bestA = 0;
  for (const x1 of L){
    for (const x2 of R){
      const w = x2 - x1;
      if (w < minSide - EPS || w <= EPS || w * BW <= bestA) continue;
      const blocks = [];
      for (const p of placed){
        if (p.x < x2 - EPS && p.x + p.w > x1 + EPS) blocks.push([p.y, p.y + p.h]);
      }
      blocks.sort((a,b) => a[0] - b[0]);
      let y = 0;
      for (let i = 0; i <= blocks.length; i++){
        const top = (i < blocks.length) ? blocks[i][0] : BW;
        const gap = top - y;
        if (gap >= minSide - EPS && gap > EPS && w * gap > bestA){ bestA = w * gap; best =
{x:x1, y:y, w:w, h:gap}; }
        if (i < blocks.length) y = Math.max(y, blocks[i][1]);
      }
    }
  }
  return best;
}

/* Recorte que vale la pena marcar en el plano */
function usefulRemnant(board, BL, BW){
  if (board._rem !== undefined) return board._rem;
  const r = largestEmptyRect(board.placed, BL, BW, 120) || biggestFree(board);
  board._rem = (r && r.w * r.h >= 50000 && Math.min(r.w, r.h) >= 120) ? r : null;
  return board._rem;
}

function optimizeCore(instances, BL, BW, kerf, maxCut){
  const heavy = instances.length > 350;
  const sortKeys = heavy ? ["area","maxSide"] : Object.keys(SORTS);
  const placeModes = heavy ? ["best"] : ["best","first"];   // "first" = FFD: de mayor a menor
  const rules = heavy ? ["shortLeft","longLeft"] : RULES;

  let best = null;
  for (const sk of sortKeys){
```

```
    const sorted = instances.slice().sort(SORTS[sk]);
    for (const pm of placeModes){
      for (const rule of rules){
        const res = packAll(sorted, BL, BW, kerf, rule, maxCut, pm);
        const lastB = res.boards[res.boards.length - 1];
        const remnant = lastB ? (biggestFree(lastB) ? biggestFree(lastB).w * biggestFree(lastB).h :
0) : 0;
        const better =
          !best ||
          res.unplaced.length < best.res.unplaced.length ||
          (res.unplaced.length === best.res.unplaced.length && res.boards.length <
best.res.boards.length) ||
          (res.unplaced.length === best.res.unplaced.length && res.boards.length ===
best.res.boards.length && remnant > best.remnant);
        if (better) best = {res, remnant, sk, rule, pm};
      }
    }
  }
  return best ? best.res : {boards:[], unplaced:instances};
}

/* Búsqueda de vecindario (LNS liviano): re-empaqueta los tableros ya llenos
   para ver si las piezas entran en menos tableros. Si entra todo, el tablero
   sobrante se elimina y baja el costo de material. Solo en trabajos chicos,
   porque re-corre el tablero de heurísticas por cada intento. */
function squeezeBoards(res, instances, BL, BW, kerf, maxCut){
  if (!res.boards.length || instances.length > 200 || res.boards.length > 10) return res;
  for (let b = res.boards.length - 1; b >= 1; b--){
    const subset = [];
    for (let i = 0; i <= b; i++){
      for (const p of res.boards[i].placed){
        if (p.key !== undefined && instances[p.key]) subset.push(instances[p.key]);
      }
    }
    if (!subset.length) continue;
    const re = optimizeCore(subset, BL, BW, kerf, maxCut);
    if (!re.unplaced.length && re.boards.length <= b){
      return { boards: re.boards.concat(res.boards.slice(b + 1)), unplaced: res.unplaced };
    }
  }
  return res;
}
```

```
function optimize(instances, BL, BW, kerf, maxCut){
  instances.forEach((it, i) => { if (it.key === undefined) it.key = i; });
  const res = optimizeCore(instances, BL, BW, kerf, maxCut);
  return squeezeBoards(res, instances, BL, BW, kerf, maxCut);
}

/*
=========================================================================
=
   4. CÁLCULO

=========================================================================
= */
function compute(){
  const metal = state.cut.material === "metal";
  const trim = Math.max(0, state.cut.trim || 0);
  const BL0 = state.board.length, BW0 = state.board.width;          // medida real de la plancha
  const BL = Math.max(0, BL0 - 2 * trim), BW = Math.max(0, BW0 - 2 * trim);   // medida útil, ya
sin refilado
  const kerf = Math.max(0, state.cut.kerf || 0);
  const maxCut = (metal && state.cut.method === "cizalla") ? Math.max(0, state.cut.maxCut || 0)
: 0;
  const boardArea = (BL0 * BW0) / 1e6;
  const kgM2 = metal ? (state.board.thick || 0) * (METAL_DENS[state.cut.metalType] || 7.85) : 0;
// kg por m²
  const sheetKg = boardArea * kgM2;
  const sheetPrice = (metal && state.cut.priceBy === "kg") ? sheetKg * (state.board.price || 0) :
(state.board.price || 0);
  const perM2 = boardArea > 0 ? sheetPrice / boardArea : 0;

  const tooBig = [];
  const instances = [];
  state.pieces.forEach((p, idx) => {
    if (!(p.l > 0 && p.w > 0 && p.qty > 0)) return;
    const rot = !!state.cut.rotate && !p.grain;
    const fitsNormal = p.l <= BL + EPS && p.w <= BW + EPS;
    const fitsRot    = rot && p.w <= BL + EPS && p.l <= BW + EPS;
    if (!fitsNormal && !fitsRot){ tooBig.push(p); return; }
    for (let i = 0; i < p.qty; i++){
      instances.push({id:p.id, name:p.name, l:p.l, w:p.w, rot, color:COLORS[idx %
COLORS.length]});
    }
  });
```

```
  const res = (BL > 0 && BW > 0 && instances.length)
    ? optimize(instances, BL, BW, kerf, maxCut)
    : {boards:[], unplaced:[]};

  const nBoards = res.boards.length;
  const usedArea = res.boards.reduce((s,b) => s + b.placed.reduce((t,p) => t + p.w*p.h, 0), 0) /
1e6;
  const consumed = nBoards * boardArea;
  const wasteArea = Math.max(0, consumed - usedArea);
  const useRate = consumed > 0 ? (usedArea / consumed) * 100 : 0;

  const boardsCost = nBoards * sheetPrice;
  const netCost    = usedArea * perM2;
  const offcutCost = wasteArea * perM2;
  // "used": se cobra solo la madera que ocupan las piezas (el sobrante se reutiliza).
  // "board": se cobra cada tablero entero que se consume.
  const costMode = state.cut.costMode === "board" ? "board" : "used";
  const woodCost = costMode === "used" ? netCost : boardsCost;
  const extraWaste = woodCost * ((state.cut.waste || 0) / 100);

  // cortes reales de guillotina (pasadas de sierra), no una aproximación por pieza
  const cuts = res.boards.reduce((s,b) => s + (b.cuts || 0), 0);
  const cutM = res.boards.reduce((s,b) => s + (b.cutLen || 0), 0) / 1000;   // metros lineales de
corte
  const sMode = state.cut.serviceMode;
  const serviceRaw = (state.cut.service || 0) * (sMode === "board" ? nBoards : sMode === "cut"
? cuts : sMode === "meter" ? cutM : 1);
  const minCharge = metal ? (state.cut.minCharge || 0) : 0;
  const service = nBoards ? Math.max(serviceRaw, minCharge) : serviceRaw;
  const minApplied = nBoards > 0 && minCharge > serviceRaw;
  const extras = metal ? (state.cut.bends || 0) * (state.cut.bendPrice || 0) + (state.cut.holes || 0) *
(state.cut.holePrice || 0) : 0;

  const total = woodCost + extraWaste + service + extras;
  const costPerUsefulM2 = usedArea > 0 ? total / usedArea : 0;

  // valor que se pasa al cotizador: madera + corte. La merma se omite a propósito
  // porque el cotizador ya suma su propia contingencia sobre todo el mueble.
  const woodForQuote = woodCost + service + extras;

  return {
    BL, BW, kerf, boardArea, perM2, instances, tooBig, res, nBoards, cuts,
```

```
    usedArea, consumed, wasteArea, useRate, costMode, woodCost, woodForQuote,
    boardsCost, netCost, offcutCost, extraWaste, service, total, costPerUsefulM2,
    metal, trim, BL0, BW0, maxCut, sheetPrice, sheetKg, kgM2, cutM, serviceRaw, minApplied,
extras,
    usedKg: usedArea * kgM2, totalKg: consumed * kgM2, wasteKg: wasteArea * kgM2
  };
}

/* Exporta el pedido actual como JSON para resolverlo con el solver exacto
   (CP-SAT de Google OR-Tools) fuera del navegador. En una página estática el
   solver no puede correr acá, así que el flujo es: 1) «Exportar JSON»,
   2) correr cpsat_corte.py en una compu con Python, 3) comparar el mínimo
   de tableros con el plano que armó la app. */
function exportJob(){
  const data = {
    board: { length: +state.board.length || 0, width: +state.board.width || 0,
             thickness: +state.board.thick || 0, price: +state.board.price || 0 },
    cut:   { kerf: +state.cut.kerf || 0, trim: +state.cut.trim || 0, rotate: !!state.cut.rotate },
    pieces: state.pieces.map(p => ({
      name: p.name, l: +p.l || 0, w: +p.w || 0,
      qty: Math.max(1, Math.round(+p.qty || 1)), grain: !!p.grain
    })),
    app: (last && last.res) ? { boards: last.res.boards.length } : {}
  };
  const blob = new Blob([JSON.stringify(data, null, 2)], {type: "application/json"});
  const a = document.createElement("a");
  a.href = URL.createObjectURL(blob);
  a.download = "pedido-corte.json";
  document.body.appendChild(a);
  a.click();
  a.remove();
  setTimeout(() => URL.revokeObjectURL(a.href), 1000);
  flag("Pedido exportado como JSON");
}

/*
================================================================================
=
   4.b COTIZADOR DE MATERIALES (segunda pestaña)
   Cada ítem se costea con la lógica propia de su naturaleza: un tablero se
   paga por pieza, un perfil se compra en barras comerciales enteras (se
   redondea para arriba), y un consumible se paga según cuánto rinde el
   envase. Todo se multiplica por la cantidad de muebles del pedido.
```

```
==========================================================================
= */
function kindLabel(k){
  return k === "tablero" ? "Tablero / placa"
       : k === "barra"   ? "Barra / perfil metálico"
       : k === "consumible" ? "Consumible"
       : "Herraje / directo";
}

function matItemCost(it, qty){
  if (it.kind === "tablero"){
    const l = +it.l || 0, w = +it.w || 0, price = +it.price || 0;
    const area = (l * w) / 1e6;
    const perM2 = area > 0 ? price / area : 0;
    const note = area > 0 ? `${m2(area)} por mueble · ${money(perM2)}/m²` : "";
    return {unit: price, total: price * qty, note};
  }
  if (it.kind === "barra"){
    const price = +it.price || 0, barLen = +it.barLen || 0, need = +it.need || 0;
    const kerf = Math.max(0, +it.kerf || 0);
    const totalNeed = need * qty;
    let bars = 0;
    const cuts = Array.isArray(it.cuts) ? it.cuts.map(c => Math.max(0, +c || 0)).filter(c => c > 0) :
null;
    if (barLen > 0 && cuts && cuts.length){
      /* acomodo de cortes por barra (FFD 1D) descontando el disco de cada corte */
      const items = [];
      for (let k = 0; k < Math.max(1, Math.round(qty)); k++) for (const c of cuts) items.push(c);
      items.sort((a, b) => b - a);
      const bins = [];
      for (const c of items){
        let ok = false;
        for (let i = 0; i < bins.length; i++){
          if (bins[i] + c + kerf <= barLen + EPS){ bins[i] += c + kerf; ok = true; break; }
        }
        if (!ok) bins.push(c + kerf);
      }
      bars = bins.length;
    } else {
      bars = barLen > 0 ? Math.ceil(totalNeed / barLen - EPS) : 0;
    }
    const total = bars * price;
```

```
    const leftover = barLen > 0 ? Math.max(0, bars * barLen - totalNeed) : 0;
    const note = barLen > 0
      ? `${f0.format(bars)} barra${bars === 1 ? "" : "s"} de ${mmv(barLen)} mm · sobrante
${mmv(leftover)} mm${kerf > 0 ? ` · disco ${mmv(kerf)} mm` : ""}`
      : "";
    return {unit: qty > 0 ? total / qty : 0, total, note};
  }
  if (it.kind === "consumible"){
    const price = +it.price || 0, y = +it.yieldQty || 0, consumo = +it.consumo || 0;
    const perUnitCost = y > 0 ? (price / y) * consumo : 0;
    const unitTxt = it.yieldUnit === "un" ? "uniones" : "m²";
    const note = y > 0
      ? `rinde ${f1.format(y)} ${unitTxt}/envase · ${f1.format(consumo)} ${unitTxt} por mueble`
      : "";
    return {unit: perUnitCost, total: perUnitCost * qty, note};
  }
  // directo: herrajes, tornillería, bisagras…
  const price = +it.price || 0, q = +it.unitQty || 0;
  const unit = price * q;
  const note = q > 0 ? `${f0.format(q)} × ${money(price)} por mueble` : "";
  return {unit, total: unit * qty, note};
}

function suggestedPrice(cost, mode, pct){
  const p = Math.max(0, Math.min(99, pct || 0));
  if (mode === "margin") return p >= 100 ? cost : cost / (1 - p / 100);
  return cost * (1 + p / 100);
}

/* Cálculo compartido: sirve tanto para el mueble que se está editando (mat)
   como para cada entrada ya guardada en la lista, así el número siempre
   sale de una única fórmula. */
function computeEntryTotals(entry){
  const qty = Math.max(1, Math.round(entry.qty || 1));
  const items = (entry.items || []).map(it => Object.assign({}, it, matItemCost(it, qty)));
  const subtotal = items.reduce((s, it) => s + it.total, 0);
  const wasteCost = subtotal * ((entry.waste || 0) / 100);
  const total = subtotal + wasteCost;
  const perUnit = qty > 0 ? total / qty : 0;
  const price = suggestedPrice(perUnit, entry.marginMode, entry.marginPct);
  const profit = price - perUnit;
  return {qty, items, subtotal, wasteCost, total, perUnit, price, profit};
}
```

```
function computeMat(){ return computeEntryTotals(mat); }

/*
================================================================================
   5. DIBUJO DEL PLANO (SVG a escala)

================================================================================ */
function esc(s){ return String(s).replace(/[&<>"]/g, c =>
({"&":"&amp;","<":"&lt;",">":"&gt;",'"':"&quot;"}[c])); }

function boardSVG(board, BL, BW, kerf){
  const big = Math.max(BL, BW);
  const m = big * 0.062;                  // margen para acotaciones
  const sw = big / 1100;                  // grosor de línea base
  const fs = big / 52;                   // tipografía base
  const hatch = big / 95;
  const p = [];

  p.push(`<svg viewBox="${(-m).toFixed(1)} ${(-m).toFixed(1)} ${(BL + m*2).toFixed(1)} ${(BW + m*2).toFixed(1)}" role="img" aria-label="Plano de corte del tablero">`);
  p.push(`<defs><pattern id="h${board.uidx}" width="${hatch}" height="${hatch}" patternUnits="userSpaceOnUse" patternTransform="rotate(45)">
    <rect width="${hatch}" height="${hatch}" fill="#eef0e8"/>
    <line x1="0" y1="0" x2="0" y2="${hatch}" stroke="#c0c7bc" stroke-width="${sw*1.2}"/>
  </pattern></defs>`);

  // placa + desperdicio
  p.push(`<rect x="0" y="0" width="${BL}" height="${BW}" fill="url(#h${board.uidx})" stroke="#143e45" stroke-width="${sw*2.2}"/>`);

  // mayor recorte reutilizable
  const rem = usefulRemnant(board, BL, BW);
  if (rem){
    p.push(`<rect x="${rem.x}" y="${rem.y}" width="${rem.w}" height="${rem.h}" fill="#ffffff" fill-opacity=".55" stroke="#3f7d34" stroke-width="${sw*2}" stroke-dasharray="${big/90} ${big/150}"/>`);
    if (rem.w > big*0.1 && rem.h > fs*2.2){
      p.push(`<text x="${rem.x + rem.w/2}" y="${rem.y + rem.h/2 + fs*0.32}" text-anchor="middle" font-size="${fs*0.78}" fill="#3f7d34">${mmv(rem.w)} × ${mmv(rem.h)}</text>`);
    }
  }
```

```
  // piezas
  for (const pc of board.placed){
    p.push(`<rect x="${pc.x}" y="${pc.y}" width="${pc.w}" height="${pc.h}" fill="${pc.color}"
stroke="#143e45" stroke-width="${sw*1.6}" stroke-opacity=".8"/>`);

    const vertical = pc.h > pc.w * 1.25 && pc.w < BL * 0.14;
    const along = vertical ? pc.h : pc.w;      // largo disponible para el texto
    const across = vertical ? pc.w : pc.h;     // alto disponible
    const label = pc.name + (pc.rotated ? " ↻" : "");
    const dims  = mmv(pc.w) + " × " + mmv(pc.h);

    let size = Math.min(fs, (along * 0.9) / (Math.max(label.length, dims.length) * 0.56), across /
2.7);
    if (size < fs * 0.30) continue;            // pieza demasiado chica: sin texto

    const cx = pc.x + pc.w / 2, cy = pc.y + pc.h / 2;
    const tr = vertical ? ` transform="rotate(-90 ${cx} ${cy})"` : "";
    const twoLines = size >= fs * 0.42 && across > size * 2.6;

    p.push(`<g${tr} fill="#143e45">`);
    if (twoLines){
      p.push(`<text x="${cx}" y="${cy - size*0.15}" text-anchor="middle" font-size="${size}"
font-weight="600">${esc(label)}</text>`);
      p.push(`<text x="${cx}" y="${cy + size*0.95}" text-anchor="middle" font-size="${size*0.86}"
fill="#4a5854">${dims}</text>`);
    }else{
      p.push(`<text x="${cx}" y="${cy + size*0.35}" text-anchor="middle" font-size="${size}"
font-weight="600">${esc(label)} · ${dims}</text>`);
    }
    p.push(`</g>`);
  }

  // acotaciones exteriores
  const dy = -m * 0.42, dx = -m * 0.42, t = m * 0.16;
  p.push(`<g stroke="#4a5854" stroke-width="${sw*1.4}" fill="none">
    <line x1="0" y1="${dy}" x2="${BL}" y2="${dy}"/>
    <line x1="0" y1="${dy-t}" x2="0" y2="${dy+t}"/>
    <line x1="${BL}" y1="${dy-t}" x2="${BL}" y2="${dy+t}"/>
    <line x1="${dx}" y1="0" x2="${dx}" y2="${BW}"/>
    <line x1="${dx-t}" y1="0" x2="${dx+t}" y2="0"/>
    <line x1="${dx-t}" y1="${BW}" x2="${dx+t}" y2="${BW}"/>
  </g>`);
```

```
  p.push(`<text x="${BL/2}" y="${dy - t*1.1}" text-anchor="middle" font-size="${fs*0.9}"
fill="#4a5854">${mmv(BL)} mm</text>`);
  p.push(`<text x="${dx - t*1.1}" y="${BW/2}" text-anchor="middle" font-size="${fs*0.9}"
fill="#4a5854" transform="rotate(-90 ${dx - t*1.1} ${BW/2})">${mmv(BW)} mm</text>`);

  p.push(`</svg>`);
  return p.join("");
}

/*
================================================================================
   6. RENDER

================================================================================ */
/* m² y costo imputado de una pieza: se usa al dibujar la fila y al refrescar sus celdas en vivo */
function pieceNumbers(p, c){
  const area = (p.l * p.w * p.qty) / 1e6;
  const share = c.usedArea > 0 ? area / c.usedArea : 0;
  return {area, cost: share * c.total};
}

/* Refresca solo los números de cada fila SIN redibujarla: así el campo que se
   está editando conserva el foco y los botones de la fila no desaparecen
   entre el "mouse abajo" y el "mouse arriba" (si no, el primer clic se pierde). */
function updatePieceCells(c){
  state.pieces.forEach(p => {
    const tr = document.querySelector('#tbody tr[data-id="' + p.id + '"]');
    if (!tr) return;
    const n = pieceNumbers(p, c);
    const a = tr.querySelector('td[data-l="Superficie"]');
    const k = tr.querySelector('td[data-l="Costo imputado"]');
    if (a) a.textContent = f3.format(n.area);
    if (k) k.textContent = money(n.cost);
  });
  document.getElementById("pieceCount").textContent = state.pieces.length
    ? state.pieces.reduce((t, x) => t + (x.qty || 0), 0) + " piezas en " + state.pieces.length + "
medidas" : "";
}

function renderPieces(c){
  const tb = document.getElementById("tbody");
```

```
  const empty = document.getElementById("tblEmpty");
  document.getElementById("pieceCount").textContent =
    state.pieces.length ? state.pieces.reduce((s,p) => s + (p.qty||0), 0) + " piezas en " +
state.pieces.length + " medidas" : "";

  if (!state.pieces.length){
    tb.innerHTML = "";
    empty.style.display = "block";
    return;
  }
  empty.style.display = "none";

  const rows = state.pieces.map((p, i) => {
    const {area, cost} = pieceNumbers(p, c);
    return `<tr data-id="${p.id}" style="--c:${COLORS[i % COLORS.length]}">
      <td data-l="Color"><span class="swatch" style="background:${COLORS[i %
COLORS.length]}"></span></td>
      <td class="wide" data-l="Pieza"><input type="text" value="${esc(p.name)}" data-f="name"
aria-label="Nombre"></td>
      <td class="r" data-l="Largo"><input type="number" value="${p.l}" min="1" step="1" data-f="l"
inputmode="numeric" aria-label="Largo"></td>
      <td class="r" data-l="Ancho"><input type="number" value="${p.w}" min="1" step="1"
data-f="w" inputmode="numeric" aria-label="Ancho"></td>
      <td class="r" data-l="Cantidad"><input type="number" value="${p.qty}" min="1" step="1"
data-f="qty" inputmode="numeric" aria-label="Cantidad"></td>
      <td data-l="Veta"><button class="grainbtn" type="button" data-act="grain"
aria-pressed="${p.grain}" title="Bloquear la rotación de esta pieza para respetar la
veta">${p.grain ? "fija" : "libre"}</button></td>
      <td class="r num" data-l="Superficie">${f3.format(area)}</td>
      <td class="r num" data-l="Costo imputado">${money(cost)}</td>
      <td class="wide" data-l=""><span class="rowbtns">
        <button class="x dup" type="button" data-act="dup" title="Duplicar esta pieza"
aria-label="Duplicar ${esc(p.name)}">${DUP_SVG}</button>
        <button class="x" type="button" data-act="del" title="Eliminar esta pieza"
aria-label="Eliminar ${esc(p.name)}">✕</button>
      </span></td>
    </tr>`;
  });
  tb.innerHTML = rows.join("");
}

function renderPlan(c){
  const stats = document.getElementById("planStats");
```

```
  const boardsEl = document.getElementById("boards");
  const notices = document.getElementById("notices");

  // avisos
  const msgs = [];
  if (c.tooBig.length){
    msgs.push(`<b>${c.tooBig.length} medida${c.tooBig.length>1?"s no entran":" no entra"}
${c.metal ? "en la plancha" : "en el tablero"}:</b> ` +
      c.tooBig.map(p => esc(p.name) + " " + mmv(p.l) + "×" + mmv(p.w)).join(", ") +
      `. Ajustá las medidas, ${c.metal ? "la plancha" : "el tablero"}, o permití la rotación.`);
  }
  if (c.res.unplaced && c.res.unplaced.length){
    msgs.push(`<b>${c.res.unplaced.length} piezas quedaron sin ubicar.</b> Revisá el grosor
del disco y las medidas${c.maxCut ? ", o el largo máximo de corte (¿la cuchilla alcanza para
cruzar la plancha?)" : ""}.`);
  }
  notices.innerHTML = msgs.map(m => `<div class="notice">${m}</div>`).join("");

  if (!c.nBoards){
    stats.innerHTML = "";
    boardsEl.innerHTML = `<div class="placeholder">
      <strong>El plano se dibuja solo</strong>
      Cargá ${c.metal ? "la plancha" : "el tablero"} y al menos una pieza y vas a ver acá la
distribución a escala, ${c.metal ? "plancha por plancha" : "placa por placa"}.
    </div>`;
    return;
  }

  stats.innerHTML =
    `<span>${c.metal ? "Planchas" : "Tableros"} <b>${c.nBoards}</b></span>` +
    `<span>Aprovechado <b>${pct(c.useRate)}</b></span>` +
    `<span>Útil <b>${m2(c.usedArea)}</b></span>` +
    `<span>Desperdicio <b>${m2(c.wasteArea)}</b></span>`;

  const out = c.res.boards.map((b, i) => {
    b.uidx = i;
    const used = b.placed.reduce((s,p) => s + p.w*p.h, 0) / 1e6;
    const rate = c.boardArea > 0 ? used / c.boardArea * 100 : 0;
    const rem = usefulRemnant(b, c.BL, c.BW);
    const remTxt = rem ? ` · <em>recorte útil ${mmv(rem.w)} × ${mmv(rem.h)}</em>` : "";
    return `<div class="board">
      <header>
        <h3>${c.metal ? "Plancha" : "Placa"} ${i+1}</h3>
```

```
      <span class="meta">${b.placed.length} piezas · ${pct(rate)} aprovechado · ${m2(c.boardArea - used)} de sobra${remTxt}</span>
    </header>
    ${boardSVG(b, c.BL, c.BW, c.kerf)}
  </div>`;
 });
 boardsEl.innerHTML = out.join("");
}

function renderCosts(c){
 const serviceLabel = state.cut.serviceMode === "board"
    ? `${f0.format(c.nBoards)} ${c.metal ? "plancha" : "tablero"}${c.nBoards === 1 ? "" : "s"} × ${money(state.cut.service||0)}`
  : state.cut.serviceMode === "cut"
    ? `${f0.format(c.cuts)} corte${c.cuts === 1 ? "" : "s"} × ${money(state.cut.service||0)}`
  : state.cut.serviceMode === "meter"
    ? `${f1.format(c.cutM)} m de corte × ${money(state.cut.service||0)}`
    : `monto fijo del pedido`;
 const svcTxt = c.minApplied ? `cargo mínimo (el corte calculado daba ${money(c.serviceRaw)})` : serviceLabel;

 const used = c.costMode === "used";
 const rows = used ? [
  [c.metal ? "Chapa que usa el trabajo" : "Madera que usa el mueble", `${m2(c.usedArea)} × ${money(c.perM2)}/m²`, money(c.netCost), ""],
  [c.metal ? "Sobrante de chapa" : "Sobrante reutilizable", `${m2(c.wasteArea)} · ${pct(100 - c.useRate)} ${c.metal ? "de la plancha" : "del tablero"} · vale ${money(c.offcutCost)}, queda en tu taller`, "no se cobra", "sub muted"],
  [`Merma adicional ${pct(state.cut.waste||0)}`, `sobre la ${c.metal ? "chapa" : "madera"} usada: roturas, fallas y reprocesos`, money(c.extraWaste), ""],
  ["Servicio de corte", svcTxt, money(c.service), ""]
 ] : [
  [c.metal ? "Planchas consumidas" : "Tableros consumidos", `${f0.format(c.nBoards)} × ${money(c.sheetPrice)}`, money(c.boardsCost), ""],
  [c.metal ? "Chapa efectivamente usada" : "Madera efectivamente usada", `${m2(c.usedArea)} de superficie útil`, money(c.netCost), "sub"],
  ["Recorte y desperdicio del plano", `${m2(c.wasteArea)} · ${pct(100 - c.useRate)} del material`, money(c.offcutCost), "sub"],
  [`Merma adicional ${pct(state.cut.waste||0)}`, `sobre ${c.metal ? "la plancha completa" : "el tablero completo"}: roturas, fallas y reprocesos`, money(c.extraWaste), ""],
  ["Servicio de corte", svcTxt, money(c.service), ""]
 ];
```

```
  if (c.extras > 0) rows.push(["Plegados y perforaciones", `${f0.format(state.cut.bends||0)}
plegado${(state.cut.bends||0) === 1 ? "" : "s"} · ${f0.format(state.cut.holes||0)}
perforacion${(state.cut.holes||0) === 1 ? "" : "es"}`, money(c.extras), ""]);
  if (c.metal && c.kgM2 > 0) rows.push(
    ["Peso", `${f1.format(c.usedKg)} kg de piezas · ${f1.format(c.wasteKg)} kg de sobrante ·
${f1.format(c.totalKg)} kg consumidos`, "", "sub muted"],
    ["Largo total de corte", `${f1.format(c.cutM)} m en ${f0.format(c.cuts)} cortes`, "", "sub
muted"]);

  const html = rows.map(r =>
    `<div class="costline ${r[3]}"><div class="k">${r[0]}<small>${r[1]}</small></div><div
class="v">${r[2]}</div></div>`
  ).join("") +
  `<div class="costline"><div class="k">Costo por m² útil<small>lo que realmente cuesta cada
m² que queda en el mueble</small></div><div
class="v">${money(c.costPerUsefulM2)}</div></div>` +
  `<div class="costline total"><div class="k">Costo total del despiece<small>${used ? (c.metal ?
"chapa usada" : "madera usada") : (c.metal ? "planchas" : "tableros")} + merma + servicio de
corte</small></div><div class="v">${money(c.total)}</div></div>` +
  (c.nBoards ? `<div class="sendbar noprint"><button class="btn primary" id="btnToQuote"
type="button">Pasar ${money(c.woodForQuote)} al cotizador de materiales</button><p
class="hint">Se agrega como un ítem "Tablero" (${c.metal ? "chapa" : "madera"}${c.service ? "
+ corte" : ""}, sin la merma: el cotizador ya suma su propia contingencia). Ahí le sumás el metal
y el margen.</p></div>` : "");

  document.getElementById("costs").innerHTML = html;
}

function renderPrintExtras(c){
  const d = new Date();
  document.getElementById("printhead").innerHTML =
    `<h1>Plano de corte y despiece</h1>
     <div class="ph">
       <span>${c.metal ? "Plancha" : "Tablero"} <b>${mmv(c.BL0)} × ${mmv(c.BW0)}
mm${state.board.thick ? " × " + f1.format(state.board.thick) + " mm" : ""}</b></span>
       <span>Disco <b>${f1.format(c.kerf)} mm</b></span>
       <span>Rotación <b>${state.cut.rotate ? "permitida" : "bloqueada"}</b></span>
       <span>Placas <b>${f0.format(c.nBoards)}</b></span>
       <span>Aprovechamiento <b>${pct(c.useRate)}</b></span>
       <span>Total <b>${money(c.total)}</b></span>
       <span>${d.toLocaleDateString("es-AR")}</span>
     </div>`;
```

```
  const list = state.pieces.filter(p => p.l>0 && p.w>0 && p.qty>0);
  document.getElementById("printlist").innerHTML = !list.length ? "" :
    `<h2 style="font-size:12pt;margin:6mm 0 2mm">Lista de corte</h2>
     <table><thead><tr><th>Pieza</th><th class="r">Largo</th><th class="r">Ancho</th><th
class="r">Cant.</th><th>Veta</th><th class="r">m²</th></tr></thead>
     <tbody>${list.map(p => `<tr>
       <td>${esc(p.name)}</td><td class="r num">${mmv(p.l)}</td><td class="r
num">${mmv(p.w)}</td>
       <td class="r num">${f0.format(p.qty)}</td><td>${p.grain ? "fija" : "libre"}</td>
       <td class="r num">${f3.format(p.l*p.w*p.qty/1e6)}</td></tr>`).join("")}
     </tbody></table>`;
}

function updatePrintout(){
  if (activeTab === "mat"){ if (lastMat) renderMatPrintExtras(lastMat); }
  else if (activeTab === "d3"){ d3PrintExtras(); }
  else if (activeTab === "calc" || activeTab === "quote"){
    document.getElementById("printhead").innerHTML = "";
    document.getElementById("printlist").innerHTML = "";
  }
  else if (last) renderPrintExtras(last);
}

function render(){
  const c = compute();
  last = c;

  document.getElementById("oArea").textContent  = c.boardArea > 0 ? m2(c.boardArea) : "—";
  document.getElementById("oPerM2").textContent = c.perM2 > 0 ? money(c.perM2) : "—";
  document.getElementById("oKgBox").hidden = !c.metal;
  document.getElementById("oKg").textContent = c.sheetKg > 0 ? f1.format(c.sheetKg) + " kg" :
"—";
  document.getElementById("metalCard").hidden = !c.metal;
  document.getElementById("ttlBoard").textContent = c.metal ? "Plancha base" : "Tablero base";
  document.getElementById("lblPrice").textContent = c.metal
    ? (state.cut.priceBy === "kg" ? "Precio por kg ($/kg)" : "Precio de la plancha ($)") : "Precio del
tablero";
  document.getElementById("methodHint").textContent = METHODS[state.cut.method] ?
METHODS[state.cut.method].hint : "";

  document.querySelectorAll(".chip").forEach(ch => {
    ch.setAttribute("aria-pressed", String(Math.abs((state.cut.waste||0) - (+ch.dataset.waste)) <
0.001));
```

```
  });
  document.getElementById("costModeHint").textContent = c.costMode === "used"
    ? `Se cobra solo la ${c.metal ? "chapa" : "madera"} que ocupan las piezas. El sobrante queda en tu taller para otro trabajo.`
    : `Se cobra ${c.metal ? "cada plancha entera" : "cada tablero entero"}. El sobrante se considera perdido.`;
  applyMaterialLabels(c);

  renderPieces(c);
  renderPlan(c);
  renderCosts(c);
  updatePrintout();
}

/* -------- render del cotizador de materiales -------- */
function matItemBody(it){
  if (it.kind === "tablero") return `
    <div class="grid g3">
      <label><span class="lab">Precio de la pieza ($)</span><input type="number" data-f="price" min="0" step="0.01" inputmode="decimal" value="${it.price || ""}"></label>
      <label><span class="lab">Largo (mm)</span><input type="number" data-f="l" min="0" step="1" inputmode="numeric" value="${it.l || ""}"></label>
      <label><span class="lab">Ancho (mm)</span><input type="number" data-f="w" min="0" step="1" inputmode="numeric" value="${it.w || ""}"></label>
    </div>`;
  if (it.kind === "barra") return `
    <div class="grid g3">
      <label><span class="lab">Precio por barra ($)</span><input type="number" data-f="price" min="0" step="0.01" inputmode="decimal" value="${it.price || ""}"></label>
      <label><span class="lab">Largo de barra (mm)</span><input type="number" data-f="barLen" min="1" step="1" inputmode="numeric" value="${it.barLen || 6000}"></label>
      <label><span class="lab">Largo necesario (mm)</span><input type="number" data-f="need" min="0" step="1" inputmode="numeric" value="${it.need || ""}"></label>
    </div>
    <p class="hint">Suma el largo de todas las piezas de este perfil que lleva <b>un</b> mueble (patas + refuerzos). Se redondea hacia arriba en barras enteras.</p>`;
  if (it.kind === "consumible") return `
    <div class="grid g2">
      <label class="full" style="grid-column:1/-1"><span class="lab">Material</span>
        <select data-f="preset">
          ${Object.keys(PRESETS).map(k => `<option value="${k}"${it.preset===k?" selected":""}>${PRESETS[k].label}</option>`).join("")}
        </select>
```

```
      </label>
      <label><span class="lab">Precio del envase ($)</span><input type="number" data-f="price"
min="0" step="0.01" inputmode="decimal" value="${it.price || ""}"></label>
      <label><span class="lab">Rinde</span><input type="number" data-f="yieldQty" min="0"
step="0.1" inputmode="decimal" value="${it.yieldQty || ""}"></label>
      <label><span class="lab">Unidad de rendimiento</span>
        <select data-f="yieldUnit">
          <option value="m2"${it.yieldUnit==="m2"?" selected":""}>m² / envase</option>
          <option value="un"${it.yieldUnit==="un"?" selected":""}>uniones / envase</option>
        </select>
      </label>
      <label><span class="lab">Consumo por mueble
(${it.yieldUnit==="un"?"uniones":"m²"})</span><input type="number" data-f="consumo" min="0"
step="0.1" inputmode="decimal" value="${it.consumo || ""}"></label>
    </div>
    <p class="hint">Rendimiento estimado de plaza — ajustalo apenas tengas tu propio
consumo medido.</p>`;
  return `
    <div class="grid g2">
      <label><span class="lab">Precio unitario ($)</span><input type="number" data-f="price"
min="0" step="0.01" inputmode="decimal" value="${it.price || ""}"></label>
      <label><span class="lab">Cantidad por mueble</span><input type="number"
data-f="unitQty" min="0" step="1" inputmode="numeric" value="${it.unitQty || ""}"></label>
    </div>`;
}

function priceStatusHTML(it){
  const d = daysSince(it.priceDate);
  if (d === null) return `<span class="pstatus" data-role="pstatus">precio sin fechar</span>`;
  const txt = d === 0 ? "precio de hoy" : d === 1 ? "precio de ayer" : `precio de hace
${f0.format(d)} días`;
  return `<span class="pstatus${d >= PRICE_STALE_DAYS ? " stale" : ""}"
data-role="pstatus">${txt}</span>`;
}

function matItemCardHTML(it){
  const u = safeUrl(it.url);
  return `<div class="mitem" data-id="${it.id}">
    <div class="mitem-head">
      <input type="text" class="mname" data-f="name" placeholder="Nombre (ej. Tapa de
escritorio)" value="${esc(it.name)}" aria-label="Nombre del material">
      <select class="mkind" data-f="kind" aria-label="Tipo de material">
        <option value="tablero"${it.kind==="tablero"?" selected":""}>Tablero / placa</option>
```

```
      <option value="barra"${it.kind==="barra"?" selected":""}>Barra / perfil metálico</option>
      <option value="consumible"${it.kind==="consumible"?" selected":""}>Pintura, barniz, tinte o suelda</option>
      <option value="directo"${it.kind==="directo"?" selected":""}>Herraje / otro directo</option>
     </select>
     <span class="rowbtns">
      <button class="x dup" type="button" data-act="mdup" title="Duplicar" aria-label="Duplicar ${esc(it.name || "material")}">${DUP_SVG}</button>
      <button class="x" type="button" data-act="mdel" title="Eliminar" aria-label="Eliminar ${esc(it.name || "material")}">✕</button>
     </span>
    </div>
    <div class="mitem-body">${matItemBody(it)}
     <div class="mitem-link">
      <input type="url" data-f="url" placeholder="Link del proveedor o publicación (https://…)" value="${esc(it.url || "")}" aria-label="Enlace de referencia del precio">
      ${u ? `<a class="btn quiet linkbtn" href="${esc(u)}" target="_blank" rel="noopener noreferrer">Ver en ${esc(hostOf(u))}</a>` : ""}
     </div>
    </div>
    <div class="mitem-foot">
     <span class="mnote" data-role="note">${it.note || ""}</span>
     ${priceStatusHTML(it)}
     <span class="mcost" data-role="cost">${money(it.total)}</span>
    </div>
  </div>`;
}

function renderMatSummary(m){
  const rows = m.items.length ? m.items.map(it =>
    `<div class="costline sub"><div class="k">${esc(it.name || "Material sin nombre")}<small>${it.note || kindLabel(it.kind)}</small></div><div class="v">${money(it.total)}</div></div>`
  ).join("") : `<div class="costline"><div class="k">Todavía no cargaste materiales<small>agregalos en la lista de la izquierda</small></div><div class="v">—</div></div>`;

  const totalSale = m.price * m.qty;
  const stale = m.items.filter(it => { const d = daysSince(it.priceDate); return d !== null && d >= PRICE_STALE_DAYS; }).length;
  const undated = m.items.filter(it => !it.priceDate).length;
  let warn = "";
  if (stale || undated){
```

```
    const parts = [];
    if (stale) parts.push(`${f0.format(stale)} con más de ${PRICE_STALE_DAYS} días`);
    if (undated) parts.push(`${f0.format(undated)} sin fechar`);
    warn = `<div class="costline warnline"><div class="k">Revisá los precios<small>${parts.join(" · ")} — abrí el link de cada material y actualizá el valor</small></div><div class="v">⚠</div></div>`;
  }

  const marginDesc = mat.marginMode === "margin"
    ? `${pct(mat.marginPct||0)} de margen sobre el precio final`
    : `${pct(mat.marginPct||0)} de markup sobre el costo`;

  document.getElementById("mCosts").innerHTML = rows + warn +
    `<div class="costline"><div class="k">Subtotal materiales<small>ya multiplicado por ${f0.format(m.qty)} mueble${m.qty===1?"":"s"}</small></div><div class="v">${money(m.subtotal)}</div></div>` +
    `<div class="costline"><div class="k">Contingencia ${pct(mat.waste||0)}<small>reserva por sobrantes, fallas y reprocesos</small></div><div class="v">${money(m.wasteCost)}</div></div>` +
    `<div class="costline"><div class="k">Costo por mueble<small>total del pedido dividido ${f0.format(m.qty)}</small></div><div class="v">${money(m.perUnit)}</div></div>` +
    `<div class="costline total"><div class="k">Costo total de materiales<small>para ${f0.format(m.qty)} mueble${m.qty===1?"":"s"}</small></div><div class="v">${money(m.total)}</div></div>` +
    `<div class="costline total price"><div class="k">Precio de venta sugerido<small>${marginDesc} · ${money(m.price)} c/u</small></div><div class="v">${money(totalSale)}</div></div>`;

  document.getElementById("mMarginHint").innerHTML =
    `Con el costo actual (<b>${money(m.perUnit)}</b> por mueble), este margen sugiere venderlo a <b>${money(m.price)}</b> — una ganancia de ${money(m.profit)} por unidad. Esto no incluye tu mano de obra: sumala como un ítem "Herraje / directo" si querés que forme parte del costo.`;
}

function renderMatPrintExtras(m){
  const d = new Date();
  document.getElementById("printhead").innerHTML =
    `<h1>Cotización de materiales</h1>
     <div class="ph">
       <span>Muebles a cotizar <b>${f0.format(mat.qty||1)}</b></span>
       <span>Contingencia <b>${pct(mat.waste||0)}</b></span>
       <span>Subtotal <b>${money(m.subtotal)}</b></span>
```

```
      <span>Costo por mueble <b>${money(m.perUnit)}</b></span>
      <span>Total <b>${money(m.total)}</b></span>
      <span>Precio sugerido <b>${money(m.price)}</b> c/u</span>
      <span>${d.toLocaleDateString("es-AR")}</span>
    </div>`;

  // El detalle por ítem ya se imprime en la tarjeta de costos de la derecha;
  // acá no se repite para no duplicar la misma información dos veces.
  document.getElementById("printlist").innerHTML = "";
}

function updateMatTotalsOnly(){
  const m = computeMat();
  lastMat = m;
  m.items.forEach(it => {
    const card = document.querySelector('.mitem[data-id="' + it.id + '"]');
    if (!card) return;
    const noteEl = card.querySelector('[data-role="note"]');
    const costEl = card.querySelector('[data-role="cost"]');
    const stEl   = card.querySelector('[data-role="pstatus"]');
    if (noteEl) noteEl.textContent = it.note || "";
    if (costEl) costEl.textContent = money(it.total);
    if (stEl){
      const tmp = document.createElement("div");
      tmp.innerHTML = priceStatusHTML(it);
      stEl.replaceWith(tmp.firstChild);
    }
  });
  renderMatSummary(m);
  updatePrintout();
}

function renderMat(){
  const m = computeMat();
  lastMat = m;
  document.getElementById("mItemCount").textContent =
    mat.items.length ? mat.items.length + " ítem" + (mat.items.length === 1 ? "" : "s") : "";
  document.getElementById("mEmpty").style.display = mat.items.length ? "none" : "block";
  document.getElementById("mItems").innerHTML = m.items.map(matItemCardHTML).join("");
  renderMatSummary(m);
  updateSaveBtnLabel();
  updatePrintout();
}
```

```
/* -------- lista de muebles guardados -------- */
function updateSaveBtnLabel(){
  const btn = document.getElementById("btnMSave");
  if (!btn) return;
  const editing = matEditingId && book.entries.some(e => e.id === matEditingId);
  btn.textContent = editing ? "Actualizar mueble" : "Guardar mueble";
}

function bookRowHTML(e, idx, total){
  const c = computeEntryTotals(e);
  const editing = e.id === matEditingId;
  return `<div class="mbook-row${editing ? " editing" : ""}" data-id="${e.id}">
    <span class="mbook-num">${idx + 1}</span>
    <div class="mbook-info">
      <div class="mbook-name">${esc(e.name)}${editing ? " · editando" : ""}</div>
      <div class="mbook-meta">${f0.format(c.qty)} unidad${c.qty === 1 ? "" : "es"} · costo
${money(c.perUnit)} c/u</div>
    </div>
    <div class="mbook-price">
      <span class="k">Precio sugerido</span>
      <span class="v">${money(c.price)}</span>
    </div>
    <span class="rowbtns">
      <button class="x" type="button" data-act="up" ${idx === 0 ? "disabled" : ""} title="Subir en la
lista" aria-label="Subir ${esc(e.name)}">▲</button>
      <button class="x" type="button" data-act="down" ${idx === total - 1 ? "disabled" : ""}
title="Bajar en la lista" aria-label="Bajar ${esc(e.name)}">▼</button>
      <button class="x" type="button" data-act="load" title="Cargar para editar" aria-label="Editar
${esc(e.name)}">✎</button>
      <button class="x dup" type="button" data-act="bdup" title="Duplicar" aria-label="Duplicar
${esc(e.name)}">${DUP_SVG}</button>
      <button class="x" type="button" data-act="bdel" title="Eliminar de la lista"
aria-label="Eliminar ${esc(e.name)}">✕</button>
    </span>
  </div>`;
}

function renderBook(){
  document.getElementById("bookCount").textContent =
    book.entries.length ? book.entries.length + " mueble" + (book.entries.length === 1 ? "" : "s") :
"";
  document.getElementById("bookEmpty").style.display = book.entries.length ? "none" : "block";
```

```
  document.getElementById("bookList").innerHTML =
    book.entries.map((e, i) => bookRowHTML(e, i, book.entries.length)).join("");
}

/*
================================================================================
   7. ENTRADA / SALIDA DE FORMULARIOS

================================================================================ */
const F = {
  price:"board.price", thick:"board.thick", bl:"board.length", bw:"board.width",
  kerf:"cut.kerf", waste:"cut.waste", service:"cut.service",
  maxCut:"cut.maxCut", trim:"cut.trim", minCharge:"cut.minCharge",
  bends:"cut.bends", bendPrice:"cut.bendPrice", holes:"cut.holes", holePrice:"cut.holePrice"
};

/* -------- modo chapa metálica -------- */
/* Cambia los textos de madera por los de chapa y deja cada selector mostrando el valor vigente */
function applyMaterialLabels(c){
  const m = c.metal, $ = id => document.getElementById(id);
  const t = (id, w, k) => { $(id).textContent = m ? k : w; };
  t("lblBL", "Largo del tablero (mm)", "Largo de la plancha (mm)");
  t("lblBW", "Ancho del tablero (mm)", "Ancho de la plancha (mm)");
  t("lblThick", "Grosor del material (mm)", "Espesor (mm)");
  t("lblArea", "Superficie del tablero", "Superficie de la plancha");
  t("lblCostMode", "Cómo se cobra el material", "Cómo se cobra la chapa");
  t("optUsed", "Solo la madera usada", "Solo la chapa usada");
  t("optBoard", "Tablero completo", "Plancha completa");
  t("lblKerfLeg", "Separación por disco de sierra", "Separación por el corte (disco)");
  $("hintRot").innerHTML = m
    ? "Aprovecha mejor la plancha. Si la chapa es cepillada o tiene dibujo, bloqueá la rotación pieza por pieza con el botón <b>veta</b> de la lista."
    : "Aprovecha mejor la placa, pero cruza la veta. Podés bloquear la rotación pieza por pieza con el botón <b>veta</b> de la lista.";
  $("metalPick").hidden = !m;
  $("maxCutBox").hidden = state.cut.method !== "cizalla";
  const th = state.board.thick || 0, g = $("gauge"), sp = $("sheetPreset");
  const og = [...g.options].find(o => o.value && Math.abs(+o.value - th) < 0.005);
  g.value = og ? og.value : "";
  const os = [...sp.options].find(o => {
```

```
    if (!o.value) return false;
    const d = o.value.split("x").map(Number);
    return (d[0] === c.BL0 && d[1] === c.BW0) || (d[0] === c.BW0 && d[1] === c.BL0);
  });
  sp.value = os ? os.value : "";
}
const METAL_DENS = {acero:7.85, galva:7.85, inox:7.9, alu:2.7};     // g/cm³: kg de una chapa = m² × mm × densidad
const METHODS = {
  cizalla:  {kerf:0,   hint:"Corte recto de lado a lado, sin perder material. Cargá abajo el largo de la cuchilla en «Largo máximo de corte»."},
  amoladora:{kerf:2,   hint:"Disco fino ≈ 1–2 mm, disco común ≈ 2,5–3 mm. Ajustá el «Grosor del disco» según el tuyo y usá la calculadora de abajo para sacar tu costo por metro."},
  plasma:   {kerf:2,   hint:"El ancho de corte depende de la boquilla y del espesor: confirmalo con tu equipo o proveedor."},
  laser:    {kerf:0.3, hint:"Corte muy fino. Muchos talleres cobran por metro de corte más un cargo mínimo."}
};
function bindMetal(){
  const $ = id => document.getElementById(id);
  const set = (k, v) => { state.cut[k] = v; save(); render(); };
  $("material").addEventListener("change", e => set("material", e.target.value === "metal" ? "metal" : "wood"));
  $("metalType").addEventListener("change", e => set("metalType", e.target.value));
  $("priceBy").addEventListener("change", e => set("priceBy", e.target.value === "kg" ? "kg" : "sheet"));
  $("gauge").addEventListener("change", e => {
    const v = parseFloat(e.target.value);
    if (v > 0){ state.board.thick = v; $("thick").value = v; save(); }
    render();
  });
  $("sheetPreset").addEventListener("change", e => {
    const d = e.target.value.split("x").map(Number);
    if (d.length === 2 && d[0] > 0){ state.board.length = d[0]; state.board.width = d[1]; $("bl").value = d[0]; $("bw").value = d[1]; save(); }
    render();
  });
  $("method").addEventListener("change", e => {
    const m = METHODS[e.target.value];
    state.cut.method = m ? e.target.value : "";
    if (m){ state.cut.kerf = m.kerf; $("kerf").value = m.kerf; }
    save(); render();
  });
```

```
  // costo por metro de corte con amoladora: disco + tu tiempo
  const perMeter = () => {
    const g = id => Math.max(0, parseFloat($(id).value) || 0);
    const disc = g("hDisc") > 0 && g("hDiscM") > 0 ? g("hDisc") / g("hDiscM") : 0, time = g("hMin")
/ 60 * g("hRate");
    $("hResult").textContent = disc + time > 0
      ? "Te cuesta " + money(disc + time) + " por metro de corte (disco " + money(disc) + " + tu
tiempo " + money(time) + ")."
      : "Completá los datos para ver el costo por metro.";
    return disc + time;
  };
  ["hDisc","hDiscM","hMin","hRate"].forEach(id => $(id).addEventListener("input", perMeter));
  $("hUse").addEventListener("click", () => {
    const v = perMeter();
    if (!(v > 0)){ flag("Completá los datos del disco y de tu tiempo"); return; }
    state.cut.service = Math.round(v * 100) / 100; state.cut.serviceMode = "meter";
    $("service").value = state.cut.service; $("serviceMode").value = "meter";
    save(); render(); flag("Servicio cargado por metro de corte");
  });
  perMeter();
}

function fillForm(){
  for (const id in F){
    const [a,b] = F[id].split(".");
    document.getElementById(id).value = state[a][b] === 0 &&
["price","service","maxCut","trim","minCharge","bends","bendPrice","holes","holePrice"].includes(
id) ? "" : state[a][b];
  }
  document.getElementById("rotate").checked = !!state.cut.rotate;
  document.getElementById("serviceMode").value = state.cut.serviceMode;
  document.getElementById("costMode").value = state.cut.costMode === "board" ? "board" :
"used";
  document.getElementById("material").value = state.cut.material === "metal" ? "metal" :
"wood";
  document.getElementById("metalType").value = METAL_DENS[state.cut.metalType] ?
state.cut.metalType : "acero";
  document.getElementById("priceBy").value = state.cut.priceBy === "kg" ? "kg" : "sheet";
  document.getElementById("method").value = METHODS[state.cut.method] ? state.cut.method
: "";
}

function bindForm(){
```

```
  bindMetal();
  for (const id in F){
    document.getElementById(id).addEventListener("input", e => {
      const [a,b] = F[id].split(".");
      let v = parseFloat(e.target.value);
      if (!isFinite(v) || v < 0) v = 0;
      if ((id === "bl" || id === "bw") && v < 1) v = 0;
      state[a][b] = v;
      save(); render();
    });
  }
  document.getElementById("rotate").addEventListener("change", e => {
    state.cut.rotate = e.target.checked; save(); render();
  });
  document.getElementById("serviceMode").addEventListener("change", e => {
    state.cut.serviceMode = e.target.value; save(); render();
  });
  document.getElementById("costMode").addEventListener("change", e => {
    state.cut.costMode = e.target.value === "board" ? "board" : "used"; save(); render();
  });
  // el panel de costos se redibuja entero, por eso el clic se delega en el contenedor
  document.getElementById("costs").addEventListener("click", e => {
    if (e.target.closest("#btnToQuote")) sendToQuote();
  });
  document.querySelectorAll(".chip").forEach(ch => {
    ch.addEventListener("click", () => {
      state.cut.waste = +ch.dataset.waste;
      document.getElementById("waste").value = state.cut.waste;
      save(); render();
    });
  });
}

function addPiece(){
  const n = document.getElementById("pName");
  const L = document.getElementById("pL");
  const W = document.getElementById("pW");
  const Q = document.getElementById("pQ");
  const l = Math.round(parseFloat(L.value)), w = Math.round(parseFloat(W.value));
  const q = Math.max(1, Math.round(parseFloat(Q.value) || 1));

  if (!(l > 0)){ L.focus(); L.select(); flag("Falta el largo"); return; }
  if (!(w > 0)){ W.focus(); W.select(); flag("Falta el ancho"); return; }
```

```
  state.pieces.push({
    id: uid(),
    name: (n.value || "").trim() || ("Pieza " + (state.pieces.length + 1)),
    l, w, qty: q, grain: false
  });
  n.value = ""; L.value = ""; W.value = ""; Q.value = 1;
  n.focus();
  save(); render();
}

function bindList(){
  document.getElementById("btnAdd").addEventListener("click", addPiece);
  ["pName","pL","pW","pQ"].forEach(id => {
    document.getElementById(id).addEventListener("keydown", e => {
      if (e.key === "Enter"){ e.preventDefault(); addPiece(); }
    });
  });

  const tb = document.getElementById("tbody");

  tb.addEventListener("input", e => {
    const tr = e.target.closest("tr"); if (!tr) return;
    const p = state.pieces.find(x => x.id === tr.dataset.id); if (!p) return;
    const f = e.target.dataset.f; if (!f) return;
    if (f === "name") p.name = e.target.value;
    else{
      let v = parseFloat(e.target.value);
      if (!isFinite(v) || v < 0) v = 0;
      p[f] = (f === "qty") ? Math.max(0, Math.round(v)) : Math.round(v);
    }
    save();
    const c = compute(); last = c;
    renderPlan(c); renderCosts(c); updatePieceCells(c); updatePrintout();
  });

  // al salir del campo se muestra el valor ya normalizado (p. ej. vacío -> 0) sin redibujar la fila
  tb.addEventListener("change", e => {
    const f = e.target.dataset.f;
    if (!f || f === "name") return;
    const tr = e.target.closest("tr"); if (!tr) return;
    const p = state.pieces.find(x => x.id === tr.dataset.id);
    if (p) e.target.value = p[f];
```

```
  });

  tb.addEventListener("click", e => {
    const btn = e.target.closest("[data-act]"); if (!btn) return;
    const tr = e.target.closest("tr");
    const id = tr.dataset.id;
    let focusId = null;

    if (btn.dataset.act === "del"){
      state.pieces = state.pieces.filter(p => p.id !== id);
    }else if (btn.dataset.act === "grain"){
      const p = state.pieces.find(x => x.id === id);
      if (p) p.grain = !p.grain;
    }else if (btn.dataset.act === "dup"){
      const i = state.pieces.findIndex(x => x.id === id);
      if (i < 0) return;
      const o = state.pieces[i];
      const copy = {id:uid(), name:o.name, l:o.l, w:o.w, qty:o.qty, grain:o.grain};
      state.pieces.splice(i + 1, 0, copy);   // la copia queda justo debajo del original
      focusId = copy.id;
    }

    save(); render();

    if (focusId){
      const inp = document.querySelector('tr[data-id="' + focusId + '"] input[data-f="name"]');
      if (inp){ inp.focus(); inp.select(); }
    }
  });

  document.getElementById("btnClearList").addEventListener("click", () => {
    if (!state.pieces.length) return;
    if (confirm("¿Vaciar la lista de piezas? El tablero y los ajustes se mantienen.")){
      state.pieces = []; save(); render();
    }
  });
}

/*
================================================================================
  8. EJEMPLO, RESET, IMPRESIÓN
```

```
================================================================================
= */
const DEMO = [
  {name:"Lateral",         l:720, w:580, qty:2},
  {name:"Techo",          l:800, w:580, qty:1},
  {name:"Piso",           l:800, w:580, qty:1},
  {name:"Estante",        l:764, w:560, qty:3},
  {name:"Puerta",         l:716, w:396, qty:2},
  {name:"Frente de cajón", l:396, w:180, qty:3},
  {name:"Lateral cajón",   l:500, w:150, qty:6},
  {name:"Fondo cajón",     l:366, w:480, qty:3},
  {name:"Zócalo",         l:800, w:100, qty:2}
];

function loadDemo(){
  state = defaults();
  state.board.price = 92000;
  state.pieces = DEMO.map(d => Object.assign({id:uid(), grain:false}, d));
  fillForm(); save(); render();
  flag("Ejemplo cargado");
}

function reset(){
  if (!confirm("¿Borrar todos los datos cargados y empezar de cero?")) return;
  state = defaults();
  try{ localStorage.removeItem(KEY); }catch(e){}
  fillForm(); render();
  flag("Listo para empezar");
}

/* -------- ejemplo y reset del cotizador de materiales -------- */
const MAT_DEMO = [
  {kind:"tablero",    name:"Tapa de escritorio",              price:135000, l:1400, w:600},
  {kind:"barra",      name:"Estructura 30×40 (patas y refuerzos)", price:45000, barLen:6000,
need:8240},
  {kind:"consumible", name:"Tinte + barniz",  price:60000, consumo:2.5, preset:"tinte",
yieldQty:12, yieldUnit:"m2"},
  {kind:"consumible", name:"Suelda con flux", price:18000, consumo:12,  preset:"suelda",
yieldQty:25, yieldUnit:"un"},
  {kind:"directo",    name:"Herrajes / tornillería", price:1500, unitQty:8}
];

function fillMatForm(){
```

```
  document.getElementById("mName").value = mat.name || "";
  document.getElementById("mQty").value = mat.qty;
  document.getElementById("mWaste").value = mat.waste;
  document.getElementById("mMargin").value = mat.marginPct;
  document.getElementById("mMarginMode").value = mat.marginMode;
}

/* Pasa el costo de la madera del plano de corte al cotizador como un ítem
   "Tablero". Si ya se había pasado antes, actualiza ese mismo ítem. */
function sendToQuote(){
  const c = last;
  if (!c || !c.nBoards) return;
  const placed = c.res.boards.reduce((s,b) => s.concat(b.placed), []);
  const n = placed.length;
  const name = c.costMode === "used"
    ? `${c.metal ? "Chapa" : "Madera"} cortada (${f0.format(n)} pieza${n === 1 ? "" : "s"},
${m2(c.usedArea)})`
    : `${c.metal ? "Planchas" : "Tableros"} (${f0.format(c.nBoards)} × ${money(c.sheetPrice)})`;
  const price = Math.round(c.woodForQuote * 100) / 100;
  // con una sola pieza se guardan sus medidas, así el cotizador muestra m² y $/m²
  const dims = (c.costMode === "used" && n === 1)
    ? (placed[0].rotated
        ? {l: Math.round(placed[0].h), w: Math.round(placed[0].w)}
        : {l: Math.round(placed[0].w), w: Math.round(placed[0].h)})
    : {l: 0, w: 0};

  let it = mat.items.find(x => x.fromCut && x.kind === "tablero");
  if (it){
    Object.assign(it, {name, price, priceDate: todayISO()}, dims);
  }else{
    it = Object.assign(matItemDefaults("tablero"),
      {id: uid(), name, price, priceDate: todayISO(), fromCut: true}, dims);
    mat.items.unshift(it);
  }
  saveMat(); renderMat(); setTab("mat");
  flag(c.metal ? "Chapa pasada al cotizador" : "Madera pasada al cotizador");
}

function addMatItem(){
  const it = Object.assign(matItemDefaults("tablero"), {id: uid()});
  mat.items.push(it);
  saveMat(); renderMat();
  const el = document.querySelector('.mitem[data-id="' + it.id + '"] input[data-f="name"]');
```

```
  if (el) el.focus();
}

function loadMatDemo(){
  mat = matDefaults();
  mat.name = "Escritorio de oficina";
  mat.items = MAT_DEMO.map(d => Object.assign(matItemDefaults(d.kind), d, {id: uid(),
priceDate: todayISO()}));
  matEditingId = null;
  fillMatForm(); saveMat(); renderMat();
  flag("Ejemplo cargado");
}

function resetMat(){
  if (!confirm("¿Borrar el mueble que estás editando ahora y empezar de cero? Los muebles ya
guardados en la lista no se borran.")) return;
  mat = matDefaults();
  matEditingId = null;
  try{ localStorage.removeItem(MKEY); }catch(e){}
  fillMatForm(); renderMat();
  flag("Listo para empezar");
}

/* -------- guardar / cargar / reordenar / eliminar muebles -------- */
function snapshotMatItems(){
  // copia los ítems del borrador actual, con ids propios e independientes
  return mat.items.map(it => Object.assign({}, it, {id: uid()}));
}

function saveMueble(){
  if (!mat.items.length){ flag("Agregá al menos un material antes de guardar"); return; }
  const name = (mat.name || "").trim() || "Mueble sin nombre";
  mat.name = name;

  const existing = matEditingId ? book.entries.find(e => e.id === matEditingId) : null;
  const entry = {
    id: existing ? existing.id : uid(),
    name, qty: mat.qty, waste: mat.waste,
    marginMode: mat.marginMode, marginPct: mat.marginPct,
    items: snapshotMatItems()
  };

  if (existing){
```

```
    const i = book.entries.findIndex(e => e.id === existing.id);
    book.entries[i] = entry;
  }else{
    book.entries.push(entry);
    matEditingId = entry.id;
  }

  saveBook(); saveMat(); renderBook(); renderMat();
  flag(existing ? "Mueble actualizado" : "Mueble guardado");
}

function newMueble(){
  matEditingId = null;
  mat = matDefaults();
  saveMat(); fillMatForm(); renderMat();
  flag("Listo para cargar un mueble nuevo");
}

function loadMueble(id){
  const e = book.entries.find(x => x.id === id); if (!e) return;
  matEditingId = id;
  mat = {
    name: e.name, qty: e.qty, waste: e.waste,
    marginMode: e.marginMode, marginPct: e.marginPct,
    items: e.items.map(it => Object.assign({}, it, {id: uid()}))
  };
  saveMat(); fillMatForm(); renderMat(); renderBook();
  flag("Cargado para editar");
}

function duplicateMueble(id){
  const i = book.entries.findIndex(x => x.id === id); if (i < 0) return;
  const src = book.entries[i];
  const copy = {
    id: uid(), name: src.name + " (copia)", qty: src.qty, waste: src.waste,
    marginMode: src.marginMode, marginPct: src.marginPct,
    items: src.items.map(it => Object.assign({}, it, {id: uid()}))
  };
  book.entries.splice(i + 1, 0, copy);
  saveBook(); renderBook();
}

function deleteMueble(id){
```

```
  const e = book.entries.find(x => x.id === id); if (!e) return;
  if (!confirm('¿Eliminar "' + e.name + '" de la lista de muebles guardados?')) return;
  book.entries = book.entries.filter(x => x.id !== id);
  if (matEditingId === id){
    matEditingId = null;
    saveMat();
  }
  saveBook(); renderBook(); renderMat();
}

function moveMueble(id, dir){
  const i = book.entries.findIndex(x => x.id === id); if (i < 0) return;
  const j = i + dir; if (j < 0 || j >= book.entries.length) return;
  const tmp = book.entries[i];
  book.entries[i] = book.entries[j];
  book.entries[j] = tmp;
  saveBook(); renderBook();
}

function bindMat(){
  document.getElementById("btnMAdd").addEventListener("click", addMatItem);
  document.getElementById("btnMSave").addEventListener("click", saveMueble);
  document.getElementById("btnMNew").addEventListener("click", newMueble);

  document.getElementById("mName").addEventListener("input", e => {
    mat.name = e.target.value; saveMat();
  });
  document.getElementById("mQty").addEventListener("input", e => {
    let v = parseInt(e.target.value, 10);
    if (!isFinite(v) || v < 1) v = 1;
    mat.qty = v; saveMat(); renderMat();
  });
  document.getElementById("mWaste").addEventListener("input", e => {
    let v = parseFloat(e.target.value);
    if (!isFinite(v) || v < 0) v = 0;
    mat.waste = v; saveMat(); updateMatTotalsOnly();
  });
  document.getElementById("mMargin").addEventListener("input", e => {
    let v = parseFloat(e.target.value);
    if (!isFinite(v) || v < 0) v = 0;
    if (v > 99) v = 99;
    mat.marginPct = v; saveMat(); updateMatTotalsOnly();
  });
```

```
  document.getElementById("mMarginMode").addEventListener("change", e => {
    mat.marginMode = e.target.value === "margin" ? "margin" : "markup";
    saveMat(); updateMatTotalsOnly();
  });

  const box = document.getElementById("mItems");

  box.addEventListener("input", e => {
    const f = e.target.dataset.f;
    if (!f || f === "kind" || f === "preset") return;
    const card = e.target.closest(".mitem"); if (!card) return;
    const it = mat.items.find(x => x.id === card.dataset.id); if (!it) return;

    if (f === "name"){ it.name = e.target.value; }
    else if (f === "url"){
      it.url = e.target.value.trim();   // se valida al guardar/renderizar
      saveMat();
      return;                           // no hace falta recalcular costos
    }
    else{
      let v = parseFloat(e.target.value);
      if (!isFinite(v) || v < 0) v = 0;
      // al tocar el precio, queda fechado hoy: así el aviso de "precio viejo" es fiable
      if (f === "price" && v !== it.price) it.priceDate = todayISO();
      it[f] = v;
    }
    saveMat(); updateMatTotalsOnly();
  });

  // al salir del campo de URL se normaliza y se redibuja para mostrar el botón
  box.addEventListener("blur", e => {
    if (e.target.dataset.f !== "url") return;
    const card = e.target.closest(".mitem"); if (!card) return;
    const it = mat.items.find(x => x.id === card.dataset.id); if (!it) return;
    const clean = safeUrl(it.url);
    if (clean !== it.url){
      if (!clean && it.url) flag("El enlace tiene que empezar con https://");
      it.url = clean;
    }
    e.target.value = clean;
    // se actualiza solo el botón "Ver en…": redibujar la tarjeta haría perder el clic en lo que se
toque a continuación
    const row = card.querySelector(".mitem-link");
```

```
    const old = row.querySelector("a.linkbtn");
    if (old) old.remove();
    if (clean){
      const a = document.createElement("a");
      a.className = "btn quiet linkbtn";
      a.href = clean; a.target = "_blank"; a.rel = "noopener noreferrer";
      a.textContent = "Ver en " + hostOf(clean);
      row.appendChild(a);
    }
    saveMat();
  }, true);

  box.addEventListener("change", e => {
    const f = e.target.dataset.f;
    const card = e.target.closest(".mitem"); if (!card) return;
    const it = mat.items.find(x => x.id === card.dataset.id); if (!it) return;

    if (f === "kind"){
      const name = it.name;
      Object.assign(it, matItemDefaults(e.target.value), {id: it.id, name, fromCut: false});
      saveMat(); renderMat();
    }else if (f === "preset"){
      Object.assign(it, {preset: e.target.value}, PRESETS[e.target.value] || PRESETS.otro);
      saveMat(); renderMat();
    }
  });

  box.addEventListener("click", e => {
    const btn = e.target.closest("[data-act]"); if (!btn) return;
    const card = e.target.closest(".mitem"); if (!card) return;
    const id = card.dataset.id;
    if (btn.dataset.act === "mdel"){
      mat.items = mat.items.filter(x => x.id !== id);
    }else if (btn.dataset.act === "mdup"){
      const i = mat.items.findIndex(x => x.id === id); if (i < 0) return;
      mat.items.splice(i + 1, 0, Object.assign({}, mat.items[i], {id: uid()}));
    }
    saveMat(); renderMat();
  });
}

function bindBook(){
  document.getElementById("bookList").addEventListener("click", e => {
```

```
    const btn = e.target.closest("[data-act]");
    if (!btn || btn.disabled) return;
    const row = e.target.closest(".mbook-row"); if (!row) return;
    const id = row.dataset.id;
    const act = btn.dataset.act;
    if (act === "up") moveMueble(id, -1);
    else if (act === "down") moveMueble(id, 1);
    else if (act === "load") loadMueble(id);
    else if (act === "bdup") duplicateMueble(id);
    else if (act === "bdel") deleteMueble(id);
  });
}

/*
================================================================================
=
   8.b CALCULADORA (tercera pestaña)
   Parser propio con descenso recursivo: nunca se usa eval(), así que lo que
   se escriba no puede ejecutar código. El % funciona como en un celular:
   "50 + 10 %" suma el 10 % de 50; "10 %" solo, vale 0,1.

================================================================================
= */
const CKEY = "calc_historial_v1";
const calc = { expr:"", justEval:false, error:"", last:"0", lastRaw:0, hist:[] };
const fRes = new Intl.NumberFormat("es-AR", {maximumFractionDigits: 10});
const CALC_MAX_LEN = 80;

function calcTokenize(src){
  const out = [];
  let i = 0;
  while (i < src.length){
    const ch = src[i];
    if (ch >= "0" && ch <= "9" || ch === "."){
      let j = i, dots = 0;
      while (j < src.length && (src[j] >= "0" && src[j] <= "9" || src[j] === ".")){
        if (src[j] === ".") dots++;
        j++;
      }
      const txt = src.slice(i, j);
      if (dots > 1 || txt === ".") throw new Error("Número no válido");
      out.push({t:"n", v:parseFloat(txt)});
      i = j;
```

```
    }else if ("+-*/".includes(ch)){ out.push({t:"o", v:ch}); i++; }
    else if (ch === "(" || ch === ")" || ch === "%"){ out.push({t:ch}); i++; }
    else throw new Error("Carácter no válido");
  }
  return out;
}

function calcEval(src){
  const s0 = src.replace(/[+\-*/(]+$/, "");          // se ignoran operadores o "(" colgados al final
  if (s0 === "") return {ok:true, empty:true};
  const open = (s0.match(/\(/g) || []).length;
  const close = (s0.match(/\)/g) || []).length;
  if (close > open) return {ok:false, error:"Hay un paréntesis de más"};
  const s = s0 + ")".repeat(open - close);            // cierra automáticamente los que falten

  let toks;
  try{ toks = calcTokenize(s); }catch(e){ return {ok:false, error:e.message}; }

  let i = 0;
  const peek = () => toks[i];
  const fail = (m, div0) => { const e = new Error(m); e.div0 = !!div0; throw e; };
  const val = f => f.pct ? f.v / 100 : f.v;

  function factor(){
    const t = toks[i++];
    if (!t) fail("Expresión incompleta");
    if (t.t === "o" && t.v === "-"){ const f = factor(); return {v:-f.v, pct:f.pct}; }
    if (t.t === "o" && t.v === "+") return factor();
    let v, pct = false;
    if (t.t === "n") v = t.v;
    else if (t.t === "("){
      v = expr();
      const c = toks[i++];
      if (!c || c.t !== ")") fail("Falta cerrar un paréntesis");
    }else fail("Expresión no válida");
    while (peek() && peek().t === "%"){ i++; if (pct) v = v / 100; pct = true; }
    return {v, pct};
  }
  function term(){
    const f = factor();
    let v = f.v, pct = f.pct;
    for (;;){
      const p = peek();
```

```
      if (p && p.t === "o" && (p.v === "*" || p.v === "/")){
        i++;
        const g = factor();
        const l = pct ? v / 100 : v, r = val(g);
        pct = false;
        if (p.v === "/"){ if (r === 0) fail("No se puede dividir por 0", true); v = l / r; }
        else v = l * r;
      }else if (p && p.t === "("){                  // multiplicación implícita: 2(3+4)
        const g = factor();
        v = (pct ? v / 100 : v) * val(g);
        pct = false;
      }else break;
    }
    return {v, pct};
  }
  function expr(){
    const t = term();
    let acc = t.pct ? t.v / 100 : t.v;
    for (;;){
      const p = peek();
      if (p && p.t === "o" && (p.v === "+" || p.v === "-")){
        i++;
        const r = term();
        const amount = r.pct ? acc * r.v / 100 : r.v;  // a + b %  ->  a + (b % de a)
        acc = p.v === "+" ? acc + amount : acc - amount;
      }else break;
    }
    return acc;
  }

  try{
    const v = expr();
    if (i < toks.length) fail("Expresión no válida");
    if (!isFinite(v)) fail("Resultado fuera de rango");
    const clean = parseFloat(v.toPrecision(12));     // 0,1 + 0,2 = 0,3 y no 0,30000000000000004
    return {ok:true, value: clean === 0 ? 0 : clean};
  }catch(e){
    return {ok:false, error:e.message, div0:!!e.div0};
  }
}

function plainNum(n){
  let t = String(n);
```

```
  if (/e/i.test(t)) t = n.toLocaleString("en-US", {useGrouping:false, maximumFractionDigits:20});
  return t;
}

/* "2*(3+4)" -> "2 × (3 + 4)": espacios solo alrededor de operadores binarios */
function showExpr(e){
  const map = {"*":"×", "/":"÷", "-":"−", "+":"+"};
  let out = "";
  for (let i = 0; i < e.length; i++){
    const ch = e[i];
    if (map[ch]){
      const prev = out.replace(/\s+$/, "").slice(-1);
      const binary = prev !== "" && !"(+−×÷".includes(prev);
      out += binary ? " " + map[ch] + " " : map[ch];
    }else out += ch === "." ? "," : ch;
  }
  return out;
}

function calcPress(k){
  calc.error = "";
  let e = calc.expr;

  if (calc.justEval){
    // después de "=": un número o "(" empieza una cuenta nueva; un operador sigue con el
resultado
    if (/^[0-9.]$/.test(k) || k === "(" || k === "paren") e = "";
    calc.justEval = false;
  }
  const last = e.slice(-1);
  const curNum = (e.match(/[\d.]*$/) || [""])[0];
  const isOp = ch => ch !== "" && "+-*/".includes(ch);
  const openN = () => (e.match(/\(/g) || []).length;
  const closeN = () => (e.match(/\)/g) || []).length;

  if (k === "c"){ e = ""; }
  else if (k === "bk"){ e = e.slice(0, -1); }
  else if (k === "eq"){
    const r = calcEval(e);
    if (r.empty){ calc.expr = e; calcRender(); return; }
    if (!r.ok){ calc.error = r.error; calc.expr = e; calcRender(); return; }
    calc.hist.unshift({expr: calcClosed(e), res: plainNum(r.value)});
    calc.hist = calc.hist.slice(0, 30);
```

```
  calcHistSave(); calcHistRender();
  calc.expr = plainNum(r.value);
  calc.justEval = true;
  calcRender();
  return;
}
else if (/^[0-9]$/.test(k)){
  if (e.length >= CALC_MAX_LEN) return;
  if (last === ")" || last === "%") e += "*";
  if (curNum === "0") e = e.slice(0, -1) + k; else e += k;
}
else if (k === "."){
  if (e.length >= CALC_MAX_LEN || curNum.includes(".")) return;
  if (curNum === ""){ if (last === ")" || last === "%") e += "*"; e += "0."; } else e += ".";
}
else if ("+-*/".includes(k)){
  if (e === "" || last === "("){ if (k === "-") e += "-"; else return; }
  else if (isOp(last)){
    if (k === "-" && (last === "*" || last === "/")) e += "-";      // 5*-3
    else{
      const base = e.replace(/[+\-*/]+$/, "");
      if (base === "" || base.slice(-1) === "("){ if (k === "-") e = base + "-"; else return; }
      else e = base + k;
    }
  }else{
    if (e.length >= CALC_MAX_LEN) return;
    e += k;
  }
}
else if (k === "%"){
  if (/[0-9)]/.test(last)) e += "%"; else return;
}
else if (k === "paren"){
  if (e.length >= CALC_MAX_LEN) return;
  if (e === "" || isOp(last) || last === "(") e += "(";
  else if (openN() > closeN()) e += ")";
  else e += "*(";
}
else if (k === "("){ if (e.length >= CALC_MAX_LEN) return; e += "("; }
else if (k === ")"){
  if (openN() > closeN() && /[0-9)%]/.test(last)) e += ")"; else return;
}
else return;
```

```
  calc.expr = e;
  calcRender();
}

/* versión "prolija" de una cuenta para el historial: sin operadores colgados y con paréntesis
cerrados */
function calcClosed(e){
  const s = e.replace(/[+\-*/(]+$/, "");
  const o = (s.match(/\(/g) || []).length, c = (s.match(/\)/g) || []).length;
  return s + ")".repeat(Math.max(0, o - c));
}

function calcRender(){
  const e = calc.expr;
  document.getElementById("cExpr").textContent = e ? showExpr(e) : "\u00a0";
  const el = document.getElementById("cRes");
  el.classList.remove("err");
  if (calc.error){
    el.textContent = calc.error; el.classList.add("err");
    return;
  }
  const r = calcEval(e);
  if (r.ok && r.empty){ calc.last = "0"; calc.lastRaw = 0; }
  else if (r.ok){ calc.last = fRes.format(r.value); calc.lastRaw = r.value; }
  else if (r.div0){ el.textContent = r.error; el.classList.add("err"); return; }
  // si la cuenta está a medio escribir, se conserva el último resultado válido
  el.textContent = calc.last;
}

function calcHistLoad(){
  try{
    const d = JSON.parse(localStorage.getItem(CKEY) || "[]");
    if (Array.isArray(d)){
      calc.hist = d.filter(x => x && typeof x.expr === "string" && typeof x.res === "string" &&
isFinite(parseFloat(x.res)))
                   .slice(0, 30);
    }
  }catch(e){ /* sin almacenamiento: se sigue en memoria */ }
}
function calcHistSave(){
  try{ localStorage.setItem(CKEY, JSON.stringify(calc.hist)); }catch(e){}
}
```

```
function calcHistRender(){
  document.getElementById("cHistCount").textContent =
    calc.hist.length ? calc.hist.length + (calc.hist.length === 1 ? " cuenta" : " cuentas") : "";
  document.getElementById("cHistEmpty").style.display = calc.hist.length ? "none" : "block";
  document.getElementById("cHistClear").hidden = !calc.hist.length;
  document.getElementById("cHist").innerHTML = calc.hist.map((h, i) =>
    `<button type="button" class="chist-row" data-i="${i}"><span
class="e">${esc(showExpr(h.expr))} =</span><span
class="r">${esc(fRes.format(parseFloat(h.res)))}</span></button>`
  ).join("");
}

function fallbackCopy(txt){
  const ta = document.createElement("textarea");
  ta.value = txt; ta.setAttribute("readonly", "");
  ta.style.position = "fixed"; ta.style.opacity = "0";
  document.body.appendChild(ta); ta.select();
  let ok = false;
  try{ ok = document.execCommand("copy"); }catch(e){}
  document.body.removeChild(ta);
  return ok;
}
function copyText(txt){
  if (navigator.clipboard && window.isSecureContext){
    return navigator.clipboard.writeText(txt).then(() => true).catch(() => fallbackCopy(txt));
  }
  return Promise.resolve(fallbackCopy(txt));
}

function calcKeydown(ev){
  if (activeTab !== "calc") return;
  if (ev.ctrlKey || ev.metaKey || ev.altKey) return;
  const t = ev.target;
  if (t && (t.tagName === "INPUT" || t.tagName === "TEXTAREA" || t.tagName === "SELECT" ||
t.isContentEditable)) return;
  const onBtn = !!(t && t.tagName === "BUTTON");
  const key = ev.key;
  let k = null;
  if (/^[0-9]$/.test(key)) k = key;
  else if (key === "." || key === ",") k = ".";
  else if (key === "+" || key === "-" || key === "*" || key === "/") k = key;
  else if (key === "x" || key === "X") k = "*";
  else if (key === "%") k = "%";
```

```
  else if (key === "(" || key === ")") k = key;
  else if (key === "=") k = "eq";
  else if (key === "Enter"){ if (onBtn) return; k = "eq"; }   // sobre un botón enfocado, Enter lo
activa a él
  else if (key === "Backspace") k = "bk";
  else if (key === "Escape" || key === "Delete") k = "c";
  if (k === null) return;
  ev.preventDefault();
  calcPress(k);
}

function bindCalc(){
  document.getElementById("cKeys").addEventListener("click", ev => {
    const b = ev.target.closest("[data-k]");
    if (!b) return;
    calcPress(b.dataset.k);
    if (ev.detail > 0) b.blur();       // tras un clic de mouse/dedo, que Enter siga siendo "="
  });
  document.getElementById("cHist").addEventListener("click", ev => {
    const b = ev.target.closest("[data-i]");
    if (!b) return;
    const h = calc.hist[+b.dataset.i];
    if (!h) return;
    calc.error = ""; calc.expr = h.res; calc.justEval = true;
    calcRender();
  });
  document.getElementById("cHistClear").addEventListener("click", () => {
    if (!confirm("¿Borrar el historial de la calculadora?")) return;
    calc.hist = []; calcHistSave(); calcHistRender();
  });
  document.getElementById("cCopy").addEventListener("click", () => {
    copyText(plainNum(calc.lastRaw)).then(ok => flag(ok ? "Resultado copiado" : "No se pudo
copiar"));
  });
  document.addEventListener("keydown", calcKeydown);
}

/* -------- pestañas: Plano de corte / Cotizador de materiales / Calculadora -------- */
const TABS = [["tabCorte","corte","viewCorte"], ["tabMat","mat","viewMat"],
["tabQuote","quote","viewQuote"], ["tabCalc","calc","viewCalc"], ["tabD3","d3","viewD3"]];
function applyTab(tab){
  if (!TABS.some(t => t[1] === tab)) tab = "corte";
  activeTab = tab;
```

```
  TABS.forEach(([btn, name, view]) => {
    document.getElementById(view).hidden = tab !== name;
    document.getElementById(btn).setAttribute("aria-selected", String(tab === name));
  });
  document.querySelector(".brand span").textContent =
    tab === "mat" ? "cotización de materiales" : tab === "quote" ? "presupuestos a clientes" : tab
=== "calc" ? "calculadora" : tab === "d3" ? "diseño 3D" : "plano de corte";
  const pb = document.getElementById("btnPrint");
  pb.textContent = "Imprimir";
  pb.title = tab === "mat" ? "Imprimir la cotización de materiales" : tab === "quote" ? "Imprimir el
presupuesto" : tab === "d3" ? "Imprimir el despiece del diseño 3D" : "Imprimir el plano de corte";
  // ejemplo / empezar de cero no aplican a la calculadora ni a presupuestos; imprimir tampoco
a la calculadora
  ["btnDemo", "btnReset"].forEach(id => { document.getElementById(id).hidden = tab === "calc"
|| tab === "quote"; });
  document.getElementById("btnPrint").hidden = tab === "calc";
  document.getElementById("qBanner").hidden = !(tab === "mat" && qReturn);
  if (tab === "quote"){ qReturn = false; qFillPick(); qRefresh(); renderQBook(); }
  if (tab === "d3"){ d3Resize(); d3Draw(); }
  updatePrintout();
}
function setTab(tab){
  applyTab(tab);
  try{ localStorage.setItem(TABKEY, activeTab); }catch(e){}
  window.scrollTo(0, 0);
}

/*
==========================================================================
=
   8.c PRESUPUESTOS A CLIENTES
   Cada ítem copia costo y precio sugerido de un mueble guardado (o es una línea
   libre). Los valores NO cambian solos: si el cotizador cambió, aparece "↻".

==========================================================================
= */
const LOGO =
"
wFBMVEU1IxhoaGgtNlNQKChjYwAOYjsZZmb///8AAAAqNzgqNzgtODgqNzgqNzgqNzgqNzgq
NzgAVVUrKy8UOzoAf38pRkZVVVUAAFUAAH8rNEYsRi4pKUwULTAAfwAeHj0A/wAA//8AVQB/
AAAqVVUzZjP/AAAAP8SPxJVAAAAAD8fHx8YSEgqVSpVAFV/AH87ADsAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAqP3apAAAAMHRS
TlMOBBQGAgsFAQD60BItbk+vjwMLCAIKAwMCDwsHDAIIAQEDAgYFAQEGAwQICgYDAgSD
```

2U1RAAAfWklEQVR42u1dB3fjtrJWcnPvo9BJSrYl29nNpvf//+8eAbaZwaCoeb1rMzknsUxTxIfpD
ZvmM1/t9N+9lEoZOX3Y4nvIzy9xbZrPf3XKaGetENutaX5tXsf1uYHp1YDJgMh4CeU/G0hngGq4
nHP+P2b4Wckf3gAwHyaOaZS2W3i58JEQ2+gSwjo9c9rXCczDsHTbe2IxjiJgmt5u05f4ioH53viVi
wEYE0MgDo3J4LLV/3ytwMgJDdtIxy28adwrlZgXBUYuEsXsOY4ZRK8SGWBUs/sagZEr71ipuY
UPolfnGKnptNPq8JUBowCN6I4lDNNsMgTjulEACWf2XwUwQTDsNVyyZEXsoKoyBGNlI+dnCK
f6Lx0Y5Uxz10iLWcbyBJORMF7wQtjsrW2a2wLjCcU0dMFKJkgirZK8SUzITGj1xQIT5EpktNlWn0
owHpf4twNHfZnAGDEqWUO1i2UJRrrTcLktNLcDptPzojSRFpJXxiopd1O4BIaSXxgwvVukpiPWi
mGFq0x5SbprsmL5NtDcCphuRsMSlTTIEt4bUJZdvGeWo8kZxIOG+nKAWeWFXc2PmTQEr43/
VUbH4Hi1LHMeVEBPfiHAACJxRHY4ni20CjabpBGa0WKRxuaRub5/ubkxLl4LE05qQnwuog1hx
0DURmkRs0qn89zUfQnA9A65xEQpORNU7B1DHF79PobAnogFjcrwk/4iWGnvsAniGPNjBEfG
MbzJ1qcoCL1vpM6YOa8dGGNI6GCgcV48WK04BPw6PTQtVURevvJK230B6toMUhBbKVQplY
ljhqZzMV0wtk64t5GvW/h6TAw151ROmXjh0DFWSjD1yefCMMiEG5W7vm9wTWACpRMCMPn
w9ggNY6ewnw82MEJmhGMU1cK8WmDGsBwBRmWjlSMh9GwM3IuOXkefrcgIo5AGu66suR4
wh/GNlcaC4cFYy6bQIFV1ti4Gs3WP4F5D1JpVrxGYD24yQbFS8r86dlIq42xa26pE5OpomYD5q
puo23VNdroaMBMe4g4D47POj3OawPHqtvnXJQiGYUPdrmQUG0n602sDRi8korm0swlGS8MY
u0mC8b5D4nOdQUa+LmD0Ng3MYu36d/7TxAEHNhJjGv5zD8b3q+8uabL7WiJ4c11chmWmtdC
oXW3kFxs+W5sKNgyyZxUzLjIgr4TMNYD5ny6FoeCaIsrgCEM3maykhtrKRE+8DjJXAGaPnWm
bzbLS5eoHVsJ+ymVrByK7c4CzdKy5XgMw2BbNA6Oag0izBaIXk4/Xrb+2h4jkBrQ/PzDKkoWaU
CPGBnAdlScDjo/2HFwgmkx+wXx+YIijpxf/wAdw4/f9wyZVL7TSftZZdsRUxjDTFZC5DJj98EZoqd
r4SkK5mcDRedGrn2NG8nb9lZOrNU1P/8j1UWjj8tjV5lI2snG8ZbDorNOqHeRPtsLOHiNdGzxKZ
XNs1DYmLt3TjP77fMCEFzRJaevkzuQlZvh57+LoikyHvYXPvf2mGSugFxzHfR5gphRsWg2JDm
+kaX60mGdIFNfD8pRJlAT7kDX7GJK5VMxsLtTSg02RTK02aB9dj1SNjXzEQS7INlPxYLpY2M8
Pe2AYWn0WYOYXtM2fNhlmMdiGgfZGyJAR6hD6l0YmBEwyPpyOh11mAZ8HzMoDwVcRgolEk
Wy++we/u1ZdvFLHxzmt5tNNhVoJ8+LAgHc3D3JQzzK0juBwC84PiNjEtSFEgCnE0z+OZ4opz9Ir
lwkD2vYgrptv2pyppVcahoAZZJCaUm0zF/E2xOwN285lLSkGLjYazMsCA1cvfB7ahS6RsK/g7dT
OYlchYa9RgYo9lb/kVhVS+lCgDSamv7yZKfcvCgxRJQKEWySo3xVSkbg2vzrd0wxjjEyxDAQIma
u41mcAk31HpaDv70i6cGNcjU84Wq2mGPyloYZZoqk7GZqd1IsCI/NNEAbWrIq4+FTxZYk0ukug
GAR3X6CZQZzsHC7AuTBlsDnHqku9HSCSOPYbApuSM9AIhUzIOHRDDpkpnDznKeT/2St4BZ
vr4Tlwxfp78yT4MhaOaAaj+DcXFchb1EeQiIx7b1ViXbkalvqFgDkwuQ9oXBgFWL7jskihYkHzoW
9LWyoUvgHKLyvGVkCljuOOwVo0EB83LwIMt2eD6FhfCYRfnN9GzlJNROfiJBKJYaEfjbco1/dSu
G4N2jTmBYBJdCsKtdcz+Qvq3DDU4SMHpiLt6LW0SPwYCG+0tjVX6Am+QN0cmLQ+MpOFZjT
jDscixUkOGRdVCjl8m9uDH518dKyDNgn+1Z46u5xzc55dR/bWBBJha91jdeIk8zTmQ0wyOAGlct
Fy5L6e62PXAmOyabQH5yUiX+4eR9w0jXnybRQkCSVgANBI20pR3MrcFBhVKHAZXkRU5IBS
uQG2vcS1+C9d8+wAyaicGawu9rHrgMFNr4LKOyHTBGWj8h8XaaDQXmKZP9TJsogsyVhUlu/u
bgcMarpTsqcGrHVZjsd/L2kBFVupqg9N3OjUVJIMDkKYmwED33oSZmpbfQ12LdzASDF7/9owV
flHHUOMTEhbJ2TOq6evAsbhl9PWlWsOkRFrUjbs6OvRGkSb8B2gYhI5xYSDZGeRTA0w2DYP
m4uiLcUYEiAYJ7ESZqK8k2nIeZt7VInzJKqk73nGTAUwjyi4Py3LSFGJC2qnpn6yvms+IDYaPeX
e2GIZ47BanQFGXJh925xmwix9e3yLcHExJNCilrzdEvhumkx3koaOrMl0UcrfxYUJ2zIwyHecmdf
+WMtKGhusCOY47BJGEIkMV0KbNj1sJkoDmhsAozl5oRpXKXmheR750GTMhQIDQ1hPXrbg/h
Qv+SLR3b29MPlWBEZSZlXW5yVMpYCBL29itayIZE9KrilKh2QHiEip8ZqyAw9RDeTp5u/mJIIJ1
N7LVAqZM3thGeFfse+I2CE1xWAggTEghROXa0TKfkve+lfG8bouMCpaqpTSuGrBa4C9wjWZIC
hgbZQII7XOGdVNASny9/buYQ1nTglRGQgnZsiTxe+m3rZb37heIWkQ7u1cMfMPFI2P0u3XOB3
bQbk+3+vn+dpyft3JvLQ5P9pQxgWQyGD3S1O0UUG41nmeHa8wP7BQT5HSBqaf2jvMVYGptu

JYXJAgsoqF2eKimeaTBZUdLA3wwjo1GkLP5HZNYO5PcIgYRe3iNLVihIxKxLF1mVz1OrvJPvZ
8SdFAqIGL+mtSTD0jBSlprSg4TfETidMFpJIr0ysMyiTc2jBMQ8v2qgZeV6mTfc7Li8nf5F5lFRaDjK
N+sE4UHiX4RG8LvBRo8Lq+UkWNwWB3zYnARKk4RWbn6CcunfkweSLVMGxqf3y06dDMVV0
CXUZlmqAFCLWlcVCqYEmp755a1pm+kkUhhyQkCZilCGzsE7qm5WsqUTlJl9GmCioZIDDUQb
BytPvnLGSLXYQEgU1PvJ6Bl88L5EaKZcfZYbcxaleHkoICI36iMlDCurxEFGJ+4qlu5Kbe5CUxtu
TVZgPCioQnwyjjsWFlUGqQYqKori+tGpsVBm9Ra9LhkvBAXbM5K/KwqfSR4FeZ7kQHK5UUIe8
62LiwVM/2jUuGfDkE/rC5KOeJ8c3NqZK3PEQhQzECKRJ3P7ij47UIU9CEHIlT3eSZdJTrIhX+dV
cBxp0LiwfGZATTagSQQNM8cWgpH4kCUbbLaYSBwGzsiUCppW5GMbUjN3QSFjj5g1Ejs6oLg
5ciYERXSOXbkeF6zfe/naSZNvUqt3oSyR1XeeUbkVCUOyEsJ8E+mIkxMDInvWbOsxKOVEYm
o7mGVjInaaKc7J145lBr3vVd59I6r2+eXBzTVTkTaaAnt8j3VVRBuhTdVQw8lO4x9c9cpIjVvkxbyf
9+E8+8cDlrJ8QIesMAs1jVcSwCDH1y45QnM7AURldfAxjg9Jw2tXF1XxYLRTD57KxlHdh2R4EB
81RcFNFzzbMF0P40qjjTkNjpNbxrT44D5oGJ7usJpsIn9zWZ+duEOTZPFDyDh8co7QDh6OZXC
NVgA04hGItrbq4T2vTuyV/Xqx1f1YMs9k0MouKeREPRBJ6xEdAbwb5C3hiSFDfGJxckDZ6oKw
Fz+lUR9POJ/ZqQhqEuGxlNZA0eHY4jO0cxhkUiW+jzAFOBix8woHNuu5vVlzvg8l6hfOoVnlJdJ4
d/iWGX6u41nwWYCkIQJhvSmBSKXT0rUDzDq2sfoIkj5rS7d8mjyM8ATFlwhELmtOBdbEiF2Mm
yCd3S91ga/DXak6J7eWAq8rZjVMSlY7hTNHCFWK8F5okZPAlzj5ZCmEnRVs4+27wsGz1HCjS
RXyYVzIFoTgEmHmWpp7mTldMnrwXMo64gF5V13VGMjXS4hRorS1r/ty4XhIiy7gO67joG3ilcV
CFdZmcrkcfDVqmKIBtERDSXXWfyEXHN536wVvWLAlNjlbhjIV7jMvbzAiodkpHaECGjZMqU5XT
7lwOmovsXSzxOKxE3JpriplgmdImRB9xQ2Ckj4Z5eCpiyQKROKGeN2F0+hOif0TVMw8aBa6Xj
ZpTMgfa6fMHlwOh6oTtfDzavkvjHOnbSsQrjx32hCDDy2CnCSwaiqrlgc3NcuDZfx6VVinQozC8x
Ml6HjwM09qNH6S/2HJUVLf337YEpl2ocq0LtkXNnEuGKaMm2H8AKOfQ+n8Cx66f65sCok4Ru
Bhhdh/jwvH0kgLsxrhdS2tNchzH6S1pkvpGi3pW8DBgp6kyX8p+ZSlLUcUvGIE35SmgTQlkgCnQ
Q9aG8S4DZFQpYUnYmV71JOSnVma+8M6WjwNfepSyjtfkYeQnFVMrmGoyk2UHFJnGs7qMr
WXdpgnG8uyrYeiohe/kcsAkaHbtP7pbATCs0quT5FM1kejZmIjAhVEq2sQedmaOd5hV5wjG0K/d
mwKiZ7Zml6t0pMRt7V1eDYtIU5R6e4kbmv21IVsyijk6ivA0wc78Ddy5QekQJb8GrOlwgFXK13xG
HzXJ6qohT9A9uQzHj3sM2ClfCZcfHsnQdLigqyWp8Cjuc9GMdHSWgbwSMmhp0InUssiaFi6oIx
KQPxUF09b3lkjKoLSyE/wRvRpgQcFe3AGY3sjkoHVvGhesULDwlYKM3fYySLkhwX/lIiMYlgBnj
OzbnTm4uEzFgSMfSFc3z7iEV4rOHKnJBrh9v5QQbBz3C8hWkQt6pMBmqvwkruS0c2eEWvjdp
visGNLP1nsuVqJNQUyR9fYrg77S+XNzdSisFGJaKOitnNcUSTNLXhAE1qbPVL3VWMWI0cdC
ZSLm5HTC2u5/IRC0hS3MSvfT5YARndKTqaoZvbsIGiNSJ46poyVwGjG6mah29OIdcpi/Z7YrMQ
FPppOsa6TyNfOOBWWJ5t5ExOiSTAzAzGobjpGQVJzF3bDoy+t8qj97A2v1gZSUOHFk8K3UjO
2bAYyyLc6v05HS11PQKJazktcz4Mb00iV0ol7j0Ad3nFRJ/4MjaFWXbm/hKZoxxa/8C8zfIGx9cfpp
lzh84Yj8uVKdumSV4NOV08K55mP/3uXke//8hvmn48Pl6yCh9ZPkOVLi62ybc7prXerHAKNQt9g
LVDq8RGJcuCM47kl85MBzJaGhWubcKzGodacWURKd56asHZo4YWqmOoWTfNlWTvb56YG
byMMb7qz8qTQr59JsFZjNm47wYnkzt+CTLNwnM5NqquaJPktIb+WaB8VJmGe3OHQr2VoEZW
MetgxJsPPSofavAKC0PuSmLbxYY3gA2U9j8bQMT279237Q6Y8i8EWD4MxeDb2D4Xqy3AUwc
lvExtseAzJumGGZGefMUugpUqiT6jQBj4piMGi1hea4dI5drl79xvxy1ZBQdtgOeQq8pUP+f+DfFN
5u+Dw5A4K84HO8JJl8kUwLmzglr/aA+my3Dp2PqLI50q/AMwV3Tq2kx3jHcaKcrCoPD61HhfmX
hzL5eLa2TC9V5wDwgqGnZE6BUm6vAa3MpownuxDEZiTzqt9wX5lrmCTDrJMuzI3iuwkNPFSjo
iU12ZwOTKFlLNS+kh91hfQ0an86N4CGkeUf0QzoTPyXfysB8+M5WZWfHd8odbSxrgAEzhs8Dh
vKAYXjpTm9LKecyMB/vbHXN/Z/ZGlp74BZyv4x9cLQI7EyKcaWm5UKrUqCZClb6ZIvw1n0hX2w
x946NE4UVGoR1FjBK5Dshyj0Wg0tSQzHf5M9A6zI2bA0yU4WKawa1rtD83LOAaaNlR48ptvt6u
XSR8CXfW9Et6lIrsVKNdRNmeyHFRL4X3Q16g++yR7M/rKKsJPjSXTDIw9GHALFPNsLpMNaY

FPcZnpXGowPIgVbnAWNKvRD0vOqpCHuxMkZdC4FxilySADNhsMdjpg0nYFYjB59RLyRHMa
G+bEtPsDFnAeNO6ehEmf1RhbtYXae0gGVWpeMegnTXrc0bXNqtcxHspcBI9ti+HU8wfkDFD8s
Ghb01H5sLgGnhxohgK/5o67iaMbhkx5pb5yX1DSMY4GagU3ipp9Kt3tK5FIPP88ASJg7uS5Glg1
5srwUMMLpWtCEAtcdenQ2MJGfU5Q8gM4maR97ySFpmZWDWddv/2NiUaau03jWBATWO7IJ
cmhBSfuxZhUM6bswNj/oQ35AfynImMPeYle4ghRruW7K/1wkDqT0ZGHRepWQOXoTzoppaYC
wuUuwywPwOhdyhgSlEfiMARcULphbXuTV4LdrlXWNjrpGijpNylu+S1AHAPPy+Gy6saOxjAwfE
2RKNu2KkyhUoPUkx/8BDbtA02WP8XeqawHCmiSusm4pfm7dV50OfMpReMQF6bGKJUZC15
5qeBMzWjBYxbkAlzbMshSKSwuv6gG3VoEPdNplTygJD38JF26W2NwEm7TEXfGIY8rZEXz+R
bsjO+JOzxBn9SnusoFtG8t2IYtJNogWKucsAM/5mnIox4HJ0WzWI6pzOKI/GnkyGWNTK2r73i4H
RTQ0wcPPs37GJI3RwH4eX9UlrNzzOnAFMTLbrJ/bf8MFBVPYwZ4G5LwPjvq+0hpLSee/G8ZL
DXg6+bojGCqmybbSbCiMmkuuKcIteXSOzE2eVVMLMQPTyR+y7ycJuOiJVH5sw5Org3HzOk5
D3p7cXIyNmyRHCquGPhIbaass3kW60FaFwtc3bB4jbUU/CozFhwoMfpTJZAaUJipuChb5dEoP
UrUeBT3U1lyCTVpIFU9sk30evLD/7w6VppJti/jGtJdQWGcc3AQbJgfxO/GVTrLYeTyVdHffzwDyX
Y/8B7r3IhR1WNjkNmNDnxbt5KhtYMEmneZ34LyL7/SRWKg9xVnQHIwkv7QLWGd61ZWnjT5tL
WIgUPans+IxTgCnnKFz0jaSZy4u4WW6eRjFq99TDJ6+Q43SOHwucwAXvErfN5SFVm7ogOBs
SfP7ZIfVxjNLL+u50iokmAZrEGmElBDqjgRCMy3f9nwCMqhiHrpk7/Tjm8fDCdSDdEwFG0IZQxQ
MDngwlqSFFIoefmuabJ1Kc40rkXzNWcpM1IDPit+dvpaOY7amZyAGYez534q8f6FGCNh7ujxmJ
2WVXcwIgA8zRwnibmS+vLSKZ+GuJ63z1wVnAJIL+5SHTWEFyhazfNOcA02bsJCgTddWLCtW
cDkxLWQClVArl4KKR+PVE5TT1TU70hkkW0FLrY87PIjOqqtNKzVQ0HZsMJ7D19BIL3upRx5uc
6l1DykxmJ1OwUl1RxQFDhkA/VFXlEKMhGphnvmvOBKbP292KEYnJ0q+57us8VsKnymNFkxpi
pb/P4WLNJaflwNB87DO3rEnKVic6VRGPyVEMsh8NKcPjRiVHwyJIvetJ0xViYKISXLJZy5HCGp
C3Im+K6lB34K/oNQPjpkANUrZ6vS3K4WtSQWO1TGzx8OfZkuFaYPwB0+nS7HY5xRIUY/stDu
dK2/nYz65B5432bFF4eFC3fm34RJJ3SdaJd2G6caiZ9utm3nawNc34rh8a/D7naaULrq6Xsm9e8
nqSfTcHS656fSVNFg+JQFCz271tYK5/vQPzDsw7MO/AvAPzDsw7MO/AvANz1jV5XusHe9Rh8
FaBmWugQNhN1xTcvAWKmVryuo+jxzOdj6rfPCvNyYA5Z+mqSjTehPDVKxRLbbdp3oGZ42wa/
L9r3oFpmlX+3k1B2lcgeV+HHeNmMpGvRvK+DmDkPALTsSW6b9jyNVPWp6Yv4U0B85etaq16
g76SqmlSfIPA7FAd+sM7MJH8LVffvrmwg9m+Hpv3VQHT21cleV8NMO1yAlH7Dsw7MO/AnH+to
5Lu34F5p5h3YC5hpXdgUhZemK6pmuYdGHr9/POnpnkH5tVf78BcBZj9seu6cujxWcqu8yO5v5ir
9+tC3V0bqVIXLTlW2oVZxdbpTG453DbeZ53JSNMu+U3016mdeJDZJzQy/w3LbXOBsl/Y8l0bvi5
fDP/YZ/TXtmaoMD2hWDhzTEYaxPxd3CwBJcbfi7TPPd+ScMulnX4tcr045I2teZqASbezwfPc414
Ybqgw21OQOhKwMKZPbUvt7rBvx2ZiPLkmSKaTZVpYFTA9f3ApoYVjsguF4wWVHS6Ge+L5Ib
3w6zhg3LY4jo7v/gzTAWqASc0yxlOrMw1W3HhrnZ3yTIYFFA/+taVF63pcRiKvAUbXjM7Ntt7GQo
QMMqQtYwQY0eUJggNGFydDp1952KgKYFSpYb+IS0QzUQdTHARXhQZy/HubCpYCottlZFTc
yF8BjC3MvghfUmrV1rn9jpmFHrEQT1S32QmI3BENeZKiUjEHzD/R1gyaXi8HJQAdGB9oLURm
YbsYSFdYGGVGU1q2y3eR4r0Uzk/aEHAXATCOHL1sxjeE/aHh7fpRzMJvMtR4CR37Lj3EShfG
pMY7bpKrYlkpJmEyawe1d07DlY1dvwoAkzCCmCFrncZrxacXrD14crIeI9n7qy1MvbyPgMHPiNpj
KcHFXZj2pxTNhU0Zu+D1wggQmPZ/LbwiLlHQlEsd5m7B+NZJh8faWpUUF3Oqus79PaWYR5v
TFOSlFz78wceFVGT5GnZqkLTFeUuQYKh6OWruMBm3Lbw3A8xKog+dLQGjyiNqCwMqi8B8cs
VhNCZrYcockiIxdooBBtzA8EkSeZsaY6jzZjECps3LGDE7WPj6xZ0wyIcszCk4mTYPzDobQJSAg
ZNB+Y3FRwKov/PAsPF7U3KqO1ExaTrhP5qOnzLIAjM/XRfZBIxQ7AycfZvitnhhEBjQHf0oO954
Fy4Kj6DBy88VuKDppY4Vv21msBT3G0wxv8HRhanpusREHGyMQwKYdeJdiNjLpGpE6O7KAw
0b4g4gscdP/eOBGQfoAt5OAlOfCe2NX7KqDS4safmui2TMWn0AS1ifYL+vYSSs53p2MCW0Yy
wc2YjO2F3nJtiEXLUdmiu7L/gEYDxEBTCctwWsuPJgZpJbMxBIiCvyoVdgHJr0A15GS55i9lBuSc

hLWIz+pLfphdUAw05sWT0ld2LFJTGlE7PhATBw7P7PcLQ9CwwdPdxCMUbVo0gurAoY/vQ6nf
YZsqKXCEM4rP7TaoMCYJBbA7df8azk0uP6VE0wVp8ATNMZm0LmNFaCt8ukpXWPgGEjBIM
S44FRlHX63Dh8Gr6f/6wWGJ/PcPzZSGClFfPCAF+YSP9oTis5RFVw8yULTHyORP5kCRnvuTo
FmIYZnyP5aeKVRkzX9oOv2oOByYspgymGc5Y1EquWVXq68V/w1KqCaf473fPhPWA8xqxD74
aLXeQTOg7K7/BDbjxcnpMY18fwrPSxdfG+JYDhht8DMfaQyrrZZGizzs9pobwajPRd/YkWxItxZg
6JxaY9Aabh41asjAH23xJysxV718O73HcoHhNHYwqbHk5hBbQrikMsS4NfVQIYeioaQdlWR58z
e9fB43aKYYeCE0iCn8zXPhQjMex7U2CQ/l0mldpq5EVfQ87i2ypgVPJMO2qixfwonZP5ABKfR6
HAoBXrJgVMeRYnjK4ozFgAfFWOx4Tv1zIDDEyweLvxfrVGgtFsVU3KgiAbAdPH50oxwBSRh+L
lkBgViIGoCuEbNmGM8d6Pi8UWFPaEBVRnRyOQ4VbeT+ZkFRcZbqZJAPNQgfyKhVnHO7b3L
VZoEBjt59+paQaeGv45wsWsjif0nMbP7hznvcPDwU2dAFje+54BZlm0a1LAVEw7X8WYocEGOF
z5CA08OtVQkU22OpzvwwznJglaP36YzB8ez94qz9ye3rvlgJkswXXLY2AqkJ8Z0aA8mFLIQnMf
M5nI0We3VTqk+EKiGxYL9hNHxSw9APIHDpjpWzTjFU33wFGgwq5n1Q9fQOcmm4LVUALGV
GDvA+K6LFNJUlN2+zWU+hMJqLMUM7YI2qc0MNAN78YnT1/yBz21IEu87q4ATJs9jhyK69aVd
U1v014VmYfLAuMzhRqOYI1YKXOQJDg4OkzGzbGCyNfHTIaNrjMi78oIonMld8l45+iaqopgBgU
Gsmqfsdd1QUOqpgaY5FhhvSuHw1Cwz6U9lmfy3ucBYyoLasZhAClmGIV7BTD8LGqmEJIdP7y
mrqVIFyegCir7WAkM9ZUQ8m2mokal45LzgOYaYJgwVzxWOBEOAxOP9TZrZKNVyTqKgUjv+H
NBWP/VcR71ZNXMWQKlk5dE7tJimAir08O39+t93pyBZrBiDhRt2F97n70DP6UjBaiW51ud/RsJS
n9+XqMoGrwvqOw+oWS+G4yg4VKykD06ynCfLx76IsrlJfu+/w+ihkC4lFTf8QAAAABJRU5ErkJg
gg==";
const QKEY = "presup_borrador_v1", QBOOK_KEY = "presup_guardados_v1", QCFG_KEY = "presup_config_v1";
const Q_STATES = [["borrador","Borrador"], ["enviado","Enviado"], ["aceptado","Aceptado"], ["rechazado","Rechazado"]];
const qn = (v, d, min, max) => { v = parseFloat(v); if (!isFinite(v)) return d; if (min !== undefined && v < min) v = min; if (max !== undefined && v > max) v = max; return v; };
const qm = n => "$ " + (Math.abs(n % 1) < 0.005 ? f0 : f2).format(isFinite(n) ? n : 0);
const q2 = n => String(n).padStart(2, "0");
const qToday = () => { const d = new Date(); return d.getFullYear() + "-" + q2(d.getMonth() + 1) + "-" + q2(d.getDate()); };
const qFmt = iso => { const p = String(iso).split("-"); return p.length === 3 ? p[2] + "/" + p[1] + "/" + p[0] : ""; };
const qPlus = (iso, n) => { const p = iso.split("-").map(Number); const t = new Date(p[0], p[1] - 1, p[2] + n); return t.getFullYear() + "-" + q2(t.getMonth() + 1) + "-" + q2(t.getDate()); };
const qUp = n => Math.ceil(n / 1000 - 1e-9) * 1000;   // precio sugerido, redondeado hacia arriba a $1.000
const qNum = n => String(n).padStart(4, "0");
const qDefaults = () => ({ num:0, client:"", phone:"", date:qToday(), valid:7, delivery:"",
  notes:"Los precios corresponden a los materiales y medidas indicados. Cualquier cambio puede modificar el valor.",
  disc:0, dep:50, status:"borrador", lines:[] });
let q = qDefaults(), qEditing = null, qbook = {seq:0, entries:[]}, qcfg = {shop:"", shopPhone:""};
let qReturn = false, qTimer = null, qBookTimer = null;

```
function qCleanLine(l){
  l = l || {};
  return { id:String(l.id || uid()), name:String(l.name || "").slice(0,120), detail:String(l.detail ||
"").slice(0,200),
    qty:Math.max(1, Math.round(qn(l.qty, 1, 0))), price:qn(l.price, 0, 0), sugg:qn(l.sugg, 0, 0),
cost:qn(l.cost, 0, 0),
    entryId:l.entryId ? String(l.entryId) : "", own:!!l.own, mode:l.mode === "manual" ? "manual" :
"sugg", inc:l.inc !== false };
}
function qClean(d){
  d = d || {}; const b = qDefaults();
  return { num:Math.max(0, Math.round(qn(d.num, 0, 0))), client:String(d.client || "").slice(0,80),
phone:String(d.phone || "").slice(0,30),
    date:/^\d{4}-\d{2}-\d{2}$/.test(d.date) ? d.date : b.date, valid:Math.round(qn(d.valid, 7, 0,
365)), delivery:String(d.delivery || "").slice(0,80),
    notes:typeof d.notes === "string" ? d.notes.slice(0,600) : b.notes, disc:qn(d.disc, 0, 0, 100),
dep:qn(d.dep, 50, 0, 100),
    status:Q_STATES.some(x => x[0] === d.status) ? d.status : "borrador",
lines:Array.isArray(d.lines) ? d.lines.map(qCleanLine) : [] };
}
function qLoad(){
  try{ const d = JSON.parse(localStorage.getItem(QKEY) || "null"); if (d && d.q){ q = qClean(d.q);
qEditing = d.editing || null; } }catch(e){}
  try{
    const d = JSON.parse(localStorage.getItem(QBOOK_KEY) || "null");
    if (d && Array.isArray(d.entries)){
      qbook.seq = Math.max(0, Math.round(qn(d.seq, 0, 0)));
      qbook.entries = d.entries.map(e => Object.assign(qClean(e), {id:String(e.id || uid()),
total:qn(e.total, 0, 0)}));
    }
  }catch(e){}
  try{ const d = JSON.parse(localStorage.getItem(QCFG_KEY) || "null"); if (d) qcfg =
{shop:String(d.shop || "").slice(0,60), shopPhone:String(d.shopPhone || "").slice(0,30)};
}catch(e){}
  if (qEditing && !qbook.entries.some(e => e.id === qEditing)) qEditing = null;
}
function qSaveDraft(){
  clearTimeout(qTimer);
  qTimer = setTimeout(() => {
    try{ localStorage.setItem(QKEY, JSON.stringify({q, editing:qEditing}));
localStorage.setItem(QCFG_KEY, JSON.stringify(qcfg)); flag("Guardado"); }
    catch(e){ flag("No se pudo guardar en este navegador"); }
  }, 250);
```

```
}
function qSaveBook(){
  clearTimeout(qBookTimer);
  qBookTimer = setTimeout(() => {
    try{ localStorage.setItem(QBOOK_KEY, JSON.stringify(qbook)); flag("Guardado"); }
    catch(e){ flag("No se pudo guardar en este navegador"); }
  }, 250);
}

function qCalc(src){
  src = src || q;
  const on = src.lines.filter(l => l.inc);
  const sub = on.reduce((t, l) => t + l.qty * l.price, 0), cost = on.reduce((t, l) => t + l.qty * l.cost, 0);
  const disc = sub * src.disc / 100, total = sub - disc, dep = total * src.dep / 100;
  return {sub, cost, disc, total, dep, bal:total - dep, profit:total - cost};
}
function qLineFrom(e){
  const t = computeEntryTotals(e);
  return qCleanLine({name:e.name, qty:1, price:qUp(t.price), sugg:t.price, cost:t.perUnit,
entryId:e.id});
}
function qEntryNow(l){ const e = book.entries.find(x => x.id === l.entryId); return e ?
computeEntryTotals(e) : null; }
function qSync(l){ const t = qEntryNow(l); if (!t) return; l.cost = t.perUnit; l.sugg = t.price; if
(l.mode === "sugg") l.price = qUp(t.price); }
function qStale(l){ const t = qEntryNow(l); return !!t && (Math.abs(t.perUnit - l.cost) > 0.5 ||
Math.abs(t.price - l.sugg) > 0.5); }

function qFillPick(){
  document.getElementById("qPick").innerHTML = book.entries.length
    ? book.entries.map(e => `<option value="${esc(e.id)}">${esc(e.name)}</option>`).join("")
    : `<option value="">(Primero guardá un mueble en Materiales)</option>`;
  document.getElementById("qAddFrom").disabled = !book.entries.length;
}
function qLabel(){ document.getElementById("qSave").textContent = qEditing ? "Actualizar
presupuesto" : "Guardar presupuesto"; }
function qFillForm(){
  const $ = id => document.getElementById(id);
  $("qClient").value = q.client; $("qPhone").value = q.phone; $("qValid").value = q.valid;
$("qDelivery").value = q.delivery;
  $("qDisc").value = q.disc; $("qDep").value = q.dep; $("qNotes").value = q.notes;
  $("qShop").value = qcfg.shop; $("qShopPhone").value = qcfg.shopPhone; qLabel();
}
```

```
function qRenderLines(){
  document.getElementById("qLines").innerHTML = q.lines.map(l => `
  <div class="qline${l.inc ? "" : " off"}" data-id="${esc(l.id)}">
    <input type="text" data-f="name" value="${esc(l.name)}" placeholder="Descripción (ej. Mesa
de comedor)" aria-label="Descripción">
    <input type="text" data-f="detail" value="${esc(l.detail)}" placeholder="Lo que pidió: material y
medidas (ej. Guayaquil, 200×100×75 cm)" aria-label="Material y medidas pedidos"
style="margin-top:6px">
    <div class="qrow3">
      <label><span class="lab">Cantidad</span><input type="number" data-f="qty" min="1"
step="1" inputmode="numeric" value="${l.qty}"></label>
      <label><span class="lab">Precio unitario</span><input type="number" data-f="price"
min="0" step="any" inputmode="decimal" value="${Math.round(l.price * 100) / 100}"></label>
      <div><span class="lab">Subtotal</span><b class="qsub"></b></div>
    </div>
    <div class="qint" data-int></div>
    <div class="qact">
      <label class="check"><input type="checkbox" data-f="inc"${l.inc ? " checked" : ""}> Suma al
total (si no, queda como alternativa)</label>
      <button class="btn quiet" type="button" data-act="sync" hidden>↻ Actualizar costo</button>
      ${l.entryId ? `<button class="btn quiet" type="button" data-act="${l.own ? "edit" :
"variant"}">${l.own ? "Editar costos" : "Armar variante"}</button>
      <button class="btn quiet" type="button" data-act="sugg"${l.mode === "manual" ? "" : "
hidden"}>Usar sugerido</button>` : ""}
      <button class="btn quiet" type="button" data-act="dup" aria-label="Duplicar
ítem">⧉</button>
      <button class="btn quiet" type="button" data-act="del" aria-label="Eliminar
ítem">✕</button>
    </div>
  </div>`).join("");
  document.getElementById("qEmpty").style.display = q.lines.length ? "none" : "block";
}

function qDocHTML(t){
  if (!q.lines.length) return `<p class="hint">Agregá ítems para ver el presupuesto tal como lo va
a recibir tu cliente.</p>`;
  const on = q.lines.filter(l => l.inc), alt = q.lines.filter(l => !l.inc);
  const row = l => `<tr><td><b>${esc(l.name || "Sin descripción")}</b>${l.detail ?
`<br><small>${esc(l.detail)}</small>` : ""}</td><td class="r">${l.qty}</td><td
class="r">${qm(l.price)}</td><td class="r">${qm(l.qty * l.price)}</td></tr>`;
  const head = `<thead><tr><th>Detalle</th><th class="r">Cant.</th><th
class="r">Unitario</th><th class="r">Subtotal</th></tr></thead>`;
```

```
  return `<div class="qd-top"><div class="qd-brand"><img class="qd-logo" src="${LOGO}"
alt="${esc(qcfg.shop || "Logo")}"><div>${qcfg.shop ? `<h3>${esc(qcfg.shop)}</h3>` :
""}${qcfg.shopPhone ? `<small>${esc(qcfg.shopPhone)}</small>` : ""}</div></div>
      <div class="r"><b>Presupuesto${q.num ? " N° " + qNum(q.num) :
""}</b><br><small>${qFmt(q.date)}</small></div></div>
    ${q.client ? `<p><b>Cliente:</b> ${esc(q.client)}</p>` : ""}
    <div class="tw"><table>${head}<tbody>${on.map(row).join("")}</tbody></table></div>
    <div class="qd-tot">
      <div><span>Subtotal</span><span>${qm(t.sub)}</span></div>
      ${t.disc > 0 ? `<div><span>Descuento (${f1.format(q.disc)} %)</span><span>−
${qm(t.disc)}</span></div>` : ""}
      <div class="big"><span>TOTAL</span><span>${qm(t.total)}</span></div>
      ${q.dep > 0 && t.total > 0 ? `<div><span>Seña para comenzar (${f1.format(q.dep)}
%)</span><span>${qm(t.dep)}</span></div><div><span>Saldo a la
entrega</span><span>${qm(t.bal)}</span></div>` : ""}
    </div>
    ${alt.length ? `<p class="qd-h">Alternativas (no incluidas en el total)</p><div
class="tw"><table>${head}<tbody>${alt.map(row).join("")}</tbody></table></div>` : ""}
    <p class="qd-meta">${q.valid > 0 ? `Válido hasta el ${qFmt(qPlus(q.date, q.valid))}.` :
""}${q.delivery ? ` Plazo de entrega: ${esc(q.delivery)}.` : ""}</p>
    ${q.notes ? `<p class="qd-notes">${esc(q.notes)}</p>` : ""}`;
}
function qText(){
  const t = qCalc(), on = q.lines.filter(l => l.inc), alt = q.lines.filter(l => !l.inc);
  const ln = l => `• *${l.name || "Sin descripción"}*${l.detail ? "\n  " + l.detail : ""}\n  ${l.qty} ×
${qm(l.price)} = ${qm(l.qty * l.price)}`;
  const out = [`*Presupuesto${q.num ? " N° " + qNum(q.num) : ""}*${qcfg.shop ? " — " +
qcfg.shop : ""}`];
  if (q.client) out.push("Cliente: " + q.client);
  out.push("Fecha: " + qFmt(q.date) + (q.valid > 0 ? " · Válido hasta: " + qFmt(qPlus(q.date,
q.valid)) : ""), "", on.map(ln).join("\n"), "");
  if (t.disc > 0) out.push("Subtotal: " + qm(t.sub), "Descuento (" + f1.format(q.disc) + " %): − " +
qm(t.disc));
  out.push("*TOTAL: " + qm(t.total) + "*");
  if (q.dep > 0 && t.total > 0) out.push("Seña (" + f1.format(q.dep) + " %): " + qm(t.dep) + " ·
Saldo: " + qm(t.bal));
  if (alt.length) out.push("", "_Alternativas (no incluidas en el total):_", alt.map(ln).join("\n"));
  if (q.delivery) out.push("", "Plazo de entrega: " + q.delivery);
  if (q.notes) out.push("", q.notes);
  if (qcfg.shopPhone) out.push("", (qcfg.shop ? qcfg.shop + " · " : "") + qcfg.shopPhone);
  return out.join("\n");
}
/* Celulares argentinos: acepta 11 5555 1234, 011 5555 1234 o +54 9 11 5555 1234. Si no logra
```

```
armar un número válido, WhatsApp abre para elegir el chat. */
function qWaNum(raw){
  let d = String(raw || "").replace(/\D/g, "");
  if (d.startsWith("00")) d = d.slice(2);
  if (d.length === 11 && d[0] === "0") d = d.slice(1);
  if (d.length === 10) d = "549" + d;
  else if (d.length === 12 && d.startsWith("54") && d[2] !== "9") d = "549" + d.slice(2);
  return /^549\d{10}$/.test(d) ? d : "";
}

function qUpdate(save){
  document.querySelectorAll("#qLines .qline").forEach(el => {
    const l = q.lines.find(x => x.id === el.dataset.id); if (!l) return;
    el.querySelector(".qsub").textContent = qm(l.qty * l.price);
    const gain = l.price - l.cost, it = el.querySelector("[data-int]");
    it.className = "qint" + (l.cost > 0 && l.price < l.cost ? " bad" : "");
    it.textContent = l.cost <= 0 ? ""
      : l.price < l.cost ? "⚠ Por debajo del costo (" + qm(l.cost) + " c/u)"
      : "Costo " + qm(l.cost) + " c/u · ganancia " + qm(gain) + " (" + f1.format(l.price > 0 ? gain /
l.price * 100 : 0) + " % del precio)";
    el.querySelector('[data-act="sync"]').hidden = !qStale(l);
    const sb = el.querySelector('[data-act="sugg"]'); if (sb) sb.hidden = l.mode !== "manual";
  });
  const t = qCalc(), low = q.lines.filter(l => l.inc && l.cost > 0 && l.price < l.cost).length;
  document.getElementById("qDoc").innerHTML = qDocHTML(t);
  document.getElementById("qInternal").innerHTML =
    `<div class="qi"><span>Costo de materiales</span><b>${qm(t.cost)}</b></div>
     <div class="qi"><span>Ganancia estimada</span><b>${qm(t.profit)}${t.total > 0 ? " · " +
f1.format(t.profit / t.total * 100) + " %" : ""}</b></div>
     ${low ? `<p class="qint bad">⚠ ${low} ítem(s) por debajo del costo.</p>` : ""}
     <p class="hint">Tu mano de obra no está en el costo: cuenta como ganancia. Los ítems
libres (flete, instalación) tampoco tienen costo cargado.</p>`;
  if (save !== false) qSaveDraft();
}
function qRefresh(){ qRenderLines(); qUpdate(); }
function qGoMat(){ qSaveDraft(); qReturn = true; setTab("mat"); }

function qSaveQuote(){
  if (!q.lines.length){ flag("Agregá al menos un ítem"); return; }
  if (!q.num) q.num = ++qbook.seq;
  const entry = Object.assign(JSON.parse(JSON.stringify(q)), {id:qEditing || uid(),
total:qCalc().total});
  const i = qbook.entries.findIndex(x => x.id === entry.id);
```

```
  if (i >= 0) qbook.entries[i] = entry; else qbook.entries.unshift(entry);
  qEditing = entry.id; qSaveBook(); qSaveDraft(); qLabel(); renderQBook(); qUpdate(false);
}
function renderQBook(){
  const es = qbook.entries, by = st => es.filter(x => x.status === st);
  const tot = st => by(st).reduce((t, x) => t + x.total, 0);
  document.getElementById("qBookSummary").textContent = es.length
    ? `Enviados sin respuesta: ${by("enviado").length} (${qm(tot("enviado"))}) · Aceptados:
${by("aceptado").length} (${qm(tot("aceptado"))})` : "";
  document.getElementById("qBookEmpty").style.display = es.length ? "none" : "block";
  document.getElementById("qBook").innerHTML = es.map(x => `<div class="qrow${x.id ===
qEditing ? " cur" : ""}" data-id="${esc(x.id)}">
    <div class="qmain"><b>${x.num ? "N° " + qNum(x.num) + " · " : ""}${esc(x.client || "Sin
cliente")}</b><small>${qFmt(x.date)} · ${qm(x.total)}${x.id === qEditing ? " · editando" :
""}</small></div>
    <select data-act="status" aria-label="Estado">${Q_STATES.map(s => `<option
value="${s[0]}"${x.status === s[0] ? " selected" : ""}>${s[1]}</option>`).join("")}</select>
    <button class="btn quiet" type="button" data-act="open" title="Abrir"
aria-label="Abrir">✎</button>
    <button class="btn quiet" type="button" data-act="copy" title="Usar como base de un
presupuesto nuevo" aria-label="Duplicar">⿻</button>
    <button class="btn quiet" type="button" data-act="del" title="Eliminar"
aria-label="Eliminar">✕</button></div>`).join("");
}

function bindQuote(){
  const $ = id => document.getElementById(id);
  const map = {qClient:"client", qPhone:"phone", qValid:"valid", qDelivery:"delivery",
qNotes:"notes", qDisc:"disc", qDep:"dep"};
  Object.keys(map).forEach(id => $(id).addEventListener("input", e => {
    const k = map[id], v = e.target.value;
    q[k] = k === "valid" ? Math.round(qn(v, 0, 0, 365)) : (k === "disc" || k === "dep") ? qn(v, 0, 0,
100) : v;
    qUpdate();
  }));
  $("qShop").addEventListener("input", e => { qcfg.shop = e.target.value; qUpdate(); });
  $("qShopPhone").addEventListener("input", e => { qcfg.shopPhone = e.target.value;
qUpdate(); });
  $("qAddFrom").addEventListener("click", () => {
    const e = book.entries.find(x => x.id === $("qPick").value); if (!e) return;
    q.lines.push(qLineFrom(e)); qRefresh();
  });
  $("qAddFree").addEventListener("click", () => {
```

```
  q.lines.push(qCleanLine({name:"", price:0, mode:"manual"})); qRefresh();
  const i = document.querySelector("#qLines .qline:last-child input"); if (i) i.focus();
 });

 const box = $("qLines");
 box.addEventListener("input", e => {
  const el = e.target.closest(".qline"), f = e.target.dataset.f; if (!el || !f || f === "inc") return;
  const l = q.lines.find(x => x.id === el.dataset.id); if (!l) return;
  if (f === "name" || f === "detail") l[f] = e.target.value;
  else if (f === "qty") l.qty = Math.max(1, Math.round(qn(e.target.value, 1, 0)));
  else if (f === "price"){ l.price = qn(e.target.value, 0, 0); l.mode = "manual"; }
  qUpdate();
 });
 box.addEventListener("change", e => {
  if (e.target.dataset.f !== "inc") return;
  const el = e.target.closest(".qline"), l = q.lines.find(x => x.id === el.dataset.id); if (!l) return;
  l.inc = e.target.checked; el.classList.toggle("off", !l.inc); qUpdate();
 });
 box.addEventListener("click", e => {
  const b = e.target.closest("[data-act]"); if (!b) return;
  const el = b.closest(".qline"), i = q.lines.findIndex(x => x.id === el.dataset.id); if (i < 0) return;
  const l = q.lines[i], a = b.dataset.act;
  if (a === "del") q.lines.splice(i, 1);
  else if (a === "dup") q.lines.splice(i + 1, 0, Object.assign({}, l, {id:uid()}));
  else if (a === "sync") qSync(l);
  else if (a === "sugg"){ l.mode = "sugg"; l.price = qUp(l.sugg); }
  else if (a === "variant"){
   const si = book.entries.findIndex(x => x.id === l.entryId);
   if (si < 0){ flag("El mueble de origen ya no está guardado"); return; }
   duplicateMueble(l.entryId);
   const copy = book.entries[si + 1];
   copy.name = book.entries[si].name + " — " + (q.client || "variante");
   l.entryId = copy.id; l.own = true;
   saveBook(); loadMueble(copy.id); qGoMat(); return;
  }
  else if (a === "edit"){
   if (!book.entries.some(x => x.id === l.entryId)){ flag("Ese mueble ya no está guardado");
return; }
   loadMueble(l.entryId); qGoMat(); return;
  }
  qRefresh();
 });
```

```
  $("qBack").addEventListener("click", () => {
    q.lines.forEach(l => { if (l.entryId && l.entryId === matEditingId) qSync(l); });
    qReturn = false; setTab("quote");
  });
  $("qCopy").addEventListener("click", () => copyText(qText()).then(ok => flag(ok ? "Texto
copiado: pegalo en WhatsApp" : "No se pudo copiar")));
  $("qWa").addEventListener("click", () => window.open("https://wa.me/" + qWaNum(q.phone) +
"?text=" + encodeURIComponent(qText()), "_blank", "noopener"));
  $("qPrint").addEventListener("click", () => window.print());
  $("qSave").addEventListener("click", qSaveQuote);
  $("qNew").addEventListener("click", () => {
    if (q.lines.length && !qEditing && !confirm("¿Descartar este presupuesto sin guardar?"))
return;
    q = qDefaults(); qEditing = null; qFillForm(); qRefresh(); renderQBook(); qSaveDraft();
  });

  const bk = $("qBook");
  bk.addEventListener("change", e => {
    const row = e.target.closest("[data-id]"); if (!row || e.target.dataset.act !== "status") return;
    const en = qbook.entries.find(x => x.id === row.dataset.id); if (!en) return;
    en.status = e.target.value; if (qEditing === en.id) q.status = en.status;
    qSaveBook(); qSaveDraft(); renderQBook();
  });
  bk.addEventListener("click", e => {
    const b = e.target.closest("button[data-act]"), row = e.target.closest("[data-id]"); if (!b || !row)
return;
    const id = row.dataset.id, i = qbook.entries.findIndex(x => x.id === id); if (i < 0) return;
    const en = qbook.entries[i], a = b.dataset.act;
    if (a === "open"){ q = qClean(en); qEditing = id; qFillForm(); qRefresh(); renderQBook();
qSaveDraft(); window.scrollTo(0, 0); flag("Presupuesto abierto"); }
    else if (a === "copy"){
      q = qClean(en); q.num = 0; q.client = ""; q.phone = ""; q.date = qToday(); q.status =
"borrador"; qEditing = null;
      qFillForm(); qRefresh(); renderQBook(); qSaveDraft(); window.scrollTo(0, 0); flag("Copia
lista: cargá el cliente nuevo");
    }else if (a === "del"){
      if (!confirm("¿Eliminar este presupuesto guardado?")) return;
      qbook.entries.splice(i, 1); if (qEditing === id) qEditing = null;
      qSaveBook(); qSaveDraft(); qLabel(); renderQBook();
    }
  });
}
```

```
/*
================================================================================
   10. DISEÑO 3D — estado, catálogo, plantillas y geometría
   Piezas = cajas (tableros, perfiles huecos, chapa) y grupos de cajas (plegadas).
   Coordenadas en mm: X = largo, Y = altura (piso = 0), Z = fondo.

================================================================================ */
const D3KEY = "diseno3d_v1";

const D3_PROFILES = [
  { id:"sq20", name:"Caño 20×20", a:20, b:20, walls:[1.2,1.4,1.6,2] },
  { id:"sq30", name:"Caño 30×30", a:30, b:30, walls:[1.2,1.4,1.6,2] },
  { id:"sq40", name:"Caño 40×40", a:40, b:40, walls:[1.2,1.4,1.6,2] },
  { id:"sq50", name:"Caño 50×50", a:50, b:50, walls:[1.2,1.4,1.6,2] },
  { id:"re4020", name:"Caño 40×20", a:40, b:20, walls:[1.2,1.4,1.6,2] },
  { id:"re6040", name:"Caño 60×40", a:60, b:40, walls:[1.2,1.4,1.6,2] }
];
const d3Prof = id => D3_PROFILES.find(p => p.id === id) || D3_PROFILES[0];
const d3Kgm = (a, b, e) => 2 * e * (a + b - 2 * e) * 0.00785;   // kg/m de sección rectangular hueca
const d3KgChapa = (areaMm2, t) => areaMm2 * t * 7.85e-6;         // kg de chapa (acero)

function d3Defaults(){
  return {
    pieces: [], cotas: [],
    magnet: true, capture: 8, rotStep: 15, holgura: 1, kerfBarra: 3, solape: 100,
    cat: {},                                  // "sq40|1.6" -> {price, barLen}
    ple: { plegadora: 0, cizalla: 0 },
    cam: { az: -35, el: 22, scale: 0.18, tx: 400, ty: 200, tz: 250 },
    sel: null, mode: "mover", regla: null, angPts: [], snaps: [], marks: []
  };
}
let d3 = d3Defaults();
let d3SaveTimer = null;

function d3Save(){
  clearTimeout(d3SaveTimer);
  d3SaveTimer = setTimeout(() => {
    try{
      localStorage.setItem(D3KEY, JSON.stringify({
        pieces: d3.pieces, cotas: d3.cotas, cat: d3.cat, ple: d3.ple, cam: d3.cam,
```

```
      magnet: d3.magnet, capture: d3.capture, rotStep: d3.rotStep,
      holgura: d3.holgura, kerfBarra: d3.kerfBarra, solape: d3.solape
    }));
  }catch(e){ /* sin almacenamiento: se sigue en memoria */ }
 }, 400);
}
function d3Load(){
 try{
  const raw = localStorage.getItem(D3KEY);
  if (!raw) return;
  const s = JSON.parse(raw);
  const base = d3Defaults();
  d3 = Object.assign(base, s, {
   sel: null, mode: "mover", regla: null, angPts: [], snaps: [], marks: [],
   pieces: Array.isArray(s.pieces) ? s.pieces.filter(p => p && p.id) : [],
   cotas: Array.isArray(s.cotas) ? s.cotas.slice(0, 40) : [],
   cat: s.cat && typeof s.cat === "object" ? s.cat : {},
   ple: Object.assign(base.ple, s.ple || {}),
   cam: Object.assign(base.cam, s.cam || {})
  });
 }catch(e){ /* estado por defecto */ }
}

/* catálogo: precio y largo de barra por perfil + pared */
const d3CatKey = (pid, wall) => pid + "|" + wall;
function d3CatGet(pid, wall){
 const k = d3CatKey(pid, wall);
 if (!d3.cat[k]) d3.cat[k] = { price: 0, barLen: 6000 };
 return d3.cat[k];
}

/* --- dimensiones del AABB sin rotar (dx, dy, dz) --- */
function d3Dims(p){
 if (p.kind === "madera" || p.kind === "chapa"){
  return p.axis === "y" ? [p.t, p.l, p.w] : [p.l, p.t, p.w];
 }
 if (p.kind === "perfil"){
  const pr = d3Prof(p.profId);
  if (p.axis === "y") return [pr.a, p.len, pr.b];
  if (p.axis === "z") return [pr.b, pr.a, p.len];
  return [p.len, pr.a, pr.b];
 }
 let X = 0, Y = 0, Z = 0;
```

```
  for (const s of p.segs){ X = Math.max(X, s.x + s.dx); Y = Math.max(Y, s.y + s.dy); Z =
Math.max(Z, s.z + s.dz); }
  return [X, Y, Z];
}

/* punto local -> mundo: rota alrededor de Y sobre el centro de la huella */
function d3World(p, lx, ly, lz){
  const D = d3Dims(p);
  const r = (p.rot || 0) * Math.PI / 180, co = Math.cos(r), si = Math.sin(r);
  const ax = lx - D[0] / 2, az = lz - D[2] / 2;
  return {
    x: p.x + D[0] / 2 + ax * co - az * si,
    y: p.y + ly,
    z: p.z + D[2] / 2 + ax * si + az * co
  };
}

function d3AABB(p){
  const D = d3Dims(p);
  let x1 = Infinity, y1 = Infinity, z1 = Infinity, x2 = -Infinity, y2 = -Infinity, z2 = -Infinity;
  for (let i = 0; i < 8; i++){
    const w = d3World(p, (i & 1) ? D[0] : 0, (i & 2) ? D[1] : 0, (i & 4) ? D[2] : 0);
    if (w.x < x1) x1 = w.x; if (w.x > x2) x2 = w.x;
    if (w.y < y1) y1 = w.y; if (w.y > y2) y2 = w.y;
    if (w.z < z1) z1 = w.z; if (w.z > z2) z2 = w.z;
  }
  return { x1, y1, z1, x2, y2, z2, cx:(x1+x2)/2, cy:(y1+y2)/2, cz:(z1+z2)/2, dx:x2-x1, dy:y2-y1,
dz:z2-z1 };
}

/* cajas locales para el render */
function d3Boxes(p){
  if (p.kind === "plegada") return p.segs;
  const D = d3Dims(p);
  return [{ x:0, y:0, z:0, dx:D[0], dy:D[1], dz:D[2] }];
}

/* --- chapa plegada: desarrollo y dobleces (a 90°, convención de taller:
   los tramos se miden por AFUERA con calibre)
   BA = (π/2)·(r + K·t)   BD = 2·(r + t) − BA   desarrollo = Σ tramos − n·BD --- */
function d3Bend(t, r, K){
  const BA = (Math.PI / 2) * (r + K * t);
  return { BA, BD: 2 * (r + t) - BA };
```

```
}

const D3_TMPL = {
  L:       { label:"L", segs:["Base (mm)", "Alto total (mm)"], bends:1 },
  U:       { label:"U", segs:["Alto izq. (mm)", "Ancho total (mm)", "Alto der. (mm)"], bends:2 },
  C:       { label:"C", segs:["Ala inferior (mm)", "Alma alta (mm)", "Ala superior (mm)"], bends:2 },
  Z:       { label:"Z", segs:["Ancho de alas (mm)", "Alto total (mm)"], bends:2 },
  bandeja: { label:"Bandeja", segs:["Ancho total (mm)", "Alto de paredes (mm)"], bends:4 }
};

/* arma la plegada: cajas locales + desarrollo + avisos */
function d3Plegada(tmpl, dims, t, L, r, K){
  const { BD } = d3Bend(t, r, K);
  let segs = [], devA = 0, bends = D3_TMPL[tmpl] ? D3_TMPL[tmpl].bends : 0, warns = [];
  if (tmpl === "L"){
    const [A, H] = dims;
    segs = [{x:0,y:0,z:0,dx:A,dy:t,dz:L}, {x:A-t,y:0,z:0,dx:t,dy:H,dz:L}];
    devA = Math.max(0, A + H - BD);
    if (Math.min(A, H) < 4*t) warns.push("Pestaña corta: se recomienda 4 a 6 veces el espesor");
  } else if (tmpl === "U"){
    const [H1, A, H2] = dims;
    segs = [{x:0,y:0,z:0,dx:t,dy:H1,dz:L}, {x:0,y:0,z:0,dx:A,dy:t,dz:L},
{x:A-t,y:0,z:0,dx:t,dy:H2,dz:L}];
    devA = Math.max(0, H1 + A + H2 - 2*BD);
    if (Math.min(H1, H2, A) < 4*t) warns.push("Pestaña corta: se recomienda 4 a 6 veces el
espesor");
  } else if (tmpl === "C"){
    const [c1, H, c2] = dims;
    segs = [{x:0,y:0,z:0,dx:c1,dy:t,dz:L}, {x:0,y:0,z:0,dx:t,dy:H,dz:L},
{x:0,y:H-t,z:0,dx:c2,dy:t,dz:L}];
    devA = Math.max(0, c1 + H + c2 - 2*BD);
    if (Math.min(c1, c2, H) < 4*t) warns.push("Pestaña corta: se recomienda 4 a 6 veces el
espesor");
  } else if (tmpl === "Z"){
    const [A, H] = dims;
    segs = [{x:t,y:0,z:0,dx:A,dy:t,dz:L}, {x:0,y:0,z:0,dx:t,dy:H,dz:L}, {x:t,y:H-t,z:0,dx:A,dy:t,dz:L}];
    devA = Math.max(0, 2*A + H - 2*BD);
    if (Math.min(A, H) < 4*t) warns.push("Pestaña corta: se recomienda 4 a 6 veces el espesor");
  } else {                              /* bandeja */
    const [W, h] = dims;
    segs = [
      {x:0,y:0,z:0,dx:W,dy:t,dz:L},          /* fondo */
      {x:0,y:t,z:0,dx:t,dy:h,dz:L},          /* pared izq. */
```

```
      {x:W-t,y:t,z:0,dx:t,dy:h,dz:L},        /* pared der. */
      {x:0,y:t,z:0,dx:W,dy:h,dz:t},          /* frente */
      {x:0,y:t,z:L-t,dx:W,dy:h,dz:t}         /* fondo trasero */
    ];
    devA = Math.max(0, W + 2*(h - BD));
    warns.push("Esquinas aproximadas: cortá alivios de ~" + mmv(h) + "×" + mmv(h) + " mm en las cuatro esquinas del desarrollo");
  }
  const devB = tmpl === "bandeja" ? Math.max(0, L + 2*(dims[1] - BD)) : L;
  if (d3.ple.plegadora > 0 && L > d3.ple.plegadora)
    warns.push("El plegado de " + mmv(L) + " mm supera tu plegadora (" + mmv(d3.ple.plegadora) + " mm)");
  if (d3.ple.cizalla > 0 && Math.max(devA, devB) > d3.ple.cizalla)
    warns.push("El desarrollo de " + mmv(Math.max(devA, devB)) + " mm supera tu cizalla (" + mmv(d3.ple.cizalla) + " mm)");
  return { segs, devA, devB, bends, weight: d3KgChapa(devA * devB, t), warns };
}

/* peso de una pieza (madera sin densidad confiable: 0) */
function d3PieceWeight(p){
  if (p.kind === "perfil"){
    const pr = d3Prof(p.profId);
    return d3Kgm(pr.a, pr.b, p.wall) * (p.len || 0) / 1000;
  }
  if (p.kind === "chapa") return d3KgChapa((p.l || 0) * (p.w || 0), p.t || 0);
  if (p.kind === "plegada") return d3KgChapa((p.devA || 0) * (p.devB || 0), p.t || 0);
  return 0;
}

function d3DefaultName(p){
  if (p.kind === "madera") return "Tablero " + mmv(p.l) + "×" + mmv(p.w) + "×" + mmv(p.t);
  if (p.kind === "perfil") return d3Prof(p.profId).name + " ×" + p.wall + " — " + mmv(p.len) + " mm";
  if (p.kind === "chapa") return "Chapa " + p.t + " mm — " + mmv(p.l) + "×" + mmv(p.w);
  return (D3_TMPL[p.tmpl] ? D3_TMPL[p.tmpl].label : p.tmpl) + " " + p.t + " mm — " + mmv(p.L) + " mm";
}

/* pieza nueva, apoyada en el piso al centro de la vista */
function d3NewPiece(kind, o){
  const p = Object.assign({ id: uid(), kind, name: "", x: 0, y: 0, z: 0, rot: 0, axis: "x", _sent: false }, o);
  const D = d3Dims(p);
```

```
  p.x = Math.round(d3.cam.tx - D[0] / 2);
  p.z = Math.round(d3.cam.tz - D[2] / 2);
  p.y = 0;
  if (!p.name) p.name = d3DefaultName(p);
  return p;
}

/*
================================================================================
   10.b DISEÑO 3D — renderizador propio (canvas 2D + algoritmo del pintor)
   Sin CDN ni librerías: cajas sombreadas, caños huecos, piso con grilla.

================================================================================ */
const d3cv = document.getElementById("d3Canvas");
const d3ctx = d3cv.getContext("2d");

const D3_COLORS = {
  madera:  [216, 201, 168],
  perfil:  [176, 188, 193],
  chapa:   [203, 210, 213],
  plegada: [203, 210, 213]
};
const D3_LIGHT = (() => { const s = Math.hypot(0.42, 0.85, 0.32); return { x: 0.42/s, y: 0.85/s, z: 0.32/s }; })();

function d3CamVecs(){
  const az = d3.cam.az * Math.PI / 180, el = d3.cam.el * Math.PI / 180;
  const f = { x: Math.cos(el) * Math.cos(az), y: -Math.sin(el), z: Math.cos(el) * Math.sin(az) };
  const rl = Math.hypot(f.x, f.z) || 1;
  const r = { x: -f.z / rl, y: 0, z: f.x / rl };
  const u = { x: r.y*f.z - r.z*f.y, y: r.z*f.x - r.x*f.z, z: r.x*f.y - r.y*f.x };   // u = r × f
  return { f, r, u };
}

function d3Resize(){
  const dpr = Math.min(2, window.devicePixelRatio || 1);
  const w = d3cv.clientWidth, h = d3cv.clientHeight;
  if (!w || !h){
    // Fallback when tab just became visible and layout hasn't run yet
    d3cv.width = 800 * dpr;
    d3cv.height = 500 * dpr;
```

```
    d3ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
    return true;
  }
  const pw = Math.round(w * dpr), ph = Math.round(h * dpr);
  if (d3cv.width !== pw || d3cv.height !== ph){ d3cv.width = pw; d3cv.height = ph; }
  d3ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
  return true;
}

function d3GridStep(S){
  const steps = [10, 25, 50, 100, 250, 500, 1000, 2500, 5000];
  for (const s of steps){ if (s * S >= 48) return s; }
  return 10000;
}

function d3Draw(){
  if (!d3Resize()) return;
  const W = d3cv.clientWidth, H = d3cv.clientHeight;
  const T = { x: d3.cam.tx, y: d3.cam.ty, z: d3.cam.tz };
  const { f, r, u } = d3CamVecs();
  const S = d3.cam.scale;
  const P = (wx, wy, wz) => {
    const dx = wx - T.x, dy = wy - T.y, dz = wz - T.z;
    return {
      x: W/2 + (dx*r.x + dy*r.y + dz*r.z) * S,
      y: H/2 - (dx*u.x + dy*u.y + dz*u.z) * S,
      z: dx*f.x + dy*f.y + dz*f.z
    };
  };
  d3ctx.clearRect(0, 0, W, H);

  /* piso con grilla adaptativa */
  let step = d3GridStep(S);
  let halfX = (W / S) * 0.75 + 100, halfZ = (H / S) * 0.9 + 100;
  while ((2*halfX/step + 2*halfZ/step) > 600) step *= 2;
  d3ctx.lineWidth = 1;
  d3ctx.strokeStyle = "rgba(20,62,69,.10)";
  d3ctx.beginPath();
  for (let x = Math.ceil((T.x - halfX)/step)*step; x <= T.x + halfX; x += step){
    const a = P(x, 0, T.z - halfZ), b = P(x, 0, T.z + halfZ);
    d3ctx.moveTo(a.x, a.y); d3ctx.lineTo(b.x, b.y);
  }
  for (let z = Math.ceil((T.z - halfZ)/step)*step; z <= T.z + halfZ; z += step){
```

```
    const a = P(T.x - halfX, 0, z), b = P(T.x + halfX, 0, z);
    d3ctx.moveTo(a.x, a.y); d3ctx.lineTo(b.x, b.y);
  }
  d3ctx.stroke();
  /* ejes: X (largo) y Z (fondo) */
  d3ctx.strokeStyle = "rgba(20,62,69,.28)";
  d3ctx.beginPath();
  const ax = P(0, 0, 0), ax2 = P(1200, 0, 0), az = P(0, 0, 1200);
  d3ctx.moveTo(ax.x, ax.y); d3ctx.lineTo(ax2.x, ax2.y);
  d3ctx.moveTo(ax.x, ax.y); d3ctx.lineTo(az.x, az.y);
  d3ctx.stroke();

  /* caras de las piezas, ordenadas del fondo hacia la cámara */
  const faces = [];
  for (const p of d3.pieces){
    const base = D3_COLORS[p.kind] || D3_COLORS.madera;
    const boxes = d3Boxes(p);
    for (const b of boxes){
      const C = [];
      for (let i = 0; i < 8; i++) C.push(d3World(p, b.x + ((i&1)?b.dx:0), b.y + ((i&2)?b.dy:0), b.z +
((i&4)?b.dz:0)));
      const center = d3World(p, b.x + b.dx/2, b.y + b.dy/2, b.z + b.dz/2);
      const FACES = [
        [[0,4,6,2], [-1,0,0]], [[1,7,5,3], [1,0,0]],
        [[0,1,5,4], [0,-1,0]], [[2,6,7,3], [0,1,0]],
        [[0,2,3,1], [0,0,-1]], [[4,5,7,6], [0,0,1]]
      ];
      for (const [idx, nAxis] of FACES){
        const q = idx.map(i => C[i]);
        const nrm = {
          x: (q[1].y-q[0].y)*(q[2].z-q[0].z) - (q[1].z-q[0].z)*(q[2].y-q[0].y),
          y: (q[1].z-q[0].z)*(q[2].x-q[0].x) - (q[1].x-q[0].x)*(q[2].z-q[0].z),
          z: (q[1].x-q[0].x)*(q[2].y-q[0].y) - (q[1].y-q[0].y)*(q[2].x-q[0].x)
        };
        const nl = Math.hypot(nrm.x, nrm.y, nrm.z) || 1;
        let nx = nrm.x/nl, ny = nrm.y/nl, nz = nrm.z/nl;
        const toC = { x: center.x - (q[0].x+q[2].x)/2, y: center.y - (q[0].y+q[2].y)/2, z: center.z -
(q[0].z+q[2].z)/2 };
        if (nx*toC.x + ny*toC.y + nz*toC.z > 0){ nx = -nx; ny = -ny; nz = -nz; }
        const facing = nx*f.x + ny*f.y + nz*f.z;
        // dibujar todas las caras (sin backface culling) para que se vean sólidas
        let k = 0.55 + 0.45 * Math.max(0, nx*D3_LIGHT.x + ny*D3_LIGHT.y + nz*D3_LIGHT.z);
        if (p.id === d3.sel) k = Math.min(1, k + 0.07);
```

```
      // bias de profundidad para painter's algorithm: caras frontales se dibujan sobre las traseras
      // para caras visibles usamos maxZ (punto más cercano a la cámara) para evitar que caras delgadas queden detrás
      const thickness = (p.kind === "chapa" || p.kind === "plegada") ? (p.t || 1.2) : (p.wall || 1.6);
      const zs = [P(q[0].x, q[0].y, q[0].z).z, P(q[1].x, q[1].y, q[1].z).z, P(q[2].x, q[2].y, q[2].z).z, P(q[3].x, q[3].y, q[3].z).z];
      const maxZ = Math.max.apply(null, zs);
      // sesgo fuerte en unidades de cámara (mm): 3x el espesor visible hacia la cámara + 2mm constante
      const depthBias = -facing * 3.0 * thickness + 2.0;
      // clave secundaria: distancia del centro de la pieza a la cámara (más cerca = se dibuja encima)
      const pieceCenterZ = P(d3World(p, d3Dims(p)[0]/2, d3Dims(p)[1]/2, d3Dims(p)[2]/2).x, d3World(p, d3Dims(p)[0]/2, d3Dims(p)[1]/2, d3Dims(p)[2]/2).y, d3World(p, d3Dims(p)[0]/2, d3Dims(p)[1]/2, d3Dims(p)[2]/2).z).z;
      faces.push({
        z: maxZ - depthBias,
        pieceZ: pieceCenterZ,  // clave primaria: más cerca = Z mayor = se dibuja después (arriba)
        z2: maxZ - depthBias,  // clave secundaria: cara
        pts: q.map(w => P(w.x, w.y, w.z)),
        fill: "rgb(" + Math.round(base[0]*k) + "," + Math.round(base[1]*k) + "," + Math.round(base[2]*k) + ")",
        hole: (p.kind === "perfil" && nAxis[0] !== 0 ? "x" : p.kind === "perfil" && nAxis[1] !== 0 ? "y" : p.kind === "perfil" && nAxis[2] !== 0 ? "z" : null),
        holeAt: (nAxis[0] === -1 || nAxis[1] === -1 || nAxis[2] === -1) ? 0 : 1,
        piece: p, box: b
      });
    }
  }
  }
 faces.sort((a, b) => b.z - a.z || b.pieceZ - a.pieceZ);

 for (const fc of faces){
  d3ctx.beginPath();
  d3ctx.moveTo(fc.pts[0].x, fc.pts[0].y);
  for (let i = 1; i < 4; i++) d3ctx.lineTo(fc.pts[i].x, fc.pts[i].y);
  d3ctx.closePath();
  d3ctx.fillStyle = fc.fill;
  d3ctx.fill();
  d3ctx.strokeStyle = "rgba(20,62,69,.22)";
  d3ctx.stroke();
```

```
    /* tapa hueca del caño: rectángulo interior oscuro en cada extremo */
    if (fc.hole){
      const b = fc.box, p = fc.piece, e = p.wall || 1;
      const D = d3Dims(p);
      let inner;
      const at = fc.holeAt;                          // 0 = cara en el mínimo del eje
      if (fc.hole === "x"){
        const lx = at ? D[0] : 0;
        inner = [[lx, e, e], [lx, D[1]-e, e], [lx, D[1]-e, D[2]-e], [lx, e, D[2]-e]];
      } else if (fc.hole === "y"){
        const ly = at ? D[1] : 0;
        inner = [[e, ly, e], [D[0]-e, ly, e], [D[0]-e, ly, D[2]-e], [e, ly, D[2]-e]];
      } else {
        const lz = at ? D[2] : 0;
        inner = [[e, e, lz], [D[0]-e, e, lz], [D[0]-e, D[1]-e, lz], [e, D[1]-e, lz]];
      }
      const iq = inner.map(v => { const w = d3World(p, v[0], v[1], v[2]); return P(w.x, w.y, w.z); });
      d3ctx.beginPath();
      d3ctx.moveTo(iq[0].x, iq[0].y);
      for (let i = 1; i < 4; i++) d3ctx.lineTo(iq[i].x, iq[i].y);
      d3ctx.closePath();
      d3ctx.fillStyle = "#25444c";
      d3ctx.fill();
    }
  }

  /* contorno de la pieza seleccionada */
  const sel = d3.pieces.find(p => p.id === d3.sel);
  if (sel){
    d3ctx.strokeStyle = "#143e45";
    d3ctx.lineWidth = 1.5;
    for (const b of d3Boxes(sel).slice(0, 6)){
      const C = [];
      for (let i = 0; i < 8; i++) C.push(d3World(sel, b.x + ((i&1)?b.dx:0), b.y + ((i&2)?b.dy:0), b.z +
((i&4)?b.dz:0)));
      const EDGES = [[0,1],[1,3],[3,2],[2,0],[4,5],[5,7],[7,6],[6,4],[0,4],[1,5],[2,6],[3,7]];
      d3ctx.beginPath();
      for (const [i, j] of EDGES){
        const a = P(C[i].x, C[i].y, C[i].z), c = P(C[j].x, C[j].y, C[j].z);
        d3ctx.moveTo(a.x, a.y); d3ctx.lineTo(c.x, c.y);
      }
      d3ctx.stroke();
```

```
    }
    const bb = d3AABB(sel);
    const lp = P(bb.cx, bb.y2, bb.cz);
    d3ctx.font = "600 12.5px 'Archivo Narrow',sans-serif";
    d3ctx.fillStyle = "#143e45";
    d3ctx.textAlign = "center";
    d3ctx.fillText(sel.name, lp.x, lp.y - 6);
    d3ctx.textAlign = "left";
  }

  /* guías del imán */
  if (d3.snaps.length){
    d3ctx.setLineDash([5, 4]);
    d3ctx.strokeStyle = "rgba(20,62,69,.8)";
    d3ctx.lineWidth = 1.2;
    d3ctx.font = "600 12px 'Archivo Narrow',sans-serif";
    for (const s of d3.snaps){
      let a, b, lx, ly;
      if (s.axis === "x"){ a = P(s.val, T.y, T.z - 5000); b = P(s.val, T.y, T.z + 5000); lx = a.x + 4; ly =
a.y - 4; }
      else if (s.axis === "y"){ a = P(T.x - 5000, s.val, T.z); b = P(T.x + 5000, s.val, T.z); lx = 6; ly =
a.y - 4; }
      else { a = P(T.x - 5000, T.y, s.val); b = P(T.x + 5000, T.y, s.val); lx = a.x + 4; ly = a.y - 4; }
      d3ctx.beginPath(); d3ctx.moveTo(a.x, a.y); d3ctx.lineTo(b.x, b.y); d3ctx.stroke();
      const txt = s.label;
      const tw = d3ctx.measureText(txt).width;
      d3ctx.fillStyle = "rgba(208,224,211,.92)";
      d3ctx.fillRect(lx - 2, ly - 12, tw + 6, 15);
      d3ctx.fillStyle = "#143e45";
      d3ctx.fillText(txt, lx + 1, ly);
    }
    d3ctx.setLineDash([]);
  }

  /* mediciones: marcas en curso + cotas fijas */
  const mark = (w, strong) => {
    const s = P(w.x, w.y, w.z);
    d3ctx.beginPath();
    d3ctx.arc(s.x, s.y, strong ? 5 : 4, 0, Math.PI*2);
    d3ctx.strokeStyle = "#a52816";
    d3ctx.lineWidth = 1.5;
    d3ctx.stroke();
  };
```

```
  const line = (a, b, label) => {
    const s1 = P(a.x, a.y, a.z), s2 = P(b.x, b.y, b.z);
    d3ctx.setLineDash([6, 4]);
    d3ctx.strokeStyle = "#a52816";
    d3ctx.lineWidth = 1.3;
    d3ctx.beginPath(); d3ctx.moveTo(s1.x, s1.y); d3ctx.lineTo(s2.x, s2.y); d3ctx.stroke();
    d3ctx.setLineDash([]);
    if (label){
      d3ctx.font = "600 12.5px 'Archivo Narrow',sans-serif";
      const tw = d3ctx.measureText(label).width;
      const mx = (s1.x + s2.x)/2, my = (s1.y + s2.y)/2 - 8;
      d3ctx.fillStyle = "rgba(251,230,226,.95)";
      d3ctx.fillRect(mx - tw/2 - 4, my - 12, tw + 8, 16);
      d3ctx.fillStyle = "#a52816";
      d3ctx.textAlign = "center";
      d3ctx.fillText(label, mx, my);
      d3ctx.textAlign = "left";
    }
  };
  for (const m of d3.marks) mark(m, true);
  if (d3.regla) mark(d3.regla, true);
  if (d3.regla && d3.marks.length) {}   // la cota en curso se dibuja abajo con el último punto
  for (const c of d3.cotas){
    if (c.type === "ang"){
      line(c.p2, c.p1, null); line(c.p2, c.p3, null);
      const v = P(c.p2.x, c.p2.y, c.p2.z);
      d3ctx.font = "700 13px 'Archivo Narrow',sans-serif";
      d3ctx.fillStyle = "#a52816";
      d3ctx.textAlign = "center";
      d3ctx.fillText(c.val.toFixed(1) + "°", v.x, v.y - 8);
      d3ctx.textAlign = "left";
    } else {
      line(c.a, c.b, c.txt);
    }
  }
}

/* rayo de pantalla al mundo (cámara ortográfica) */
function d3Ray(mx, my){
  const W = d3cv.clientWidth, H = d3cv.clientHeight;
  const { f, r, u } = d3CamVecs();
  const T = { x: d3.cam.tx, y: d3.cam.ty, z: d3.cam.tz };
  const cx = (mx - W/2) / d3.cam.scale, cy = (H/2 - my) / d3.cam.scale;
```

```
  return {
    o: { x: T.x + r.x*cx + u.x*cy - f.x*1e5, y: T.y + r.y*cx + u.y*cy - f.y*1e5, z: T.z + r.z*cx + u.z*cy -
f.z*1e5 },
    d: f
  };
}
function d3RayAABB(ray, bb){
  let tmin = 0, tmax = Infinity;   /* sin tope: el origen del rayo está lejos y la escena puede quedar
detrás del target */
  for (const ax of ["x", "y", "z"]){
    const inv = 1 / (ray.d[ax] || 1e-12);
    let t1 = (bb[ax + "1"] - ray.o[ax]) * inv, t2 = (bb[ax + "2"] - ray.o[ax]) * inv;
    if (t1 > t2){ const t = t1; t1 = t2; t2 = t; }
    tmin = Math.max(tmin, t1); tmax = Math.min(tmax, t2);
    if (tmin > tmax) return null;
  }
  return tmin;
}
function d3RayPlane(ray, px, py, pz, nx, ny, nz){
  const den = ray.d.x*nx + ray.d.y*ny + ray.d.z*nz;
  if (Math.abs(den) < 1e-9) return null;
  const t = ((px - ray.o.x)*nx + (py - ray.o.y)*ny + (pz - ray.o.z)*nz) / den;
  return t > 0 ? t : null;
}
function d3Pick(mx, my){
  const ray = d3Ray(mx, my);
  let best = null, bt = Infinity;
  for (const p of d3.pieces){
    const t = d3RayAABB(ray, d3AABB(p));
    if (t !== null && t < bt){ bt = t; best = p; }
  }
  return best;
}

/* puntos de captura: esquinas, centros de cara y centro de cada caja */
function d3SnapPoints(){
  const pts = [];
  for (const p of d3.pieces){
    for (const b of d3Boxes(p).slice(0, 6)){
      const mid = (v0, v1) => d3World(p, (v0[0]+v1[0])/2, (v0[1]+v1[1])/2, (v0[2]+v1[2])/2);
      const loc = [];
      for (let i = 0; i < 8; i++) loc.push([b.x + ((i&1)?b.dx:0), b.y + ((i&2)?b.dy:0), b.z +
((i&4)?b.dz:0)]);
```

```
      for (const v of loc) pts.push(d3World(p, v[0], v[1], v[2]));
      pts.push(mid(loc[0], loc[7]));
      pts.push(mid(loc[1], loc[6])); pts.push(mid(loc[2], loc[5])); pts.push(mid(loc[4], loc[3]));
      pts.push(mid(loc[0], loc[1])); pts.push(mid(loc[0], loc[2])); pts.push(mid(loc[0], loc[4]));
    }
  }
  return pts;
}
function d3SnapAt(mx, my){
  const W = d3cv.clientWidth, H = d3cv.clientHeight;
  const { f, r, u } = d3CamVecs();
  const T = { x: d3.cam.tx, y: d3.cam.ty, z: d3.cam.tz };
  const S = d3.cam.scale;
  let best = null, bd = 24;
  for (const w of d3SnapPoints()){
    const dx = w.x - T.x, dy = w.y - T.y, dz = w.z - T.z;
    const sx = W/2 + (dx*r.x + dy*r.y + dz*r.z) * S;
    const sy = H/2 - (dx*u.x + dy*u.y + dz*u.z) * S;
    const d = Math.hypot(sx - mx, sy - my);
    if (d < bd){ bd = d; best = w; }
  }
  return best;
}

/* encuadre automático */
function d3Fit(){
  if (!d3.pieces.length){
    d3.cam.tx = 400; d3.cam.ty = 200; d3.cam.tz = 250;
    d3.cam.scale = 0.18;
    return;
  }
  let x1 = Infinity, y1 = Infinity, z1 = Infinity, x2 = -Infinity, y2 = -Infinity, z2 = -Infinity;
  for (const p of d3.pieces){
    const a = d3AABB(p);
    x1 = Math.min(x1, a.x1); x2 = Math.max(x2, a.x2);
    y1 = Math.min(y1, a.y1); y2 = Math.max(y2, a.y2);
    z1 = Math.min(z1, a.z1); z2 = Math.max(z2, a.z2);
  }
  d3.cam.tx = (x1 + x2) / 2;
  d3.cam.ty = Math.max(60, (y1 + y2) / 2);
  d3.cam.tz = (z1 + z2) / 2;
  /* escala que respete el ancho Y el alto proyectados (media extensión) */
  const { r, u } = d3CamVecs();
```

```
  let hw = 0, hh = 0;
  for (let i = 0; i < 8; i++){
    const wx = (i & 1) ? x2 : x1, wy = (i & 2) ? y2 : y1, wz = (i & 4) ? z2 : z1;
    const px = (wx - d3.cam.tx) * r.x + (wy - d3.cam.ty) * r.y + (wz - d3.cam.tz) * r.z;
    const py = (wx - d3.cam.tx) * u.x + (wy - d3.cam.ty) * u.y + (wz - d3.cam.tz) * u.z;
    hw = Math.max(hw, Math.abs(px)); hh = Math.max(hh, Math.abs(py));
  }
  const W = d3cv.clientWidth || 600, H = d3cv.clientHeight || 400;
  d3.cam.scale = Math.max(0.02, Math.min(2,
    Math.min((W * 0.42) / Math.max(hw, 60), (H * 0.42) / Math.max(hh, 60))));
}

/*
================================================================================
=
   10.c DISEÑO 3D — interacción: mover con imán, cámara, regla y transportador
   El imán solo ajusta la posición mientras arrastrás: no crea ataduras.

================================================================================
= */
let d3Drag = null;
let d3Ptrs = {};
let d3Pinch = null;

/* limpia punteros que quedaron colgados (sin su pointerup) */
function d3PrunePtrs(){
  const now = Date.now();
  for (const id of Object.keys(d3Ptrs)){
    if (now - d3Ptrs[id].t > 3000) delete d3Ptrs[id];
  }
}

function d3Local(e){
  const r = d3cv.getBoundingClientRect();
  return { x: e.clientX - r.left, y: e.clientY - r.top };
}

function d3SetMode(m){
  d3.mode = m;
  d3.regla = null; d3.angPts = []; d3.marks = []; d3.snaps = [];
  const map = { mover:"Mover", cam:"Cam", regla:"Regla", angulo:"Ang" };
  for (const k of Object.keys(map))
    document.getElementById("d3m" + map[k]).setAttribute("aria-pressed", String(m === k));
```

```
  d3cv.className = m === "cam" ? "" : "mode-" + m;
  d3Status();
  d3Draw();
}

/* imán: alinea ras / centros / apoya en piso. Devuelve posición y guías. */
function d3Magnet(p, pos){
  const out = { x: pos.x, y: pos.y, z: pos.z, snaps: [] };
  if (!d3.magnet) return out;
  const cap = Math.max(1, +d3.capture || 8);
  const D = d3Dims(p);
  for (const axis of ["x", "y", "z"]){
    const size = axis === "x" ? D[0] : axis === "y" ? D[1] : D[2];
    const at = out[axis];
    const mine = [at, at + size, at + size / 2];
    const kinds = ["ras", "ras", "centro"];
    let bestDelta = cap + 1, best = null;
    for (const q of d3.pieces){
      if (q.id === p.id) continue;
      const a = d3AABB(q);
      const theirs = axis === "x" ? [a.x1, a.x2, a.cx] : axis === "y" ? [a.y1, a.y2, a.cy] : [a.z1, a.z2,
a.cz];
      for (let i = 0; i < 3; i++){
        for (let j = 0; j < 3; j++){
          const delta = theirs[j] - mine[i];
          if (Math.abs(delta) < Math.abs(bestDelta)){
            bestDelta = delta;
            best = { val: theirs[j], label: q.name + " · " + (kinds[i] === "centro" ? "centro" : "ras") };
          }
        }
      }
    }
    for (const ev of [0]){                          /* piso (Y=0) y ejes X=0 / Z=0 */
      for (let i = 0; i < 3; i++){
        const delta = ev - mine[i];
        if (Math.abs(delta) < Math.abs(bestDelta)){
          bestDelta = delta;
          best = { val: ev, label: (axis === "y" ? "piso" : "eje " + axis.toUpperCase()) + " · " + (kinds[i]
=== "centro" ? "centro" : "ras") };
        }
      }
    }
    if (best && Math.abs(bestDelta) <= cap){
```

```
      out[axis] += bestDelta;
      out.snaps.push({ axis, val: best.val, label: best.label });
    }
  }
  return out;
}

function d3Down(e){
  if (e.button !== undefined && e.button !== 0 && e.button !== 2) return;
  try{ d3cv.setPointerCapture(e.pointerId); }catch(err){}
  d3PrunePtrs();
  const m = d3Local(e);
  d3Ptrs[e.pointerId] = { x: m.x, y: m.y, t: Date.now() };
  if (Object.keys(d3Ptrs).length === 2){
    const pts = Object.values(d3Ptrs);
    d3Pinch = { d0: Math.hypot(pts[0].x - pts[1].x, pts[0].y - pts[1].y) || 1, s0: d3.cam.scale };
    d3Drag = null;
    return;
  }
  if (e.button === 2){ d3Drag = { tipo: "pan", m0: m, t0: { tx: d3.cam.tx, ty: d3.cam.ty, tz:
d3.cam.tz } }; return; }
  if (d3.mode === "regla" || d3.mode === "angulo"){
    d3Drag = { tipo: "cam", m0: m, az0: d3.cam.az, el0: d3.cam.el, clickMode: d3.mode };
    return;
  }
  const hit = d3.mode === "mover" ? d3Pick(m.x, m.y) : null;
  if (hit){
    d3.sel = hit.id;
    d3SelRender(); d3TelRender(); d3Status(); d3Draw();
    const ray = d3Ray(m.x, m.y);
    let t = d3RayPlane(ray, 0, hit.y, 0, 0, 1, 0);
    if (t === null) t = d3RayAABB(ray, d3AABB(hit));
    const pt = t !== null
      ? { x: ray.o.x + ray.d.x*t, y: ray.o.y + ray.d.y*t, z: ray.o.z + ray.d.z*t }
      : { x: hit.x, y: hit.y, z: hit.z };
    d3Drag = { tipo: "pieza", p: hit, off: { x: pt.x - hit.x, z: pt.z - hit.z }, m0: m, baseY: hit.y, moved:
false, vertical: false };
  } else {
    d3Drag = { tipo: "cam", m0: m, az0: d3.cam.az, el0: d3.cam.el, clickMode: d3.mode };
  }
}

function d3Move(e){
```

```
if (!d3Ptrs[e.pointerId]) return;
const m = d3Local(e);
d3Ptrs[e.pointerId] = { x: m.x, y: m.y, t: Date.now() };
if (d3Pinch){
  const pts = Object.values(d3Ptrs);
  if (pts.length >= 2){
    const d = Math.hypot(pts[0].x - pts[1].x, pts[0].y - pts[1].y) || 1;
    d3.cam.scale = Math.max(0.02, Math.min(2, d3Pinch.s0 * d / d3Pinch.d0));
    d3Draw();
  }
  return;
}
if (!d3Drag) return;
const dx = m.x - d3Drag.m0.x, dy = m.y - d3Drag.m0.y;

if (d3Drag.tipo === "cam"){
  d3.cam.az = d3Drag.az0 + dx * 0.4;
  d3.cam.el = Math.max(-85, Math.min(85, d3Drag.el0 + dy * 0.3));
  d3Draw();
  return;
}
if (d3Drag.tipo === "pan"){
  const { r, u } = d3CamVecs();
  const k = 1 / d3.cam.scale;
  d3.cam.tx = d3Drag.t0.tx - (dx * r.x - dy * u.x) * k;
  d3.cam.ty = Math.max(0, d3Drag.t0.ty - (dx * r.y - dy * u.y) * k);
  d3.cam.tz = d3Drag.t0.tz - (dx * r.z - dy * u.z) * k;
  d3Draw();
  return;
}
if (d3Drag.tipo === "pieza"){
  if (Math.abs(dx) + Math.abs(dy) > 3) d3Drag.moved = true;
  const ray = d3Ray(m.x, m.y);
  const p = d3Drag.p;
  if (e.shiftKey && !d3Drag.vertical){
    const { f } = d3CamVecs();
    const fl = Math.hypot(f.x, f.z) || 1;
    d3Drag.vNormal = { x: f.x / fl, y: 0, z: f.z / fl };
    const a = d3AABB(p);
    d3Drag.vPoint = { x: a.cx, y: a.cy, z: a.cz };
    const t0 = d3RayPlane(ray, a.cx, a.cy, a.cz, d3Drag.vNormal.x, 0, d3Drag.vNormal.z);
    d3Drag.vOff = t0 !== null ? (ray.o.y + ray.d.y * t0) - p.y : 0;
    d3Drag.vertical = true;
```

```
    }
    if (d3Drag.vertical){
      const t = d3RayPlane(ray, d3Drag.vPoint.x, d3Drag.vPoint.y, d3Drag.vPoint.z,
d3Drag.vNormal.x, 0, d3Drag.vNormal.z);
      if (t !== null){
        const y = ray.o.y + ray.d.y * t - d3Drag.vOff;
        const adj = d3Magnet(p, { x: p.x, y: Math.max(0, y), z: p.z });
        p.y = Math.max(0, adj.y);
        d3.snaps = adj.snaps;
      }
    } else {
      const t = d3RayPlane(ray, 0, d3Drag.baseY, 0, 0, 1, 0);
      if (t !== null){
        const pos = {
          x: ray.o.x + ray.d.x * t - d3Drag.off.x,
          y: p.y,
          z: ray.o.z + ray.d.z * t - d3Drag.off.z
        };
        const adj = d3Magnet(p, pos);
        p.x = adj.x; p.y = adj.y; p.z = adj.z;
        d3.snaps = adj.snaps;
      }
    }
    d3Draw();
  }
}

function d3Up(e){
  const m = d3Local(e);
  const wasDrag = d3Drag;
  delete d3Ptrs[e.pointerId];
  if (Object.keys(d3Ptrs).length < 2) d3Pinch = null;
  d3Drag = null;
  d3.snaps = [];
  if (!wasDrag){ d3Draw(); return; }
  const moved = Math.hypot(m.x - wasDrag.m0.x, m.y - wasDrag.m0.y) > 5;
  if (wasDrag.tipo === "pieza"){
    if (wasDrag.moved) d3Save();
    d3SelRender();
    d3Draw();
    return;
  }
  if (!moved && wasDrag.clickMode === "regla") d3ClickRegla(m);
```

```
  else if (!moved && wasDrag.clickMode === "angulo") d3ClickAngulo(m);
  else if (!moved){
    const hit = d3Pick(m.x, m.y);
    d3.sel = hit ? hit.id : null;
    d3SelRender(); d3TelRender();
  }
  d3Status();
  d3Draw();
}

/* regla: dos puntos → distancia y diferencias por eje */
function d3ClickRegla(m){
  const w = d3SnapAt(m.x, m.y);
  if (!w) return;
  if (!d3.regla){ d3.regla = w; d3Draw(); d3Status(); return; }
  const a = d3.regla, b = w;
  const d = Math.hypot(a.x - b.x, a.y - b.y, a.z - b.z);
  d3.cotas.push({
    type: "dist", a, b,
    txt: mmv(d) + " mm  (ΔX " + mmv(b.x - a.x) + " · ΔY " + mmv(b.y - a.y) + " · ΔZ " + mmv(b.z -
a.z) + ")"
  });
  d3.regla = null; d3.marks = [];
  d3Save(); d3Draw(); d3Status();
}

/* transportador: tres puntos → ángulo en el vértice (segundo punto) */
function d3ClickAngulo(m){
  const w = d3SnapAt(m.x, m.y);
  if (!w) return;
  d3.angPts.push(w);
  if (d3.angPts.length >= 3){
    const [p1, p2, p3] = d3.angPts;
    const v1 = { x: p1.x - p2.x, y: p1.y - p2.y, z: p1.z - p2.z };
    const v2 = { x: p3.x - p2.x, y: p3.y - p2.y, z: p3.z - p2.z };
    const dot = (v1.x*v2.x + v1.y*v2.y + v1.z*v2.z) / ((Math.hypot(v1.x, v1.y, v1.z) || 1) *
(Math.hypot(v2.x, v2.y, v2.z) || 1));
    d3.cotas.push({ type: "ang", p1, p2, p3, val: Math.acos(Math.max(-1, Math.min(1, dot))) * 180
/ Math.PI });
    d3.angPts = []; d3.marks = [];
    d3Save();
  } else {
    d3.marks = d3.angPts.slice();
```

```
  }
  d3Draw(); d3Status();
}

/* barra de estado y estadísticas */
function d3Status(){
  const sel = d3.pieces.find(p => p.id === d3.sel);
  document.getElementById("d3Status").innerHTML =
    "Modo <b>" + ({ mover: "Mover", cam: "Cámara", regla: "Regla", angulo: "Transportador"
})[d3.mode] + "</b>" +
    " · Imán <b>" + (d3.magnet ? "ON (" + mmv(Math.max(1, +d3.capture || 8)) + " mm)" : "OFF")
+ "</b>" +
    " · <b>" + d3.pieces.length + "</b> pieza(s)" +
    (d3.mode === "regla" ? (d3.regla ? " · tocá el segundo punto" : " · tocá el primer punto") : "") +
    (d3.mode === "angulo" ? " · puntos: " + d3.angPts.length + "/3 (el 2° es el vértice)" : "") +
    (sel ? " · <b>" + esc(sel.name) + "</b>" : "");
  document.getElementById("d3PieceCount").textContent = d3.pieces.length ? d3.pieces.length
+ " piezas" : "";
}
function d3StatsRender(){
  const kgPer = d3.pieces.filter(p => p.kind === "perfil").reduce((s, p) => s + d3PieceWeight(p),
0);
  const kgChapa = d3.pieces.filter(p => p.kind === "chapa" || p.kind === "plegada").reduce((s, p)
=> s + d3PieceWeight(p), 0);
  const m2mad = d3.pieces.filter(p => p.kind === "madera").reduce((s, p) => s + (p.l * p.w) / 1e6,
0);
  document.getElementById("d3Stats").innerHTML =
    "<span>Caños <b>" + f1.format(kgPer) + " kg</b></span>" +
    "<span>Chapa <b>" + f1.format(kgChapa) + " kg</b></span>" +
    "<span>Madera <b>" + m2(m2mad) + "</b></span>";
}

/* eventos del visor */
d3cv.addEventListener("contextmenu", e => e.preventDefault());
d3cv.addEventListener("pointerdown", d3Down);
d3cv.addEventListener("pointermove", d3Move);
d3cv.addEventListener("pointerup", d3Up);
d3cv.addEventListener("pointercancel", e => {
  delete d3Ptrs[e.pointerId];
  d3Pinch = null; d3Drag = null; d3.snaps = [];
  d3Draw();
});
d3cv.addEventListener("wheel", e => {
```

```
  e.preventDefault();
  d3.cam.scale = Math.max(0.02, Math.min(2, d3.cam.scale * (e.deltaY > 0 ? 0.9 : 1.1)));
  d3Draw();
}, { passive: false });
if (typeof ResizeObserver !== "undefined"){
  new ResizeObserver(() => { d3Draw(); }).observe(d3cv);
}

/*
================================================================================
   10.d DISEÑO 3D — paneles, despiece e integración con Corte / Materiales

================================================================================ */
const D3_TRAMO_DEFS = { L: [80, 40], U: [40, 120, 40], C: [25, 100, 25], Z: [40, 80], bandeja: [300, 30] };

function d3TramosRender(){
  const tmpl = document.getElementById("d3fTmpl").value;
  const labels = D3_TMPL[tmpl].segs, defs = D3_TRAMO_DEFS[tmpl];
  document.getElementById("d3fTramos").innerHTML = labels.map((l, i) =>
    `<label><span class="lab">${l}</span><input type="number" class="d3seg" data-i="${i}" min="1" step="1" inputmode="numeric" value="${defs[i]}"></label>`
  ).join("");
  d3PlegadaPreview();
}

function d3PlegadaPreview(){
  const tmpl = document.getElementById("d3fTmpl").value;
  const t = numOr(document.getElementById("d3fT").value, 1.2, 0.3, 4);
  const L = numOr(document.getElementById("d3fL").value, 500, 1);
  const r = numOr(document.getElementById("d3fR").value, t, 0);
  const K = numOr(document.getElementById("d3fK").value, 0.4, 0, 1);
  const dims = [...document.querySelectorAll("#d3fTramos .d3seg")].map(i => numOr(i.value, 50, 1));
  const res = d3Plegada(tmpl, dims, t, L, r, K);
  document.getElementById("d3fOut").innerHTML =
    `<div><span class="k">Desarrollo (plano)</span><span class="v">${mmv(res.devA)} × ${mmv(res.devB)} mm</span></div>` +
    `<div><span class="k">Dobleces</span><span class="v">${res.bends}</span></div>` +
    `<div><span class="k">Peso</span><span class="v">${f2.format(res.weight)} kg</span></div>` +
```

```
    (res.warns.length ? `<div class="hint" style="flex:1 1 100%">⚠️ ${res.warns.join(" · ")}</div>`
: "");
}

function d3CatRender(){
  const pr = d3Prof(document.getElementById("d3pProf").value);
  const wall = parseFloat(document.getElementById("d3pWall").value) || pr.walls[0];
  const c = d3CatGet(pr.id, wall);
  document.getElementById("d3CatList").innerHTML = `
    <div class="d3cat-row">
      <div class="head"><b>${pr.name}</b>
        <span class="d3mini">pared ${wall} mm · ${f2.format(d3Kgm(pr.a, pr.b, wall))} kg/m ·
interior libre ${mmv(pr.a - 2*wall)} × ${mmv(pr.b - 2*wall)} mm</span></div>
      <div class="grid g2">
        <label><span class="lab">Precio por barra ($)</span><input type="number"
data-cat="price" min="0" step="0.01" inputmode="decimal" value="${c.price || ""}"></label>
        <label><span class="lab">Largo de barra (mm)</span><input type="number"
data-cat="barLen" min="1" step="100" inputmode="numeric" value="${c.barLen ||
6000}"></label>
      </div>
    </div>`;
}

/* ---------- panel de la pieza seleccionada ---------- */
function d3SelRender(){
  const box = document.getElementById("d3SelBox");
  const p = d3.pieces.find(q => q.id === d3.sel);
  if (!p){ box.innerHTML = '<p class="empty">Nada seleccionado. Tocá una pieza en el
visor.</p>'; return; }
  const D = d3Dims(p);
  let info = "";
  if (p.kind === "madera" || p.kind === "chapa") info = `${mmv(p.l)} × ${mmv(p.w)} × ${p.t} mm`;
  else if (p.kind === "perfil"){
    const pr = d3Prof(p.profId);
    info = `${pr.name} ×${p.wall} · ${mmv(p.len)} mm · ${f2.format(d3Kgm(pr.a, pr.b, p.wall))}
kg/m`;
  }
  else info = `${D3_TMPL[p.tmpl] ? D3_TMPL[p.tmpl].label : p.tmpl} ${p.t} mm · desarrollo
${mmv(p.devA)}×${mmv(p.devB)} · ${p.bends} dobleces`;
  const axisCtl = p.kind === "madera"
    ? '<select data-s="axis"><option value="x">Acostada</option><option
value="y">Parada</option></select>'
    : p.kind === "perfil"
```

```
    ? '<select data-s="axis"><option value="y">Vertical</option><option value="x">Horizontal X</option><option value="z">Horizontal Z</option></select>'
    : '<input type="text" value="—" readonly tabindex="-1" style="background:var(--panel-2)">';
  const lenCtl = p.kind === "madera" || p.kind === "perfil"
    ? `<div class="grid g2" style="margin-top:8px">
        <label><span class="lab">Largo (mm)</span><input type="number" data-s="len" step="1" inputmode="numeric" value="${p.kind === "madera" ? p.l : p.len}"></label>
        ${p.kind === "madera" ? `<label><span class="lab">Ancho (mm)</span><input type="number" data-s="w" step="1" inputmode="numeric" value="${p.w}"></label>` : "<span></span>"}
      </div>` : "";
  box.innerHTML = `
    <div class="grid g2">
      <label><span class="lab">Nombre</span><input type="text" data-s="name" value="${esc(p.name)}"></label>
      <label><span class="lab">Datos</span><input type="text" value="${esc(info)}" readonly tabindex="-1" style="background:var(--panel-2)"></label>
    </div>
    <div class="d3sel-grid">
      <label><span class="lab">X (mm)</span><input type="number" data-s="x" step="1" inputmode="numeric" value="${Math.round(p.x)}"></label>
      <label><span class="lab">Y (altura)</span><input type="number" data-s="y" step="1" inputmode="numeric" value="${Math.round(p.y)}"></label>
      <label><span class="lab">Z (mm)</span><input type="number" data-s="z" step="1" inputmode="numeric" value="${Math.round(p.z)}"></label>
    </div>
    <div class="grid g2" style="margin-top:8px">
      <label><span class="lab">Rotación (grados)</span><input type="number" data-s="rot" step="1" inputmode="numeric" value="${p.rot || 0}"></label>
      <label><span class="lab">Puesta</span>${axisCtl}</label>
    </div>
    ${lenCtl}
    <div class="d3nudge">
      <button class="btn quiet" data-n="x-100" type="button">◂ 100</button>
      <button class="btn quiet" data-n="x-10" type="button">◂ 10</button>
      <button class="btn quiet" data-n="x+10" type="button">10 ▸</button>
      <button class="btn quiet" data-n="x+100" type="button">100 ▸</button>
      <button class="btn quiet" data-n="y-10" type="button">▼ 10</button>
      <button class="btn quiet" data-n="y+10" type="button">▲ 10</button>
      <button class="btn quiet" data-n="z-100" type="button">↩ 100</button>
      <button class="btn quiet" data-n="z+100" type="button">100 ↪</button>
    </div>
    <div class="d3nudge">
```

```
      <button class="btn quiet" data-n="rot-" type="button">↺ 15°</button>
      <button class="btn quiet" data-n="rot+" type="button">15° ↻</button>
      <button class="btn quiet" data-n="piso" type="button">Apoyar en piso</button>
      <button class="btn quiet" data-n="centrar" type="button">Centrar en el eje</button>
    </div>
    <div class="savebar" style="margin-top:10px">
      <button class="btn quiet" data-n="dup" type="button">Duplicar</button>
      <button class="btn danger" data-n="del" type="button">Eliminar</button>
    </div>`;
  const selAxis = box.querySelector("select[data-s=axis]");
  if (selAxis) selAxis.value = p.axis || "x";
}

/* ---------- panel de telescopía ---------- */
function d3OuterOf(p){
  if (p.kind !== "perfil") return null;
  const a = d3AABB(p);
  return d3.pieces.find(q => {
    if (q.id === p.id || q.kind !== "perfil" || q.axis !== p.axis) return false;
    const b = d3AABB(q);
    return a.x1 >= b.x1 - 1 && a.x2 <= b.x2 + 1 && a.y1 >= b.y1 - 1 && a.y2 <= b.y2 + 1 && a.z1
>= b.z1 - 1 && a.z2 <= b.z2 + 1;
  }) || null;
}

function d3TelRender(){
  const box = document.getElementById("d3TelBox");
  const p = d3.pieces.find(q => q.id === d3.sel);
  if (!p || p.kind !== "perfil"){
    box.innerHTML = '<p class="empty">Seleccioná un perfil para ver qué caños entran
adentro.</p>';
    return;
  }
  const pr = d3Prof(p.profId), e = p.wall;
  const inA = pr.a - 2*e, inB = pr.b - 2*e;
  const h = Math.max(0, +d3.holgura || 0);
  const fits = D3_PROFILES.filter(q =>
    q.id !== pr.id &&
    ((q.a <= inA - 2*h && q.b <= inB - 2*h) || (q.a <= inB - 2*h && q.b <= inA - 2*h))
  );
  let html = `<div class="d3mini">Interior libre de ${pr.name} ×${e}: <b>${mmv(inA)} ×
${mmv(inB)} mm</b> · holgura ${h} mm por lado</div>`;
  if (fits.length){
```

```
    html += fits.map(q => `
      <div class="d3tel-fit">
        <span><b>${q.name}</b> <span class="d3mini">(${mmv(q.a)}×${mmv(q.b)}, paredes
${q.walls.join(" / ")})</span></span>
        <button class="btn quiet" data-addin="${q.id}" type="button">＋ adentro</button>
      </div>`).join("");
  } else {
    html += '<p class="empty">Ningún caño del catálogo entra con esa holgura.</p>';
  }
  if (d3OuterOf(p)){
    html += `
      <div class="grid g2" style="margin-top:10px">
        <label><span class="lab">Solape mínimo (mm)</span><input type="number"
id="d3Solape" min="0" step="10" inputmode="numeric" value="${+d3.solape ||
100}"></label><span></span>
      </div>
      <div class="d3nudge">
        <button class="btn quiet" data-tel="extender" type="button">Extender al tope</button>
        <button class="btn quiet" data-tel="cerrar" type="button">Cerrar (adentro)</button>
      </div>
      <p class="hint">Los botones solo mueven la pieza: <b>no la atan</b>. Seguis libre de
separarla cuando quieras.</p>`;
  }
  box.innerHTML = html;
}

/* ---------- despiece ---------- */
function d3CutRows(){
  const groups = [];
  for (const p of d3.pieces){
    const key = JSON.stringify([p.kind, p.name, p.l, p.w, p.t, p.len, p.profId, p.wall, p.axis, p.rot,
p.tmpl, p.devA, p.devB]);
    const g = groups.find(g => g.key === key);
    if (g) g.qty++; else groups.push({ key, p, qty: 1 });
  }
  return groups;
}

function d3OutRender(){
  const rows = d3CutRows();
  const tbl = document.getElementById("d3CutTable");
  if (!rows.length){
    tbl.innerHTML = "";
```

```
    document.getElementById("d3Summary").innerHTML = '<p class="empty">Todavía no agregaste piezas al diseño.</p>';
    d3StatsRender();
    return;
  }
  tbl.innerHTML = "<thead><tr><th>Pieza</th><th>Tipo</th><th class='r'>Largo</th><th class='r'>Ancho/alto</th><th class='r'>Cant.</th><th class='r'>Rot.</th></tr></thead><tbody>" +
    rows.map(g => {
      const p = g.p;
      let tipo = "", largo = "", ancho = "";
      if (p.kind === "madera"){ tipo = "Tablero " + p.t + " mm"; largo = mmv(p.l); ancho = mmv(p.w); }
      else if (p.kind === "perfil"){ tipo = d3Prof(p.profId).name + " ×" + p.wall; largo = mmv(p.len); ancho = "90°"; }
      else if (p.kind === "chapa"){ tipo = "Chapa " + p.t + " mm"; largo = mmv(p.l); ancho = mmv(p.w); }
      else { tipo = (D3_TMPL[p.tmpl] ? D3_TMPL[p.tmpl].label : p.tmpl) + " " + p.t + " mm (desarrollo)"; largo = mmv(p.devA); ancho = mmv(p.devB); }
      return `<tr><td>${esc(p.name)}${p._sent ? ' <span class="d3mini">✓ enviado</span>' : ""}</td><td>${esc(tipo)}</td><td class="r">${largo}</td><td class="r">${ancho}</td><td class="r">${g.qty}</td><td class="r">${p.rot || 0}°</td></tr>`;
    }).join("") + "</tbody>";

  const woods = d3.pieces.filter(p => p.kind === "madera");
  const sheets = d3.pieces.filter(p => p.kind === "chapa" || p.kind === "plegada");
  const profs = d3.pieces.filter(p => p.kind === "perfil");
  const bends = sheets.reduce((s, p) => s + (p.bends || 0), 0);
  let uniones = 0;
  for (let i = 0; i < profs.length; i++){
    for (let j = i + 1; j < profs.length; j++){
      const a = d3AABB(profs[i]), b = d3AABB(profs[j]);
      const gap = Math.max(b.x1 - a.x2, a.x1 - b.x2, b.y1 - a.y2, a.y1 - b.y2, b.z1 - a.z2, a.z1 - b.z2);
      if (gap <= 2) uniones++;
    }
  }
  document.getElementById("d3Summary").innerHTML =
    `<div class="d3out">
      <div><span class="k">Tableros</span><span class="v">${woods.length}</span></div>
      <div><span class="k">Perfiles</span><span class="v">${profs.length}</span></div>
      <div><span class="k">Chapas / desarrollos</span><span class="v">${sheets.length}</span></div>
      <div><span class="k">Dobleces</span><span class="v">${bends}</span></div>
```

```
    <div><span class="k">Uniones a soldar</span><span class="v">${uniones}</span></div>
   </div>`;
  d3StatsRender();
}

/* mandar madera y chapa al Plano de corte */
function d3SendCorte(){
  const woods = d3.pieces.filter(p => p.kind === "madera" && !p._sent);
  const sheets = d3.pieces.filter(p => (p.kind === "chapa" || p.kind === "plegada") && !p._sent);
  if (!woods.length && !sheets.length){ flag("Nada nuevo para mandar: tocá «Reenviar todo» si
querés repetir"); return; }
  let added = 0, thickWarn = false;

  if (woods.length){
    const thick = new Set(woods.map(p => String(p.t)));
    if (thick.size === 1) state.board.thick = +[...thick][0];
    else thickWarn = true;
    for (const p of woods){
      const ex = state.pieces.find(q => q.name === p.name && q.l === p.l && q.w === p.w &&
!q.grain);
      if (ex) ex.qty += 1;
      else state.pieces.push({ id: uid(), name: p.name, l: p.l, w: p.w, qty: 1, grain: false });
      p._sent = true; added++;
    }
  }

  if (sheets.length){
    state.cut.material = "metal";
    if (!state.cut.metalType) state.cut.metalType = "acero";
    const thick = new Set(sheets.map(p => String(p.t)));
    if (thick.size === 1) state.board.thick = +[...thick][0];
    let bendsAdded = 0;
    for (const p of sheets){
      const l = p.kind === "plegada" ? p.devA : p.l;
      const w = p.kind === "plegada" ? p.devB : p.w;
      const nm = p.kind === "plegada" ? p.name + " — desarrollo" : p.name;
      const ex = state.pieces.find(q => q.name === nm && q.l === l && q.w === w);
      if (ex) ex.qty += 1;
      else state.pieces.push({ id: uid(), name: nm, l, w, qty: 1, grain: false });
      if (p.kind === "plegada") bendsAdded += p.bends || 0;
      p._sent = true; added++;
    }
    if (bendsAdded) state.cut.bends = (state.cut.bends || 0) + bendsAdded;
```

```
  }
  save(); render();
  d3OutRender(); d3Save();
  flag(added + " pieza(s) mandada(s) al Plano de corte" + (thickWarn ? " · ojo: espesores
distintos, el Plano usa uno solo" : ""));
}

/* mandar perfiles al Cotizador de materiales */
function d3SendMat(){
  const profs = d3.pieces.filter(p => p.kind === "perfil" && !p._sent);
  if (!profs.length){ flag("Nada nuevo para mandar: tocá «Reenviar todo» si querés repetir");
return; }
  const groups = [];
  for (const p of profs){
    const k = p.profId + "|" + p.wall;
    const g = groups.find(g => g.k === k);
    if (g){ g.cuts.push(p.len); g.need += p.len; }
    else groups.push({ k, cuts: [p.len], need: p.len });
  }
  let n = 0;
  for (const g of groups){
    const [pid, wall] = g.k.split("|");
    const pr = d3Prof(pid);
    const c = d3CatGet(pid, parseFloat(wall));
    mat.items.push(sanitizeMatItem({
      kind: "barra",
      name: pr.name + " ×" + wall + " — " + g.cuts.length + " corte" + (g.cuts.length === 1 ? "" :
"s"),
      price: c.price, barLen: c.barLen, need: Math.round(g.need),
      kerf: Math.max(0, +d3.kerfBarra || 0),
      cuts: g.cuts.map(x => Math.round(x))
    }));
    n += g.cuts.length;
  }
  profs.forEach(p => { p._sent = true; });
  saveMat(); renderMat();
  d3OutRender(); d3Save();
  flag(n + " corte(s) de perfil mandados al Cotizador");
}

/* ---------- demo, reinicio, impresión, import/export ---------- */
function loadDemo3D(){
  d3.pieces = []; d3.cotas = []; d3.sel = null; d3.regla = null; d3.angPts = []; d3.marks = [];
```

```
  [[40, 40], [760, 40], [40, 560], [760, 560]].forEach(([x, z]) => {
    const p = d3NewPiece("perfil", { profId: "sq40", wall: 1.6, len: 700, axis: "y", name: "Pata caño
40×40" });
    p.x = x; p.z = z; p.y = 0;
    d3.pieces.push(p);
  });
  [[40], [540]].forEach(([z]) => {
    const p = d3NewPiece("perfil", { profId: "re4020", wall: 1.6, len: 680, axis: "x", name:
"Travesaño 40×20" });
    p.x = 80; p.z = z; p.y = 650;
    d3.pieces.push(p);
  });
  const tapa = d3NewPiece("madera", { name: "Tapa de mesa", l: 800, w: 600, t: 18, axis: "x" });
  tapa.y = 700; tapa.x = 0; tapa.z = 0;
  d3.pieces.push(tapa);
  const res = d3Plegada("bandeja", [280, 30], 1.2, 400, 1.2, 0.4);
  const ban = d3NewPiece("plegada", { name: "Bandeja chapa 1,2", tmpl: "bandeja", dims: [280,
30], t: 1.2, L: 400, r: 1.2, K: 0.4, segs: res.segs, devA: res.devA, devB: res.devB, bends:
res.bends });
  ban.x = 0; ban.z = 700; ban.y = 0;
  d3.pieces.push(ban);
  d3Fit();
  d3SelRender(); d3TelRender(); d3OutRender(); d3Status(); d3StatsRender(); d3Draw();
d3Save();
  flag("Ejemplo 3D cargado: mesa con patas de caño y bandeja");
}

function resetD3(){
  if (d3.pieces.length && !confirm("¿Borrar el diseño 3D completo?")) return;
  d3.pieces = []; d3.cotas = []; d3.sel = null; d3.regla = null; d3.angPts = []; d3.marks = [];
  d3SelRender(); d3TelRender(); d3OutRender(); d3Status(); d3StatsRender(); d3Draw();
d3Save();
  flag("Diseño vaciado");
}
function d3PrintExtras(){
  document.getElementById("printhead").innerHTML =
    "<h1>Diseño 3D — despiece</h1><div class='ph'><span><b>Acero y Escamas</b> ·
medidas en mm</span></div>";
  const tbl = document.getElementById("d3CutTable");
  document.getElementById("printlist").innerHTML = tbl.innerHTML ? tbl.innerHTML : "";
}

/* ---------- inicialización de la pestaña ---------- */
```

```
function d3Init(){
  d3Load();
  document.getElementById("d3Magnet").checked = !!d3.magnet;
  document.getElementById("d3Capture").value = d3.capture;
  document.getElementById("d3RotStep").value = String(d3.rotStep);
  document.getElementById("d3Holgura").value = d3.holgura;
  document.getElementById("d3KerfBarra").value = d3.kerfBarra;
  document.getElementById("d3PleMax").value = d3.ple.plegadora || "";
  document.getElementById("d3CizMax").value = d3.ple.cizalla || "";
  d3Bind();
  d3SelRender(); d3TelRender(); d3OutRender(); d3Status(); d3StatsRender();
}

function d3Bind(){
  const kindSel = document.getElementById("d3Kind");
  const boxes = { madera: "d3mBox", perfil: "d3pBox", chapa: "d3cBox", plegada: "d3fBox" };
  kindSel.addEventListener("change", () => {
    for (const k of Object.keys(boxes)) document.getElementById(boxes[k]).hidden = k !==
kindSel.value;
    if (kindSel.value === "plegada") d3TramosRender();
  });

  /* catálogo de perfiles */
  const profSel = document.getElementById("d3pProf");
  const wallSel = document.getElementById("d3pWall");
  D3_PROFILES.forEach(pr => {
    const o = document.createElement("option");
    o.value = pr.id;
    o.textContent = pr.name + " — " + f2.format(d3Kgm(pr.a, pr.b, pr.walls[1])) + " kg/m aprox";
    profSel.appendChild(o);
  });
  const fillWalls = () => {
    const pr = d3Prof(profSel.value);
    wallSel.innerHTML = "";
    pr.walls.forEach(w => {
      const o = document.createElement("option");
      o.value = String(w); o.textContent = w + " mm";
      wallSel.appendChild(o);
    });
    wallSel.value = String(pr.walls.includes(1.6) ? 1.6 : pr.walls[1]);
    d3CatRender();
  };
  profSel.addEventListener("change", fillWalls);
```

```
  wallSel.addEventListener("change", d3CatRender);
  fillWalls();

  document.getElementById("d3CatList").addEventListener("input", e => {
    const f = e.target.dataset.cat;
    if (!f) return;
    const pr = d3Prof(document.getElementById("d3pProf").value);
    const wall = parseFloat(document.getElementById("d3pWall").value) || pr.walls[0];
    const c = d3CatGet(pr.id, wall);
    c[f] = numOr(e.target.value, c[f], f === "barLen" ? 1 : 0);
    d3Save();
  });

  /* plantillas de plegada */
  document.getElementById("d3fTmpl").addEventListener("change", d3TramosRender);
  ["d3fT", "d3fL", "d3fR", "d3fK"].forEach(id =>
document.getElementById(id).addEventListener("input", d3PlegadaPreview));
  document.getElementById("d3fTramos").addEventListener("input", d3PlegadaPreview);
  d3TramosRender();

  /* agregar pieza */
  document.getElementById("d3Add").addEventListener("click", () => {
    const kind = kindSel.value;
    let p = null;
    if (kind === "madera"){
      p = d3NewPiece("madera", {
        name: document.getElementById("d3mName").value.trim(),
        l: numOr(document.getElementById("d3mL").value, 800, 1),
        w: numOr(document.getElementById("d3mW").value, 600, 1),
        t: numOr(document.getElementById("d3mT").value, 18, 0.5),
        axis: document.getElementById("d3mAxis").value
      });
    } else if (kind === "perfil"){
      p = d3NewPiece("perfil", {
        profId: d3Prof(document.getElementById("d3pProf").value).id,
        wall: parseFloat(document.getElementById("d3pWall").value) || 1.6,
        len: numOr(document.getElementById("d3pLen").value, 700, 1),
        axis: document.getElementById("d3pAxis").value
      });
    } else if (kind === "chapa"){
      p = d3NewPiece("chapa", {
        l: numOr(document.getElementById("d3cL").value, 500, 1),
        w: numOr(document.getElementById("d3cW").value, 300, 1),
```

```
      t: numOr(document.getElementById("d3cT").value, 1.2, 0.3, 4),
      axis: "x"
    });
  } else {
    const tmpl = document.getElementById("d3fTmpl").value;
    const t = numOr(document.getElementById("d3fT").value, 1.2, 0.3, 4);
    const L = numOr(document.getElementById("d3fL").value, 500, 1);
    const r = numOr(document.getElementById("d3fR").value, t, 0);
    const K = numOr(document.getElementById("d3fK").value, 0.4, 0, 1);
    const dims = [...document.querySelectorAll("#d3fTramos .d3seg")].map(i => numOr(i.value,
50, 1));
    const res = d3Plegada(tmpl, dims, t, L, r, K);
    p = d3NewPiece("plegada", { tmpl, dims, t, L, r, K, segs: res.segs, devA: res.devA, devB:
res.devB, bends: res.bends });
  }
  d3.pieces.push(p);
  d3.sel = p.id;
  d3SelRender(); d3TelRender(); d3OutRender(); d3Status(); d3StatsRender();
  d3Save(); d3Draw();
  flag("Pieza agregada");
});

/* herramientas */
[["d3mMover", "mover"], ["d3mCam", "cam"], ["d3mRegla", "regla"], ["d3mAng",
"angulo"]].forEach(([id, m]) =>
  document.getElementById(id).addEventListener("click", () => d3SetMode(m)));
document.getElementById("d3Magnet").addEventListener("change", e => { d3.magnet =
e.target.checked; d3Save(); d3Status(); });
document.getElementById("d3Capture").addEventListener("input", e => { d3.capture =
numOr(e.target.value, 8, 1, 50); d3Save(); d3Status(); });
document.getElementById("d3RotStep").addEventListener("change", e => { d3.rotStep =
+e.target.value; d3Save(); });
document.getElementById("d3Holgura").addEventListener("input", e => { d3.holgura =
numOr(e.target.value, 1, 0, 10); d3Save(); d3TelRender(); });
document.getElementById("d3KerfBarra").addEventListener("input", e => { d3.kerfBarra =
numOr(e.target.value, 3, 0, 20); d3Save(); });
document.getElementById("d3PleMax").addEventListener("input", e => { d3.ple.plegadora =
numOr(e.target.value, 0, 0); d3Save(); d3PlegadaPreview(); });
document.getElementById("d3CizMax").addEventListener("input", e => { d3.ple.cizalla =
numOr(e.target.value, 0, 0); d3Save(); d3PlegadaPreview(); });
document.querySelectorAll(".d3view").forEach(b => b.addEventListener("click", () => {
  const v = b.dataset.v;
  if (v === "frente"){ d3.cam.az = 0; d3.cam.el = 0; }
```

```
  else if (v === "lado"){ d3.cam.az = 90; d3.cam.el = 0; }
  else if (v === "arriba"){ d3.cam.az = 0; d3.cam.el = 85; }
  else if (v === "iso"){ d3.cam.az = -35; d3.cam.el = 22; }
  else if (v === "fit") d3Fit();
  else if (v === "in") d3.cam.scale = Math.min(2, d3.cam.scale * 1.25);
  else if (v === "out") d3.cam.scale = Math.max(0.02, d3.cam.scale / 1.25);
  d3Draw();
}));

/* view cube (cubito de vista en la esquina del canvas) */
const VC_VIEWS = {
  frente:  { az: 0,   el: 0 },
  lado:    { az: 90,  el: 0 },
  "lado-izq": { az: -90, el: 0 },
  atras:   { az: 180, el: 0 },
  arriba:  { az: 0,   el: 85 },
  abajo:   { az: 0,   el: -85 },
  iso:     { az: -35, el: 22 }
};
document.getElementById("d3ViewCube").addEventListener("click", e => {
  const face = e.target.closest(".vc-face, .vc-corner");
  if (!face) return;
  const v = face.dataset.view;
  const cfg = VC_VIEWS[v];
  if (cfg){ d3.cam.az = cfg.az; d3.cam.el = cfg.el; d3Draw(); }
});
document.getElementById("d3ClearCotas").addEventListener("click", () => {
  d3.cotas = []; d3.regla = null; d3.angPts = []; d3.marks = [];
  d3Save(); d3Draw(); d3Status();
  flag("Mediciones borradas");
});

/* pieza seleccionada */
const selBox = document.getElementById("d3SelBox");
selBox.addEventListener("input", e => {
  const f = e.target.dataset.s;
  if (!f) return;
  const p = d3.pieces.find(q => q.id === d3.sel);
  if (!p) return;
  if (f === "name"){ p.name = e.target.value; d3Status(); }
  else if (f === "x" || f === "y" || f === "z") p[f] = Math.round(numOr(e.target.value, p[f], 0));
  else if (f === "rot"){
    let v = numOr(e.target.value, 0, -360, 360);
```

```
      if (d3.rotStep > 0) v = Math.round(v / d3.rotStep) * d3.rotStep;
      p.rot = ((v % 360) + 360) % 360;
    }
    else if (f === "axis"){ p.axis = e.target.value; d3SelRender(); }
    else if (f === "len"){
      if (p.kind === "madera") p.l = Math.max(1, Math.round(numOr(e.target.value, p.l, 1)));
      else p.len = Math.max(1, Math.round(numOr(e.target.value, p.len, 1)));
    }
    else if (f === "w") p.w = Math.max(1, Math.round(numOr(e.target.value, p.w, 1)));
    d3Save(); d3Draw(); d3OutRender(); d3StatsRender();
  });
  selBox.addEventListener("click", e => {
    const b = e.target.closest("[data-n]");
    if (!b) return;
    const p = d3.pieces.find(q => q.id === d3.sel);
    if (!p) return;
    const a = b.dataset.n;
    if (a === "dup"){
      const copy = JSON.parse(JSON.stringify(p));
      copy.id = uid(); copy.name = p.name + " (copia)"; copy.x += 60; copy.z += 60; copy._sent =
false;
      d3.pieces.push(copy); d3.sel = copy.id;
    } else if (a === "del"){
      d3.pieces = d3.pieces.filter(q => q.id !== p.id);
      d3.sel = null;
    } else if (a === "piso"){ p.y = 0; }
    else if (a === "centrar"){
      const bb = d3AABB(p);
      p.x += -bb.cx; p.z += -bb.cz;
    } else if (a === "rot-"){ p.rot = ((p.rot - 15) % 360 + 360) % 360; }
    else if (a === "rot+"){ p.rot = ((p.rot + 15) % 360) % 360; }
    else if (a[0] === "x") p.x += +a.slice(1);
    else if (a[0] === "y") p.y = Math.max(0, p.y + +a.slice(1));
    else if (a[0] === "z") p.z += +a.slice(1);
    d3SelRender(); d3TelRender(); d3OutRender(); d3Status(); d3StatsRender();
    d3Save(); d3Draw();
  });

  /* telescopía */
  const telBox = document.getElementById("d3TelBox");
  telBox.addEventListener("input", e => {
    if (e.target.id === "d3Solape"){ d3.solape = numOr(e.target.value, 100, 0); d3Save(); }
  });
```

```
  telBox.addEventListener("click", e => {
    const addIn = e.target.closest("[data-addin]");
    const tel = e.target.closest("[data-tel]");
    if (!addIn && !tel) return;
    const p = d3.pieces.find(q => q.id === d3.sel);
    if (!p) return;
    if (addIn){
      const q = d3Prof(addIn.dataset.addin);
      const wall = q.walls.includes(p.wall) ? p.wall : q.walls[1];
      const inner = d3NewPiece("perfil", { profId: q.id, wall, len: Math.max(100, Math.round((p.len
|| 600) * 0.7)), axis: p.axis });
      const prOut = d3Prof(p.profId);
      if (p.axis === "y"){ inner.x = p.x + (prOut.a - q.a) / 2; inner.z = p.z + (prOut.b - q.b) / 2; inner.y
= p.y; }
      else if (p.axis === "x"){ inner.x = p.x; inner.y = p.y + (prOut.a - q.a) / 2; inner.z = p.z +
(prOut.b - q.b) / 2; }
      else { inner.z = p.z; inner.x = p.x + (prOut.b - q.b) / 2; inner.y = p.y + (prOut.a - q.a) / 2; }
      d3.pieces.push(inner);
      d3.sel = inner.id;
      flag("Caño interior agregado (libre: movelo cuando quieras)");
    } else if (tel){
      const outer = d3OuterOf(p);
      if (!outer) return;
      const sol = Math.max(0, +d3.solape || 100);
      const A = d3AABB(outer);
      if (p.axis === "y") p.y = tel.dataset.tel === "extender" ? Math.max(0, A.y2 - sol) :
Math.max(0, A.y1 + 10);
      else if (p.axis === "x") p.x = tel.dataset.tel === "extender" ? A.x2 - sol : A.x1 + 10;
      else p.z = tel.dataset.tel === "extender" ? A.z2 - sol : A.z1 + 10;
    }
    d3SelRender(); d3TelRender(); d3OutRender(); d3Status(); d3StatsRender();
    d3Save(); d3Draw();
  });

  /* despiece */
  document.getElementById("d3ToCorte").addEventListener("click", d3SendCorte);
  document.getElementById("d3ToMat").addEventListener("click", d3SendMat);
  document.getElementById("d3ResetMarks").addEventListener("click", () => {
    d3.pieces.forEach(p => { p._sent = false; });
    d3Save(); d3OutRender();
    flag("Marcas borradas: todo listo para reenviar");
  });
```

```
  /* exportar / importar */
  document.getElementById("d3Exp").addEventListener("click", () => {
    const data = {
      v: 1, pieces: d3.pieces, cotas: d3.cotas, cat: d3.cat, ple: d3.ple,
      magnet: d3.magnet, capture: d3.capture, rotStep: d3.rotStep,
      holgura: d3.holgura, kerfBarra: d3.kerfBarra, solape: d3.solape
    };
    const blob = new Blob([JSON.stringify(data, null, 2)], { type: "application/json" });
    const a = document.createElement("a");
    a.href = URL.createObjectURL(blob);
    a.download = "diseno-3d.json";
    document.body.appendChild(a); a.click(); a.remove();
    setTimeout(() => URL.revokeObjectURL(a.href), 1000);
    flag("Diseño exportado");
  });
  document.getElementById("d3Imp").addEventListener("click", () =>
document.getElementById("d3ImpFile").click());
  document.getElementById("d3ImpFile").addEventListener("change", e => {
    const f = e.target.files && e.target.files[0];
    if (!f) return;
    const rd = new FileReader();
    rd.onload = () => {
      try{
        const s = JSON.parse(rd.result);
        if (!s || !Array.isArray(s.pieces)) throw new Error("formato");
        d3.pieces = s.pieces.filter(p => p && p.id && p.kind);
        d3.cotas = Array.isArray(s.cotas) ? s.cotas : [];
        if (s.cat && typeof s.cat === "object") d3.cat = s.cat;
        if (s.ple) d3.ple = Object.assign(d3.ple, s.ple);
        d3.sel = null;
        d3Fit(); d3Save();
        d3SelRender(); d3TelRender(); d3OutRender(); d3Status(); d3StatsRender(); d3Draw();
        flag("Diseño importado");
      }catch(err){ flag("No se pudo importar: no es un diseño válido"); }
    };
    rd.readAsText(f);
    e.target.value = "";
  });
}

function d3ImportFromAI(){
  try{
    const d=JSON.parse(document.getElementById("d3AiJson").value);
```

```
    d3.pieces=d.pieces.map(p=>Object.assign({id:uid(),rot:0,axis:"x",mod:"base",_sent:false},p));
    if(d.name){ mat.name=d.name; document.getElementById("mName").value=d.name; }
    d3SendCorte(); d3SendMat();
    if(d.consumables) d.consumables.forEach(c=>mat.items.push(sanitizeMatItem(c)));
    saveMat(); renderMat(); d3Fit(); d3Save(); d3SelRender(); d3OutRender(); d3Status();
d3Draw();
    flag("Mueble cargado y costeado");
  }catch(e){ flag("Error en JSON"); }
}
document.getElementById("d3BtnLoadAi")?.addEventListener("click", d3ImportFromAI);

/* === FIN D3-4 === */

/*
==========================================================================
=
   9. ARRANQUE

==========================================================================
= */
load();
fillForm();
bindForm();
bindList();
loadMat();
loadBook();
fillMatForm();
bindMat();
bindBook();
renderMat();
renderBook();
calcHistLoad();
calcHistRender();
calcRender();
bindCalc();
document.getElementById("brandLogo").src = LOGO;
qLoad();
qFillForm();
bindQuote();
renderQBook();
d3Init();

try{ activeTab = localStorage.getItem(TABKEY) || "corte"; }catch(e){ activeTab = "corte"; }
```

```
applyTab(activeTab);

TABS.forEach(([btn, name]) => document.getElementById(btn).addEventListener("click", () =>
setTab(name)));
document.getElementById("btnDemo").addEventListener("click", () => activeTab === "d3" ?
loadDemo3D() : activeTab === "mat" ? loadMatDemo() : loadDemo());
document.getElementById("btnReset").addEventListener("click", () => activeTab === "d3" ?
resetD3() : activeTab === "mat" ? resetMat() : reset());
document.getElementById("btnExport").addEventListener("click", exportJob);
document.getElementById("btnPrint").addEventListener("click", () => window.print());
render();
</script>

<footer class="cred noprint">
  <div class="cred-inner">
    <span>Calculadora de taller · Plano de corte, cotizador y presupuestos</span>
    <span>Autor: <a href="https://github.com/vilchezh" target="_blank" rel="noopener
noreferrer">Acero y Escamas (vilchezh)</a> · <a
href="https://github.com/vilchezh/Calculadora-de-taller" target="_blank" rel="noopener
noreferrer">GitHub</a></span>
  </div>
</footer>
</body>
</html>
```